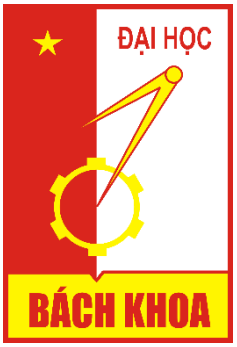
ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

----- □ □ □ -----



BÁO CÁO MÔN HỌC

Thiết kế hệ thống quản lý nhà kính trồng hoa

Giảng viên hướng dẫn: **ThS.Vũ Song Tùng**

Sinh viên thực hiện: **Nguyễn Minh Đức - 202414039**

Dương Anh Hào - 202414100

Bùi Ngọc Bảo Minh - 202414256

Hà Nội, năm 2026

MỞ ĐẦU

Trong bối cảnh cuộc cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, việc ứng dụng công nghệ thông tin vào sản xuất nông nghiệp ngày càng trở nên cần thiết nhằm nâng cao năng suất, chất lượng sản phẩm và tối ưu hóa quá trình quản lý. Đặc biệt, mô hình nhà kính trồng hoa đang được áp dụng rộng rãi nhờ khả năng kiểm soát các yếu tố môi trường như nhiệt độ, độ ẩm, ánh sáng và dinh dưỡng, góp phần tạo điều kiện thuận lợi cho cây trồng phát triển ổn định, giảm thiểu ảnh hưởng của thời tiết và nâng cao hiệu quả kinh tế.

Tuy nhiên, việc quản lý nhà kính theo phương pháp thủ công thường gặp nhiều khó khăn như tốn thời gian theo dõi, dễ xảy ra sai sót trong quá trình ghi nhận dữ liệu, khó kiểm soát đồng thời nhiều khu vực trồng trọt và thiếu khả năng thống kê, phân tích để hỗ trợ ra quyết định. Vì vậy, việc xây dựng một hệ thống quản lý nhà kính trồng hoa là nhu cầu thực tiễn, giúp tự động hóa công tác quản lý, giám sát và lưu trữ dữ liệu, từ đó nâng cao hiệu quả vận hành và giảm chi phí nhân lực.

Xuất phát từ những yêu cầu trên, nhóm chúng em lựa chọn đề tài "Thiết kế hệ thống quản lý nhà kính trồng hoa" cho bài tập lớn môn Kỹ thuật phần mềm ứng dụng. Mục tiêu của đề tài là nghiên cứu, phân tích và thiết kế một hệ thống phần mềm hỗ trợ quản lý toàn diện các hoạt động trong nhà kính như quản lý giống hoa, khu vực trồng, điều kiện môi trường, lịch chăm sóc, nhân viên vận hành và báo cáo thống kê. Thông qua quá trình thực hiện, nhóm vận dụng các kiến thức về phân tích yêu cầu, thiết kế chức năng, xây dựng cơ sở dữ liệu và phát triển ứng dụng phần mềm để giải quyết một bài toán thực tế trong lĩnh vực nông nghiệp công nghệ cao.

Mặc dù đã cố gắng nghiên cứu và hoàn thiện đề tài trong phạm vi kiến thức và thời gian cho phép, báo cáo chắc chắn vẫn còn những hạn chế và thiếu sót nhất định. Nhóm chúng em rất mong nhận được những ý kiến đóng góp quý báu từ thầy và các bạn để đề tài được hoàn thiện hơn.

Cuối cùng, nhóm xin chân thành cảm ơn thầy đã tận tình hỗ trợ, cung cấp kiến thức và định hướng trong suốt quá trình thực hiện bài tập lớn.

MỤC LỤC

MỞ ĐẦU **1**
MỤC LỤC **2**
DANH MỤC HÌNH ẢNH **4**
DANH MỤC BẢNG BIỂU **6**
DANH MỤC TỪ VIẾT TẮT **9**
CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG **10**
1.1. Đặt vấn đề 10
1.2. Mục tiêu 10
1.2.1. Mục tiêu tổng quát 10
1.2.2. Mục tiêu cụ thể 10
1.3. Phạm vi nghiên cứu 11
1.3.1. Phạm vi chức năng 11
1.3.2. Phạm vi công nghệ 11
1.4. Ngôn ngữ lập trình & Công cụ thiết kế 11
1.4.1. Ngôn ngữ lập trình 11
1.4.2. Công cụ thiết kế 11
1.5. Yêu cầu của đề tài 12
1.5.1. Yêu cầu chức năng 12
1.5.2. Yêu cầu phi chức năng 13
1.6. Các thông số môi trường 13
1.7. Kết luận chương 1 14
CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG **15**
2.1. Phân tích hệ thống 15
2.1.1. Sơ đồ tình huống sử dụng (Use Case Diagram) 15
2.1.2. Sơ đồ hoạt động (Activity Diagram) 17
2.1.3. Sơ đồ tuần tự (Sequence Diagram) 25
2.1.4. Sơ đồ luồng dữ liệu (DFD) 27
2.1.5. Sơ đồ phân cấp chức năng (FHD) 36
2.2. Thiết kế hệ thống 36
2.2.1. Kiến trúc tổng thể hệ thống 36
2.2.2. Sơ đồ quan hệ thực thể (ERD) 38
2.3. Thiết kế lớp hệ thống 48
2.3.1. Cấu trúc lớp Sensor 48
2.3.2. Cấu trúc lớp Data 50
2.3.3. Cấu trúc lớp Server & Database 52
2.3.4. Cấu trúc lớp Application (Interface) 53
2.4. Thiết kế giao tiếp hệ thống 55
2.5. Kết luận chương 2 62
CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG **63**
3.1. Công nghệ và môi trường triển khai 63

3.1.1. Hiện thực hóa cấu trúc bảng SQL (DDL Tables)....63
3.1.2. Hiện thực hóa các khung nhìn (Database Views)....69
3.1.3. Cài đặt các thủ tục lưu trữ (Stored Procedures Implementation....71
3.2. Một số mã nguồn triển khai....75
3.3. API và giao tiếp MQTT....78
3.4. Kết quả triển khai và chạy thử....79
3.5. Kiểm thử hệ thống....84
3.5.1. Kiểm thử xác thực và phân quyền....84
3.5.2. Kiểm thử quản lý dữ liệu....85
3.5.3. Kiểm thử thu thập và xử lý dữ liệu cảm biến....85
3.5.4. Kiểm thử điều khiển tự động và cảnh báo....85
3.5.5. Kiểm thử ghi nhật ký hệ thống....86
3.5.6. Đánh giá chung kết quả kiểm thử....86
3.6. Kết luận chương 3....86
KẾT LUẬN....87
Kết quả đạt được....87
Ưu điểm của hệ thống....87
Hạn chế của hệ thống....87
Hướng phát triển trong tương lai....87
Tổng kết chung....88
TÀI LIỆU THAM KHẢO....89

DANH MỤC HÌNH ẢNH

Hình 2.1. Use case diagram của Chủ trang trại.....14
Hình 2.2. Use case diagram của Kỹ thuật viên.....15
Hình 2.3. Use case diagram của Nhân viên vận hành.....15
Hình 2.4. Use case diagram của Hệ thống tự hành.....16
Hình 2.5. Sơ đồ hoạt động chức năng đăng nhập và đăng xuất hệ thống.....16
Hình 2.6. Sơ đồ hoạt động chức năng đăng ký và phê duyệt tài khoản.....17
Hình 2.7. Sơ đồ hoạt động chức năng quản lý cấu trúc Farm – Greenhouse – Zone.....18
Hình 2.8. Sơ đồ hoạt động chức năng quản lý công thức sinh trưởng và ngưỡng môi trường 19
Hình 2.9. Sơ đồ hoạt động chức năng quản lý thiết bị cảm biến và thiết bị đầu ra 20
Hình 2.10. Sơ đồ hoạt động chức năng nhận và xử lý dữ liệu cảm biến MQTT.....21
Hình 2.11. Sơ đồ hoạt động chức năng điều khiển thiết bị tự động.....21
Hình 2.12. Sơ đồ hoạt động chức năng cảnh báo vượt ngưỡng.....22
Hình 2.13. Sơ đồ hoạt động chức năng điều khiển thiết bị thủ công 23
Hình 2.14. Sơ đồ hoạt động chức năng theo dõi Dashboard, lịch sử và Audit Log.....23
Hình 2.15. Sơ đồ tuần tự tiếp nhận và xử lý dữ liệu cảm biến.....24
Hình 2.16. Sơ đồ tuần tự xử lý cảnh báo và điều khiển thiết bị.....24
Hình 2.17. Sơ đồ tuần tự điều khiển thiết bị tự động.....25
Hình 2.18. Sơ đồ tuần tự quản lý công thức sinh trưởng.....25
Hình 2.19. Sơ đồ luồng dữ liệu mức ngữ cảnh.....26
Hình 2.20. Sơ đồ DFD User Management.....27
Hình 2.21. Sơ đồ DFD Farm Management.....28
Hình 2.22. Sơ đồ DFD IoT Management.....29
Hình 2.23. Sơ đồ DFD Growth Management.....30
Hình 2.24. Sơ đồ DFD Data Collection.....31
Hình 2.25. Sơ đồ DFD Monitoring.....31
Hình 2.26. Sơ đồ DFD Reporting.....32
Hình 2.27. Sơ đồ DFD mức 2 - Quản lý sinh trưởng.....33
Hình 2.28. Sơ đồ DFD mức 2 - Thu thập dữ liệu cảm biến.....34
Hình 2.29. Sơ đồ DFD mức 2 - Giám sát và cảnh báo.....34
Hình 2.30. Sơ đồ phân cấp chức năng.....35
Hình 2.31. Sơ đồ kiến trúc hệ thống.....36

Hình 2.32. Sơ đồ quan hệ thực thể tổng quát.....37
Hình 2.33. Sơ đồ quan hệ thực thể chi tiết.....38
Hình 2.34. Bảng thực thể User.....39
Hình 2.35. Bảng thực thể Farm.....39
Hình 2.36. Bảng thực thể Greenhouse.....40
Hình 2.37. Bảng thực thể Zone.....41
Hình 2.38. Bảng thực thể Gateway.....41
Hình 2.39. Bảng thực thể Device.....42
Hình 2.40. Bảng thực thể ControlProperties.....43
Hình 2.41. Bảng thực thể Recipe.....43
Hình 2.42. Bảng thực thể GrowthStage.....44
Hình 2.43. Bảng thực thể Threshold.....44
Hình 2.44. Bảng thực thể SensorData.....45
Hình 2.45. Bảng thực thể Log.....45
Hình 2.46. Bảng thực thể AlertLog.....46
Hình 2.47. UML lớp Sensor.....48
Hình 2.48. UML lớp Data.....50
Hình 2.49. UML lớp Server & Database.....51
Hình 2.50. UML lớp Application.....53
Hình 2.51. Luồng dữ liệu cảm biến hiện tại.....59
Hình 2.52. Luồng dữ liệu cảm biến sau khi tích hợp MQTT.....60
Hình 3.1. Giao diện đăng nhập hệ thống.....78
Hình 3.2. Màn hình Dashboard theo dõi các chỉ số môi trường.....79
Hình 3.3. Màn hình Quản lý danh sách thiết bị IoT.....79
Hình 3.4. Giao diện Bảng điều khiển thiết bị chấp hành.....80
Hình 3.5. Giao diện Quản lý phân vùng trồng.....80
Hình 3.6. Màn hình Cấu hình chu kỳ sinh trưởng & ngưỡng giới hạn.....81
Hình 3.7. Trung tâm quản lý và leo thang cảnh báo.....81
Hình 3.8. Màn hình Nhật ký hoạt động Audit Log.....82
Hình 3.9. Màn hình Terminal kiểm tra kết nối MQTT Broker & Simulator Backend.....82

DANH MỤC BẢNG BIỂU

Bảng 1.1. Các công cụ được sử dụng trong quá trình thiết kế.....10
Bảng 1.2. Các yêu cầu phi chức năng.....12
Bảng 1.3. Các thông số môi trường.....12
Bảng 2.1. Các tác nhân ngoài.....26
Bảng 2.2. Các kho dữ liệu.....27
Bảng 2.3. Các tiến trình.....27
Bảng 2.4. Thuộc tính lớp Sensor.....47
Bảng 2.5. Chức năng lớp Sensor.....48
Bảng 2.6. Mã hóa sensorType.....48
Bảng 2.7. Mã hóa Status lớp Sensor.....48
Bảng 2.8. Thuộc tính lớp Data.....50
Bảng 2.9. Chức năng lớp Data.....50
Bảng 2.10. Thuộc tính lớp Server & Database.....51
Bảng 2.11. Chức năng lớp Server & Database.....51
Bảng 2.12. Mã hóa Status lớp Sensor & Database.....51
Bảng 2.13. Thuộc tính lớp Application.....52
Bảng 2.14. Chức năng lớp Application.....53
Bảng 2.15. Mã hóa Status lớp Application.....53
Bảng 2.16. Xác thực và tài khoản.....54
Bảng 2.17. Quản lý người dùng.....55
Bảng 2.18. Quản lý cấu trúc nhà kính.....55
Bảng 2.19. Quản lý thiết bị và Gateway.....56
Bảng 2.20. Quản lý Recipe, GrowthStage và Threshold.....58
Bảng 2.21. Giám sát dữ liệu và cảnh báo.....58
Bảng 2.22. API phục vụ mô phỏng dữ liệu.....59
Bảng 2.23. Thiết kế MQTT Publish.....59
Bảng 2.24. Thiết kế MQTT Subscribe.....60
Bảng 3.1. Tổng hợp các công nghệ, phiên bản và vai trò cụ thể của từng thành phần.....62
Bảng 3.2. User (Tài khoản và phân quyền).....62
Bảng 3.3. Farm (Trang trại).....63
Bảng 3.4. Bảng Greenhouse (Nhà kính).....63
Bảng 3.5. Recipe (Công thức sinh trưởng).....63

Bảng 3.6. Zone (Khu vực trồng).....64
Bảng 3.7. Gateway (Trung tâm kết nối IoT).....64
Bảng 3.8. Device (Cảm biến & Thiết bị đầu ra).....64
Bảng 3.9. Bảng ControlProperties (Thuộc tính điều khiển chấp hành).....65
Bảng 3.10. GrowthStage (Giai đoạn sinh trưởng).....65
Bảng 3.11. Bảng Threshold (Ngưỡng môi trường theo giai đoạn).....66
Bảng 3.12. Bảng SensorData (Dữ liệu chuỗi thời gian).....66
Bảng 3.13. Bảng Log (Nhật ký kiểm toán hệ thống).....67
Bảng 3.14. Bảng AlertLog (Quản lý cảnh báo sự cố).....67
Bảng 3.15. ViewUserRole (Thông tin và vai trò người dùng).....68
Bảng 3.16. ViewFarmGreenhouse (Cấu trúc Trang trại & Nhà kính).....68
Bảng 3.17. ViewZoneOverview (Tổng quan khu vực trồng).....69
Bảng 3.18. ViewDeviceStatus (Tình trạng thiết bị & Gateway).....69
Bảng 3.19. ViewControlDevice (Điều khiển thiết bị chấp hành).....70
Bảng 3.20. ViewLatestSensorData (Dữ liệu cảm biến mới nhất).....70
Bảng 3.21. ViewThreshold (Tổng hợp ngưỡng sinh trưởng).....71
Bảng 3.22. ViewAlertHistory (Lịch sử cảnh báo sự cố).....71
Bảng 3.23. ViewSystemLog (Truy vết Audit Log).....72
Bảng 3.24. Thủ tục ghi nhận dữ liệu cảm biến (sp_RecordSensorReading).....72
Bảng 3.25. Thủ tục kiểm tra ngưỡng và kích hoạt cảnh báo (sp_EvaluateThresholdAndAlert) 73
Bảng 3.26. Thủ tục điều khiển thiết bị chấp hành và ghi log (sp_UpdateDeviceControl).....73
Bảng 3.27. Thủ tục xử lý xác nhận cảnh báo sự cố (sp_ResolveAlert) 74
Bảng 3.28. Mã nguồn nhận gói tin cảm biến và phát lệnh qua MQTT (mqtt-service.js).....74
Bảng 3.29. Mã nguồn đồng bộ AlertLog và đánh giá mức độ nghiêm trọng (simulation.js).....75
Bảng 3.30. Mã nguồn Endpoint cập nhật thuộc tính điều khiển và xác thực (server.js) 76
Bảng 3.31. Danh mục 10 REST API tiêu biểu trong hệ thống.....77
Bảng 3.32. Cấu trúc Topic và định dạng Payload MQTT.....78
Bảng 3.33. Kiểm thử xác thực và phân quyền.....83

Bảng 3.34. Kiểm thử quản lý dữ liệu.....84
Bảng 3.35. Kiểm thử thu thập và xử lý dữ liệu cảm biến.....84
Bảng 3.36. Kiểm thử điều khiển tự động và cảnh báo.....84
Bảng 3.37. Kiểm thử ghi nhật ký hệ thống.....85

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Từ gốc tiếng Anh	Ý nghĩa tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CSS	Cascading Style Sheets	Ngôn ngữ định kiểu theo tầng
DDL	Data Definition Language	Ngôn ngữ định nghĩa dữ liệu
DFD	Data Flow Diagram	Sơ đồ luồng dữ liệu
ERD	Entity-Relationship Diagram	Sơ đồ quan hệ thực thể
FHD	Function Hierarchy Diagram	Sơ đồ phân cấp chức năng
FK	Foreign Key	Khóa ngoại
GPS	Global Positioning System	Hệ thống định vị toàn cầu
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
IoT	Internet of Things	Mạng lưới vạn vật kết nối Internet
JS	JavaScript	Ngôn ngữ lập trình kịch bản
JSON	JavaScript Object Notation	Định dạng mô tả đối tượng dưới dạng văn bản
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông điệp theo mô hình Publish/Subscribe
PK	Primary Key	Khóa chính
REST	Representational State Transfer	Kiến trúc chuyển đổi trạng thái đại diện
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
TC	Test Case	Kịch bản kiểm thử
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất

CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG

Chương này trình bày tổng quan về hệ thống quản lý nhà kính trồng hoa, bao gồm mục tiêu xây dựng hệ thống, phạm vi nghiên cứu, các công nghệ được sử dụng và các yêu cầu chính của hệ thống. Đây là cơ sở để thực hiện quá trình phân tích và thiết kế phần mềm ở các chương tiếp theo.

1.1. Đặt vấn đề

Trong lĩnh vực nông nghiệp hiện đại, mô hình nhà kính đang được áp dụng rộng rãi nhằm nâng cao năng suất và chất lượng cây trồng. Đối với các loại hoa có giá trị kinh tế cao, việc duy trì các điều kiện môi trường phù hợp như nhiệt độ, độ ẩm, ánh sáng, nồng độ CO_2 và độ ẩm đất đóng vai trò quan trọng đối với quá trình sinh trưởng và phát triển.

Tuy nhiên, việc giám sát và quản lý các thông số môi trường bằng phương pháp thủ công thường gặp nhiều khó khăn như tốn thời gian, dễ xảy ra sai sót và không kịp thời phát hiện các bất thường. Bên cạnh đó, khi số lượng cảm biến và thiết bị trong nhà kính tăng lên, việc quản lý dữ liệu trở nên phức tạp hơn.

Để giải quyết vấn đề này, đề tài xây dựng một hệ thống quản lý nhà kính trồng hoa ứng dụng công nghệ Internet of Things (IoT). Hệ thống cho phép thu thập dữ liệu từ các cảm biến môi trường, lưu trữ dữ liệu trên máy chủ, giám sát tình trạng nhà kính theo thời gian thực và hỗ trợ người quản lý đưa ra các quyết định điều chỉnh phù hợp.

1.2. Mục tiêu

1.2.1. Mục tiêu tổng quát

Xây dựng hệ thống quản lý nhà kính trồng hoa ứng dụng công nghệ IoT nhằm tự động thu thập, lưu trữ và giám sát các thông số môi trường trong nhà kính, hỗ trợ người quản lý theo dõi tình trạng cây trồng và đưa ra các quyết định điều chỉnh kịp thời.

1.2.2. Mục tiêu cụ thể

Hệ thống quản lý nhà kính trồng hoa được xây dựng với các mục tiêu cụ thể sau:

- Quản lý thông tin cảm biến, các thiết bị điều khiển, thông tin cây trồng và khu vực trồng trong nhà kính.
- Thu thập và lưu trữ dữ liệu môi trường theo thời gian thực.
- Giám sát các thông số ảnh hưởng đến sự phát triển của cây trồng.
- Cảnh báo khi các thông số môi trường vượt ngưỡng cho phép.
- Hỗ trợ thống kê, tra cứu và theo dõi lịch sử dữ liệu.

1.3. Phạm vi nghiên cứu

1.3.1. Phạm vi chức năng

Hệ thống hỗ trợ:

- Quản lý người dùng, cảm biến và thiết bị IoT, cây trồng và công thức sinh trưởng.
- Thu thập và lưu trữ dữ liệu môi trường.
- Theo dõi các thông số môi trường trong nhà kính.
- Cảnh báo sự cố và bất thường.
- Thống kê và tra cứu dữ liệu lịch sử.

1.3.2. Phạm vi công nghệ

Hệ thống phần mềm quản lý kết hợp với mô hình IoT, sử dụng cảm biến môi trường, bộ điều khiển IoT, cơ sở dữ liệu và giao diện quản lý.

1.4. Ngôn ngữ lập trình & Công cụ thiết kế

1.4.1. Ngôn ngữ lập trình

Ngôn ngữ lập trình chính được sử dụng trong hệ thống là **JavaScript.** JavaScript được lựa chọn do khả năng phát triển ứng dụng toàn diện trên cả phía giao diện người dùng (Frontend) và máy chủ (Backend). Trong đề tài này, JavaScript được sử dụng kết hợp với nền tảng Node.js và framework Express.js để xây dựng hệ thống quản lý nhà kính trồng hoa.

Việc lựa chọn JavaScript mang lại các lợi ích sau:

- Phát triển được cả Frontend và Backend.
- Node.js hỗ trợ xử lý bất đồng bộ, phù hợp với việc tiếp nhận dữ liệu liên tục.
- Có khả năng tích hợp REST API, MQTT và cơ sở dữ liệu.
- Hệ sinh thái thư viện phong phú và thuận lợi mở rộng.

Với những ưu điểm trên, JavaScript kết hợp Node.js là lựa chọn phù hợp cho hệ thống quản lý nhà kính trồng hoa ứng dụng công nghệ IoT.

1.4.2. Công cụ thiết kế

Bảng 1.1. Các công cụ được sử dụng trong quá trình thiết kế

Công cụ	Mục đích sử dụng
SQL Server	Lưu trữ dữ liệu hệ thống
SQL Server Management Studio (SSMS)	Thiết kế và quản trị cơ sở dữ liệu
StarUML	Thiết kế UML, Class Diagram
Draw.io	Thiết kế sơ đồ kiến trúc và ERD

GitHub	Quản lý mã nguồn và phiên bản
Visual Studio Code	Phát triển Frontend và Backend
Node.js	Môi trường thực thi Backend

Các công cụ được lựa chọn vì khả năng **tích hợp tốt với hệ sinh thái Node.js và JavaScript**, hỗ trợ hiệu quả cho việc phân tích, thiết kế, lập trình, quản lý cơ sở dữ liệu và quản lý phiên bản phần mềm trong suốt quá trình phát triển hệ thống.

1.5. Yêu cầu của đề tài

1.5.1. Yêu cầu chức năng

Hệ thống được xây dựng nhằm đáp ứng các yêu cầu chức năng sau:

a) Quản lý thiết bị IoT

- Hỗ trợ thêm mới cảm biến và thiết bị khi mở rộng quy mô nhà kính.
- Theo dõi trạng thái hoạt động của từng cảm biến.
- Quản lý thông tin thiết bị trong hệ thống.

b) Trực quan hóa dữ liệu

- Thu thập dữ liệu từ các cảm biến môi trường.
- Hiển thị dữ liệu dưới dạng biểu đồ và báo cáo trực quan.
- Hỗ trợ tra cứu lịch sử dữ liệu.

c) Điều khiển và tự động hóa hệ thống

- Tự động điều khiển thiết bị dựa trên dữ liệu cảm biến.
- Cho phép quản trị viên thực hiện điều khiển thủ công khi cần thiết.

d) Theo dõi sinh trưởng và điều chỉnh tham số

- Xây dựng công thức sinh trưởng cho từng loại hoa.
- Hỗ trợ điều chỉnh các tham số môi trường theo từng giai đoạn phát triển.
- Cho phép kỹ sư nông nghiệp điều chỉnh chu kỳ sinh trưởng khi cần thiết.

e) Phân vùng quản lý

- Chia nhà kính thành nhiều khu vực quản lý.
- Hỗ trợ thêm mới khu vực và giàn trồng.
- Áp dụng công thức sinh trưởng cho từng khu vực.

f) Chức năng cảnh báo sự cố

- Phát hiện các thông số vượt ngưỡng cho phép.
- Gửi cảnh báo khi sự cố kéo dài vượt thời gian quy định.
- Gửi cảnh báo thứ cấp nếu không nhận được phản hồi.

g) Phân quyền và nhật ký hệ thống

- Phân quyền người dùng theo vai trò.
- Ghi nhận lịch sử thao tác của người dùng.
- Lưu nhật ký lỗi và sự cố của hệ thống.

1.5.2. Yêu cầu phi chức năng

Ngoài các chức năng nghiệp vụ, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo hiệu năng, độ tin cậy và khả năng mở rộng. Các yêu cầu này tập trung vào hiệu năng, bảo mật, khả năng mở rộng, tính tương thích và trải nghiệm người dùng.

Bảng 1.2. Các yêu cầu phi chức năng

STT	Yêu cầu	Mô tả
1	Khả năng xử lý tại biên	Hỗ trợ xử lý dữ liệu tại thiết bị hoặc gateway trước khi gửi lên máy chủ
2	Độ trễ thấp	Dữ liệu được cập nhật gần với thời gian thực
3	Bảo mật	Đảm bảo an toàn dữ liệu và quyền truy cập
4	Khả năng mở rộng	Dễ dàng bổ sung cảm biến, thiết bị và khu vực mới
5	Tính tương thích	Hỗ trợ nhiều loại cảm biến và thiết bị IoT
6	Tính thân thiện	Giao diện đơn giản, dễ sử dụng

1.6. Các thông số môi trường

Để đảm bảo cây hoa sinh trưởng và phát triển trong điều kiện tối ưu, hệ thống tiến hành theo dõi các thông số môi trường quan trọng trong nhà kính. Các thông số được lựa chọn dựa trên mức độ ảnh hưởng trực tiếp đến quá trình quang hợp, hấp thụ dinh dưỡng, trao đổi chất và khả năng sinh trưởng của cây trồng trong môi trường nhà kính.

Bảng 1.3. Các thông số môi trường

STT	Thông số	Ý nghĩa
1	Nhiệt độ	Ảnh hưởng trực tiếp đến quá trình trao đổi chất, hô hấp và sinh trưởng của cây. Nhiệt độ quá cao hoặc quá thấp đều có thể làm giảm năng suất và chất lượng hoa.
2	Độ ẩm không khí	Ảnh hưởng đến quá trình thoát hơi nước, hấp thụ dinh dưỡng và nguy cơ phát sinh bệnh hại.
3	Cường độ ánh sáng	Là nguồn năng lượng cho quá trình quang hợp, quyết định tốc độ sinh trưởng và phát triển của cây.

4	Độ ẩm đất	Cho biết lượng nước sẵn có trong vùng rễ, là cơ sở để điều khiển tưới tiêu tự động.
5	Độ pH	Quyết định khả năng hấp thụ dinh dưỡng của cây. pH không phù hợp có thể làm giảm hiệu quả sử dụng phân bón.
6	Nồng độ CO_2	CO_2 là nguyên liệu của quá trình quang hợp. Nồng độ CO_2 phù hợp giúp tăng khả năng sinh trưởng và năng suất cây trồng.

1.7. Kết luận chương 1

Chương 1 đã trình bày tổng quan về hệ thống quản lý nhà kính trồng hoa ứng dụng công nghệ IoT, bao gồm vấn đề đặt ra, mục tiêu, phạm vi nghiên cứu, các công nghệ sử dụng và yêu cầu chức năng, cũng như phi chức năng của hệ thống. Chương cũng đã xác định các thông số môi trường cần giám sát và các chức năng chính. Những nội dung này là cơ sở để thực hiện quá trình phân tích và thiết kế hệ thống trong các chương tiếp theo.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích và thiết kế hệ thống quản lý nhà kính trồng hoa dựa trên các yêu cầu đã xác định ở Chương 1. Nội dung chương tập trung phân tích các chức năng và luồng xử lý của hệ thống thông qua các sơ đồ Use Case, Activity, Sequence, DFD và FHD; đồng thời thiết kế kiến trúc tổng thể, cơ sở dữ liệu, các lớp của hệ thống và giao tiếp giữa các thành phần. Các kết quả phân tích và thiết kế là cơ sở để triển khai hệ thống trong chương tiếp theo.

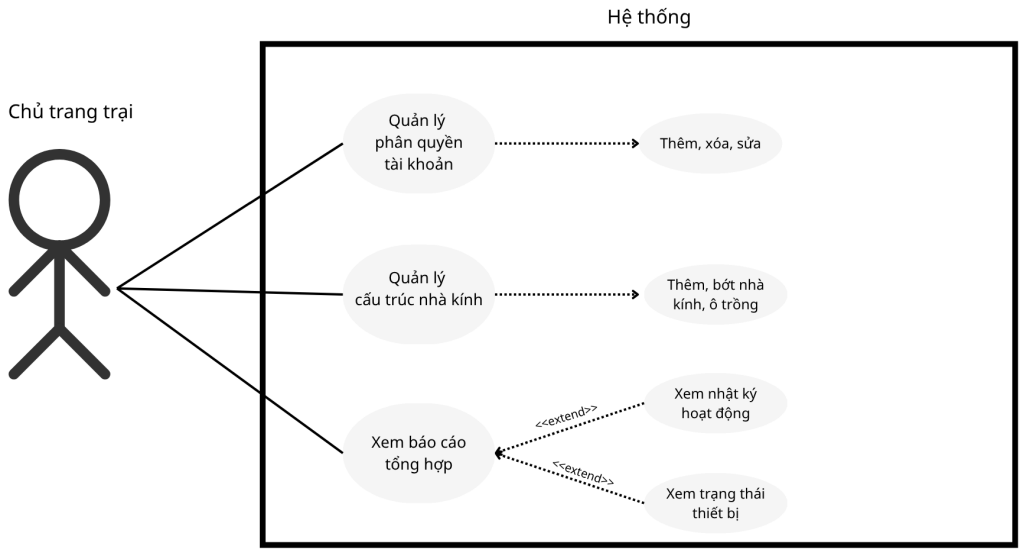
2.1. Phân tích hệ thống

2.1.1. Sơ đồ tình huống sử dụng (Use Case Diagram)

Có 4 nhóm đối tượng chính tương tác với hệ thống mà phần mềm hướng đến:

a) Chủ trang trại

Người quản lý mọi hoạt động liên quan đến nhà kính. Chủ trang trại có thể quản lý việc đăng nhập và phân quyền tài khoản của các đối tượng sử dụng khác, đồng thời có quyền thay đổi cấu trúc nhà kính trên hệ thống, thêm bớt hay sửa đổi số lượng nhà kính, khu vực trồng trọt tùy vào quy mô của trang trại. Ngoài ra, chủ trang trại còn có thể xem thêm báo cáo về tình trạng các thiết bị, hay nhật ký hoạt động của hệ thống,...

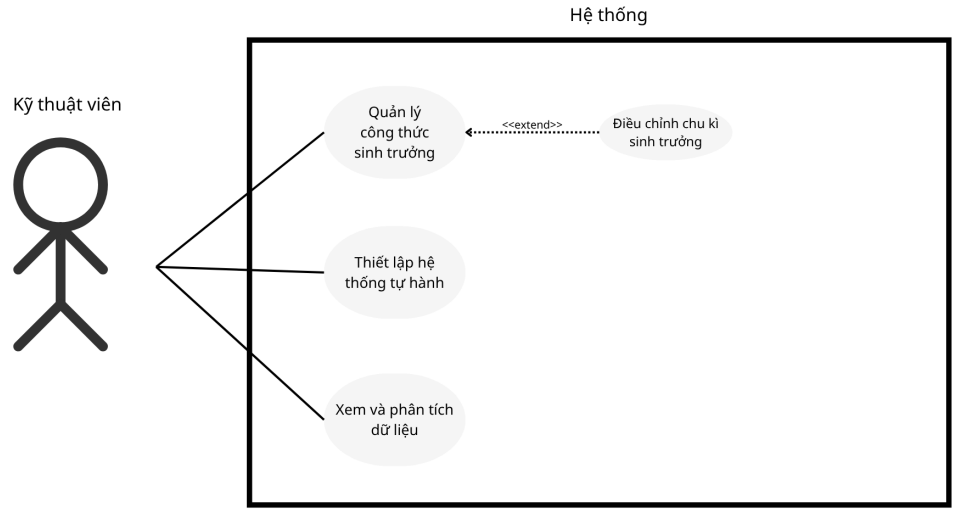


Hình 2.1. Use case diagram của Chủ trang trại

b) Kỹ thuật viên

Người có hiểu biết chuyên sâu trong lĩnh vực, đóng vai trò quan trọng trong việc thiết lập hệ thống và phân tích dữ liệu. Kỹ thuật viên có nhiệm vụ thiết lập công thức sinh trưởng

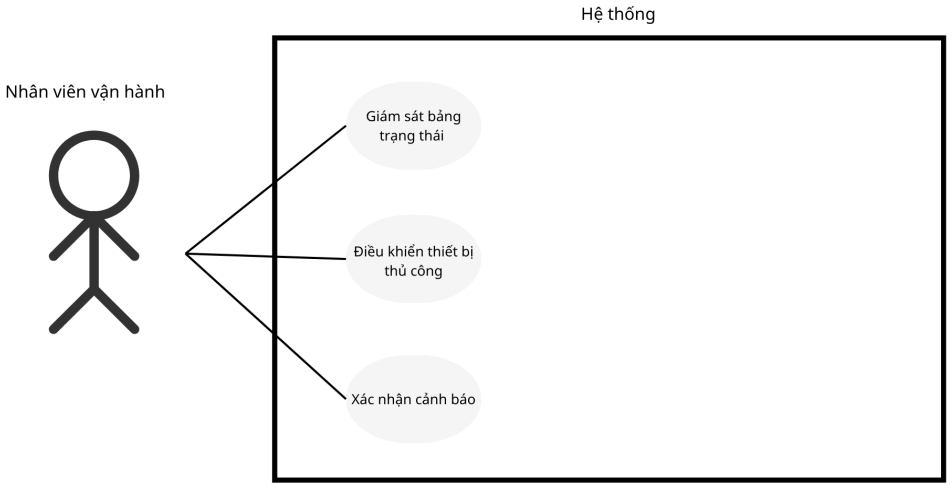
cho từng loại hoa, tạo ra những ràng buộc logic để hệ thống có thể vận hành tự động, xem dữ liệu tổng hợp được để điều chỉnh hệ thống và can thiệp vào chu kỳ sinh trưởng khi cần thiết.



Hình 2.2. Use case diagram của Kỹ thuật viên

c) Nhân viên vận hành

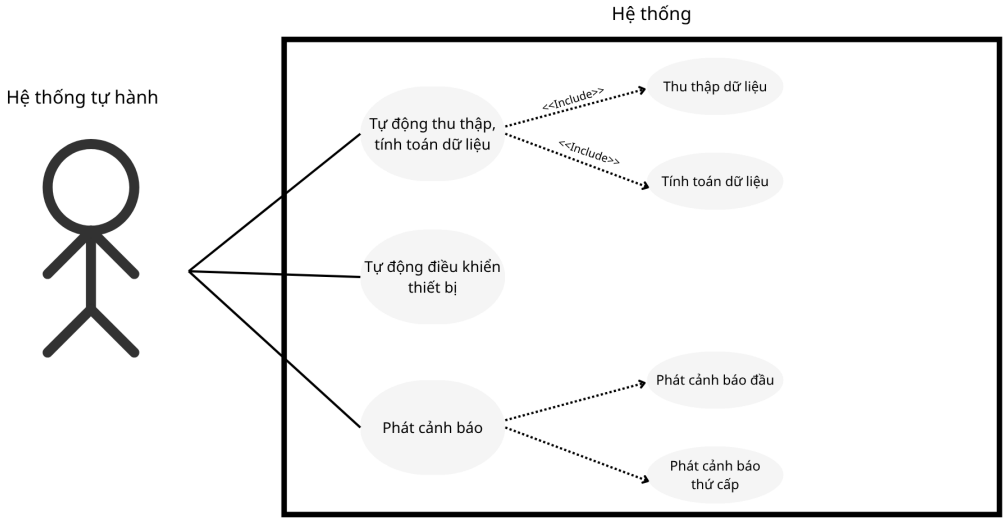
Người thường xuyên làm việc trực tiếp tại nhà kính, đảm bảo hệ thống vận hành trơn tru. Ngoài các công việc thông thường ra thì trong một nhà kính hiện đại, các nhân viên còn có nhiệm vụ trực tiếp giám sát quá trình hoạt động của hệ thống, nếu hệ thống báo lỗi, nhân viên thuộc khu vực đó sẽ nhận được thông báo và phải xác nhận, đồng thời trong trường hợp hệ thống tự hành không hoạt động đúng, nhân viên có thể thao tác thủ công trên phần mềm để kích hoạt thiết bị.



Hình 2.3. Use case diagram của Nhân viên vận hành

d) Hệ thống tự hành

Hệ thống có thể hoạt động tự động dựa trên những chỉ lệnh được hướng dẫn. Tự động thu thập dữ liệu từ môi trường, tự động tổng hợp dữ liệu, tự động điều khiển thiết bị, phát cảnh báo khi chỉ số môi trường vượt ngưỡng cho phép,...

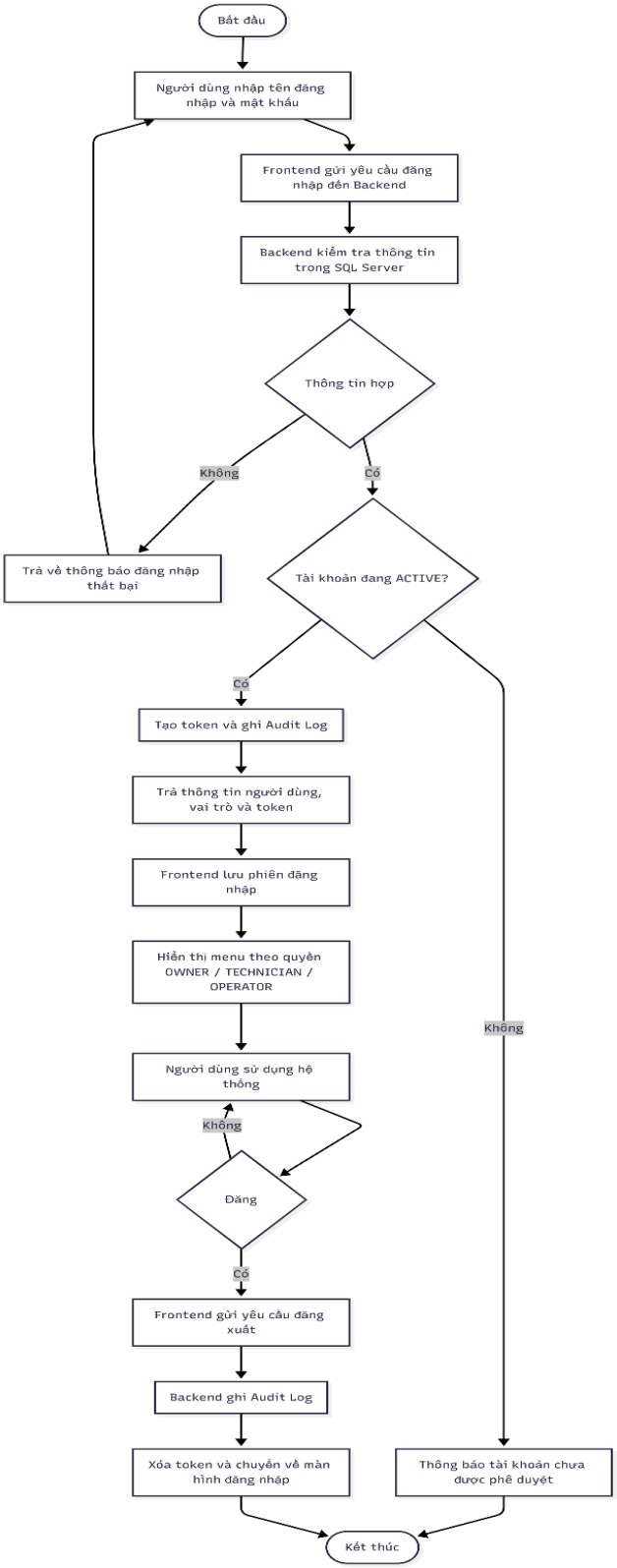


Hình 2.4. Use case diagram của Hệ thống tự hành

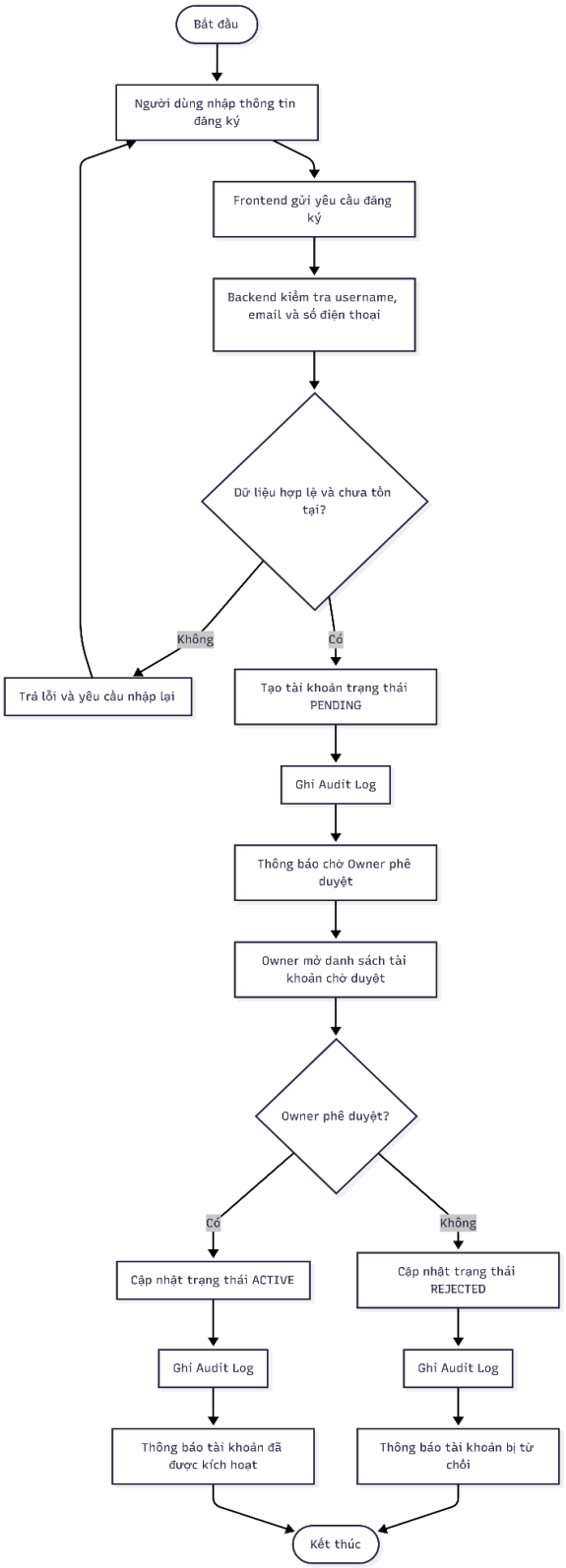
2.1.2. Sơ đồ hoạt động (Activity Diagram)

Sơ đồ hoạt động được sử dụng để mô tả trình tự xử lý của các chức năng chính trong hệ thống, từ khi người dùng hoặc thiết bị thực hiện một hành động cho đến khi hệ thống hoàn thành và trả về kết quả. Trên cơ sở các yêu cầu đã phân tích, nhóm xây dựng các sơ đồ hoạt động cho những nghiệp vụ chính của hệ thống như sau:

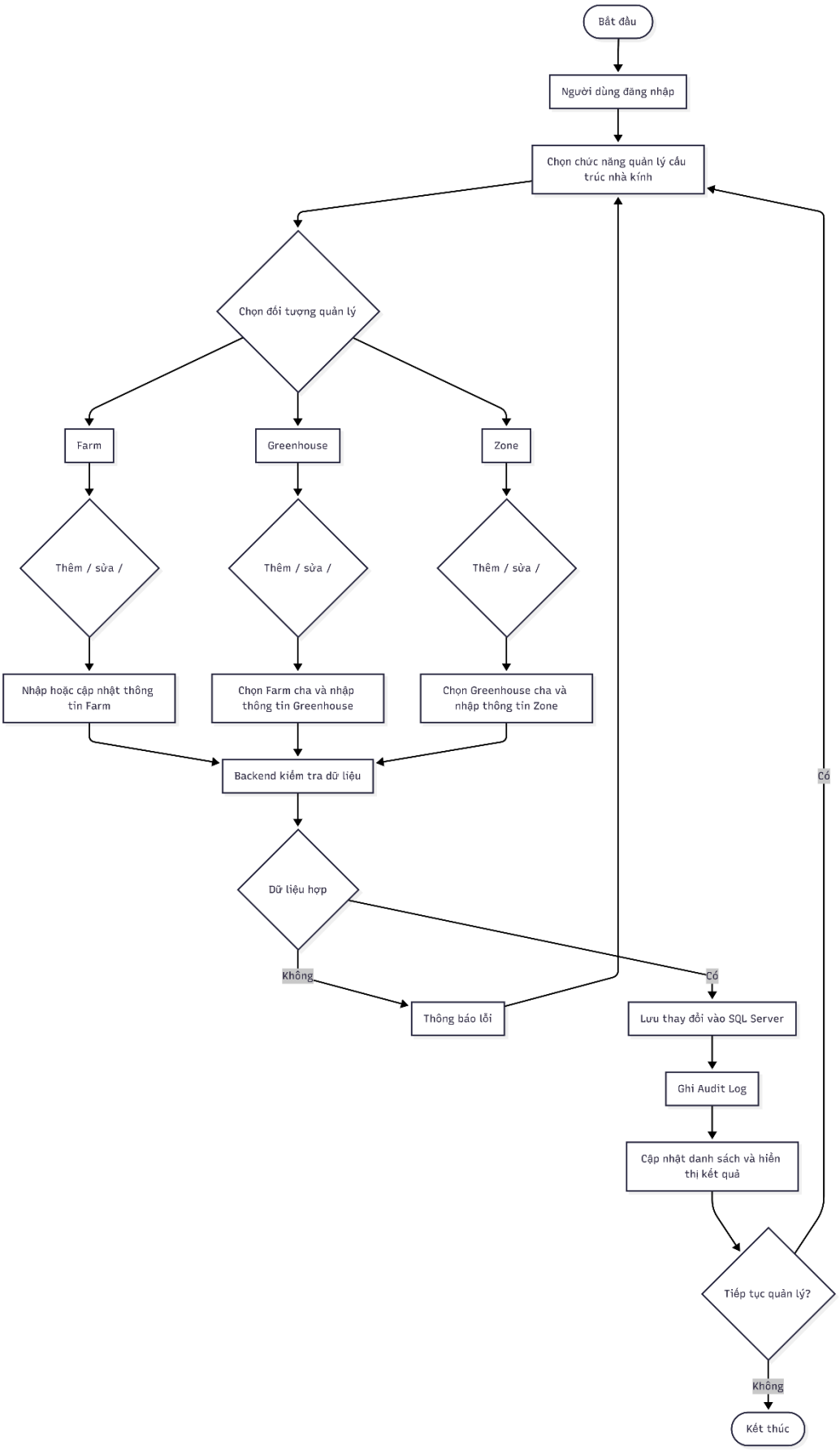
- Đăng nhập và đăng xuất hệ thống
- Đăng ký và phê duyệt tài khoản.
- Quản lý cấu trúc Farm – Greenhouse – Zone.
- Quản lý công thức sinh trưởng và ngưỡng môi trường.
- Quản lý thiết bị cảm biến và thiết bị đầu ra.
- Tiếp nhận và xử lý dữ liệu cảm biến thông qua MQTT
- Điều khiển thiết bị tự động
- Xử lý cảnh báo khi thông số môi trường vượt ngưỡng.
- Điều khiển thiết bị thủ công.
- Theo dõi Dashboard, lịch sử dữ liệu và Audit Log.



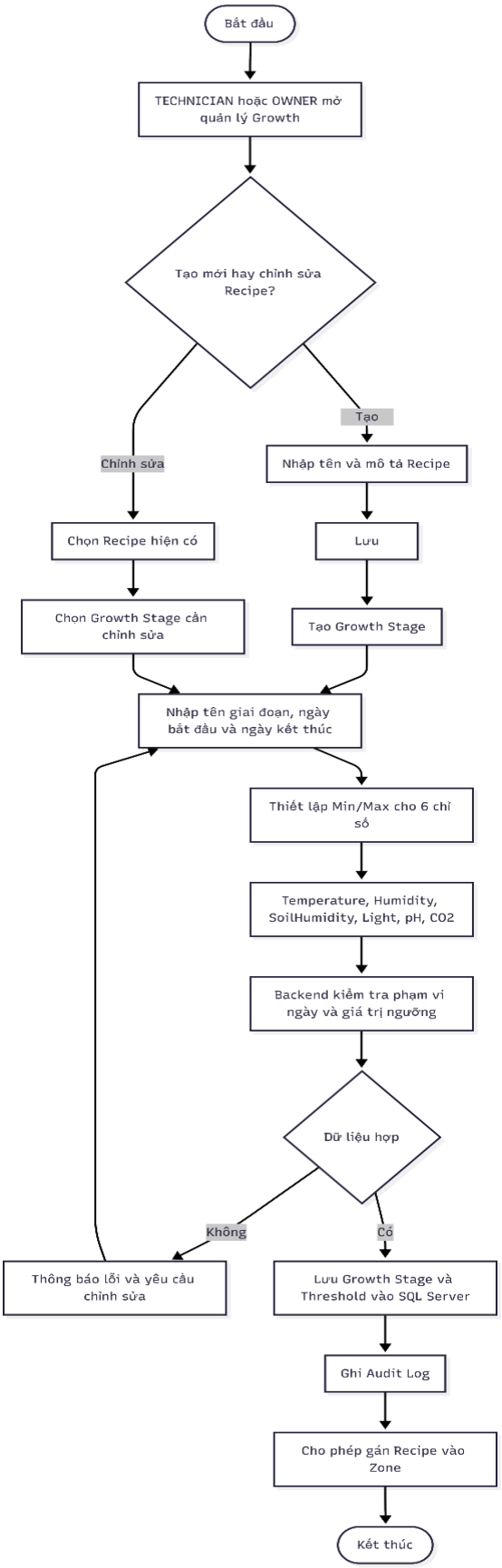
Hình 2.5. Sơ đồ hoạt động chức năng đăng nhập và đăng xuất hệ thống



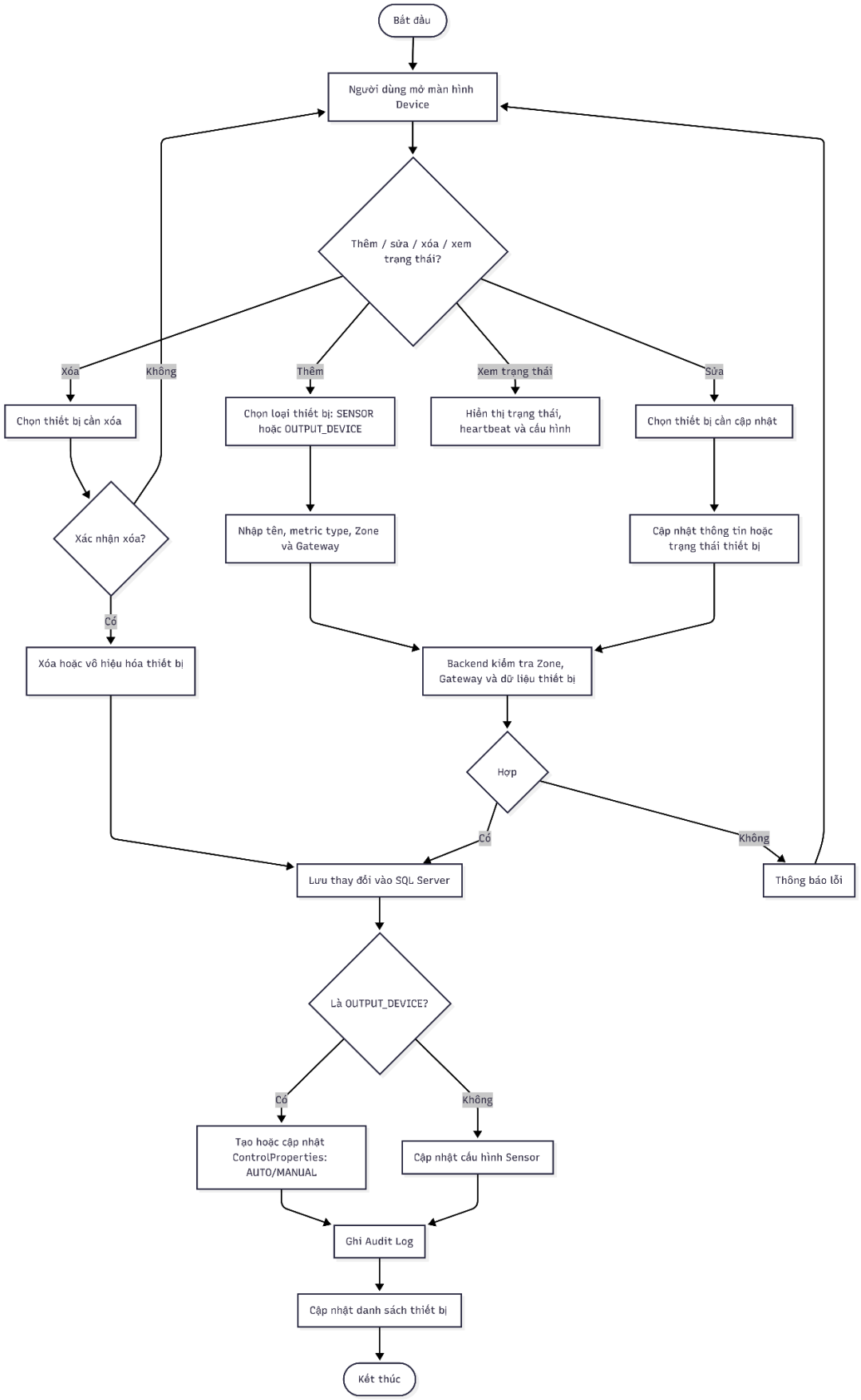
Hình 2.6. Sơ đồ hoạt động chức năng đăng ký và phê duyệt tài khoản



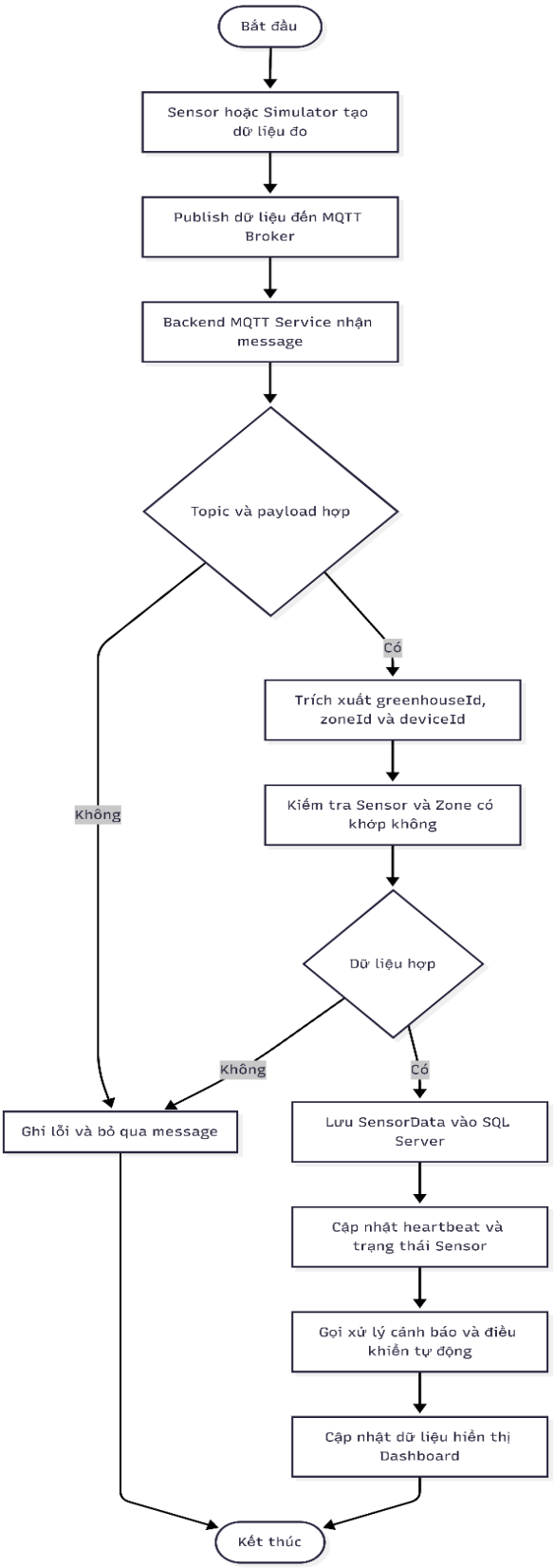
Hình 2.7. Sơ đồ hoạt động chức năng quản lý cấu trúc Farm – Greenhouse – Zone



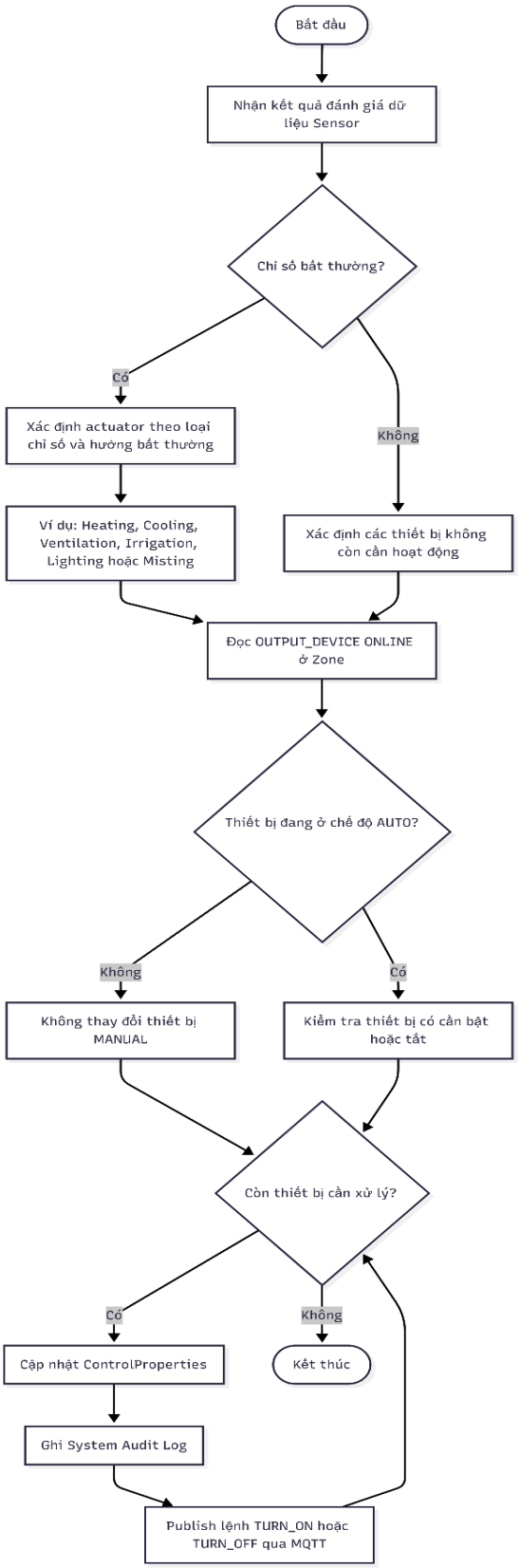
Hình 2.8. Sơ đồ hoạt động chức năng quản lý công thức sinh trưởng và ngưỡng môi trường



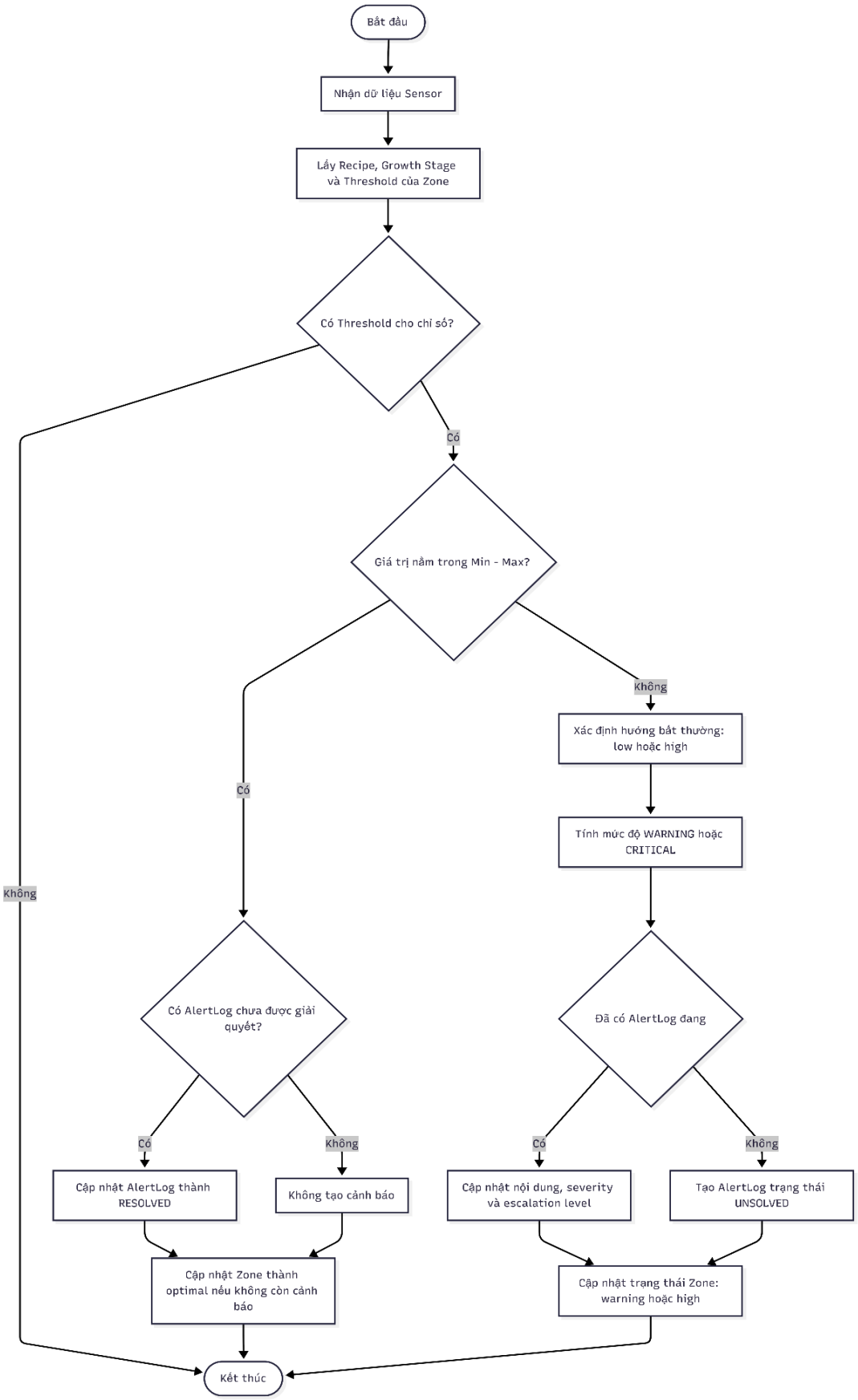
Hình 2.9. Sơ đồ hoạt động chức năng quản lý thiết bị cảm biến và thiết bị đầu ra



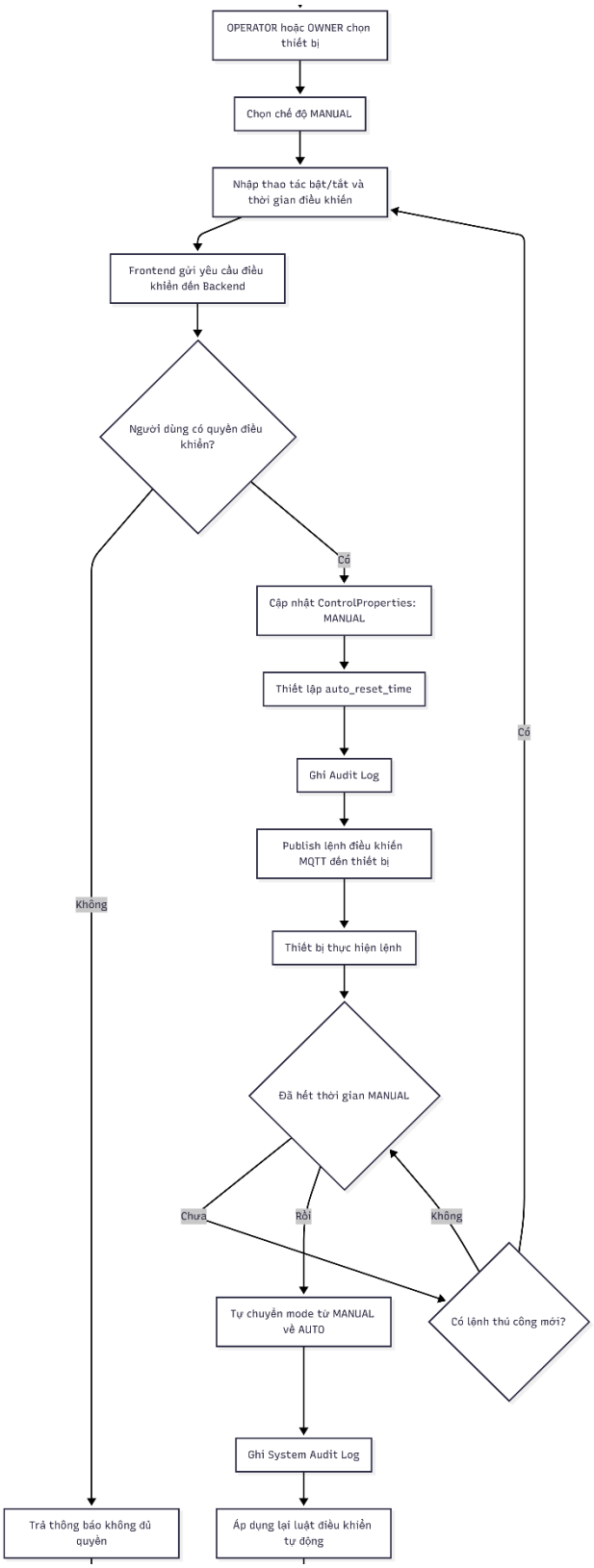
Hình 2.10. Sơ đồ hoạt động chức năng nhận và xử lý dữ liệu cảm biến MQTT



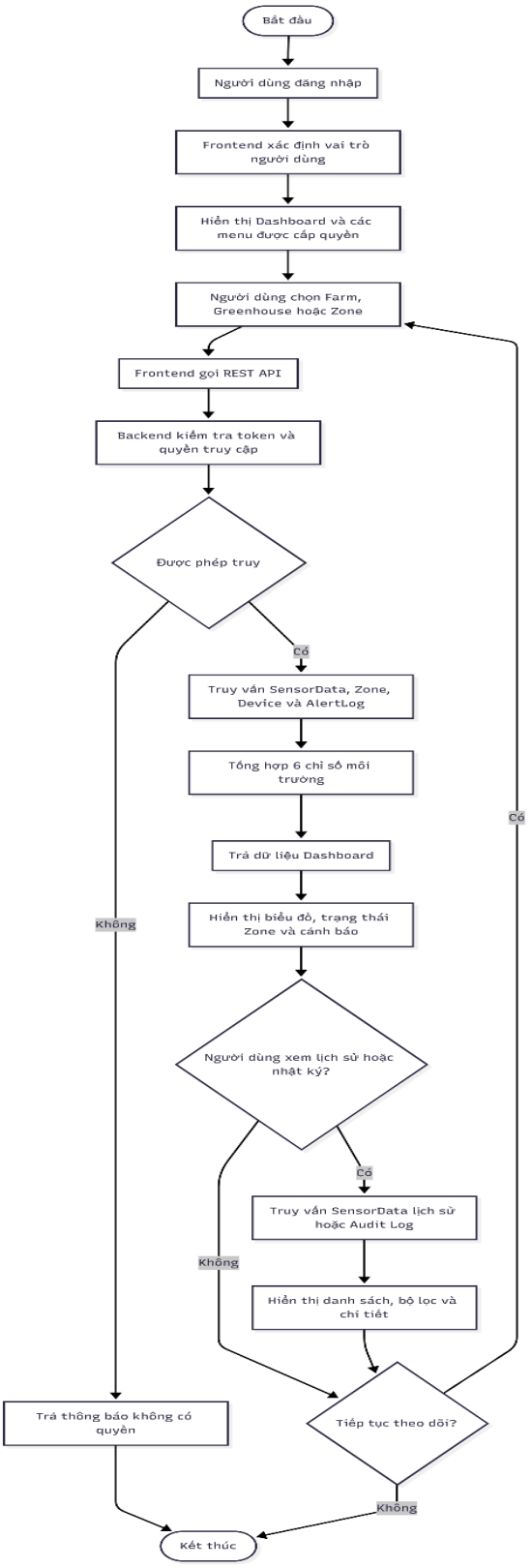
Hình 2.11. Sơ đồ hoạt động chức năng điều khiển thiết bị tự động



Hình 2.12. Sơ đồ hoạt động chức năng cảnh báo vượt ngưỡng



Hình 2.13. Sơ đồ hoạt động chức năng điều khiển thiết bị thủ công

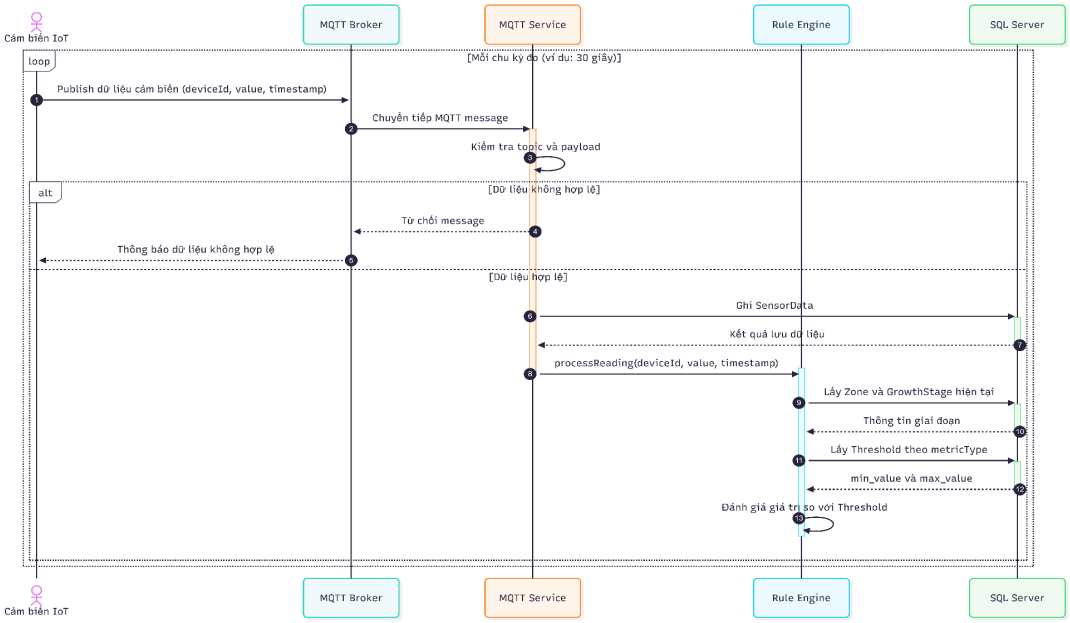


Hình 2.14. Sơ đồ hoạt động chức năng theo dõi Dashboard, lịch sử và Audit Log

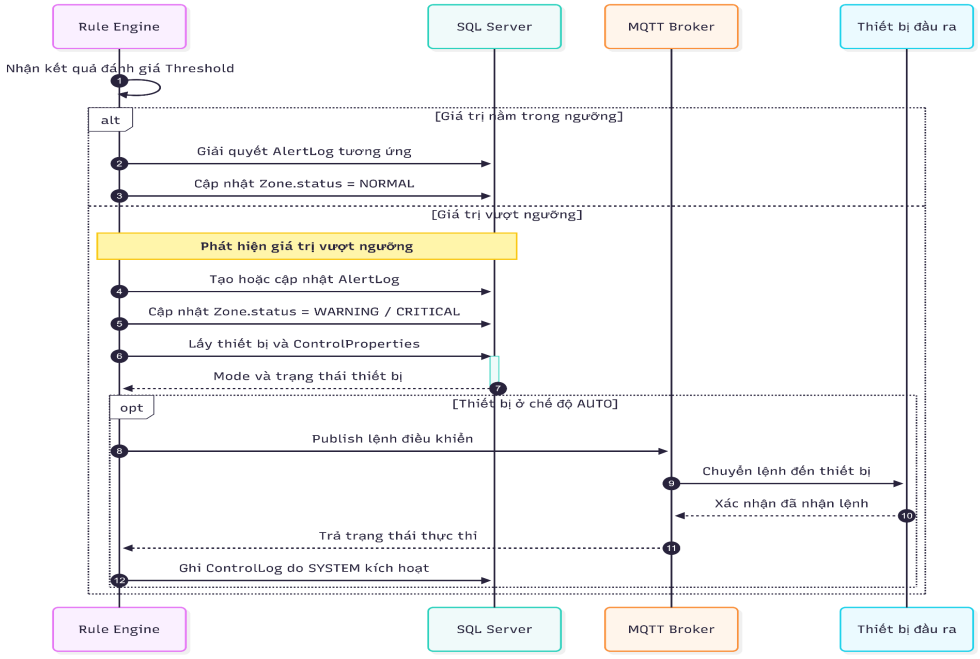
2.1.3. Sơ đồ tuần tự (Sequence Diagram)

a) Thu thập dữ liệu cảm biến và tạo cảnh báo

Cảm biến IoT định kỳ gửi dữ liệu môi trường đến MQTT Broker. MQTT Service tiếp nhận, kiểm tra tính hợp lệ và lưu dữ liệu vào bảng **SensorData**, sau đó chuyển dữ liệu đến Rule Engine để xác định Zone, GrowthStage và Threshold tương ứng. Hệ thống so sánh giá trị cảm biến với ngưỡng; nếu giá trị vượt ngưỡng, hệ thống tạo hoặc cập nhật **AlertLog**, cập nhật trạng thái Zone và điều khiển thiết bị tự động nếu thiết bị đang ở chế độ **AUTO**.



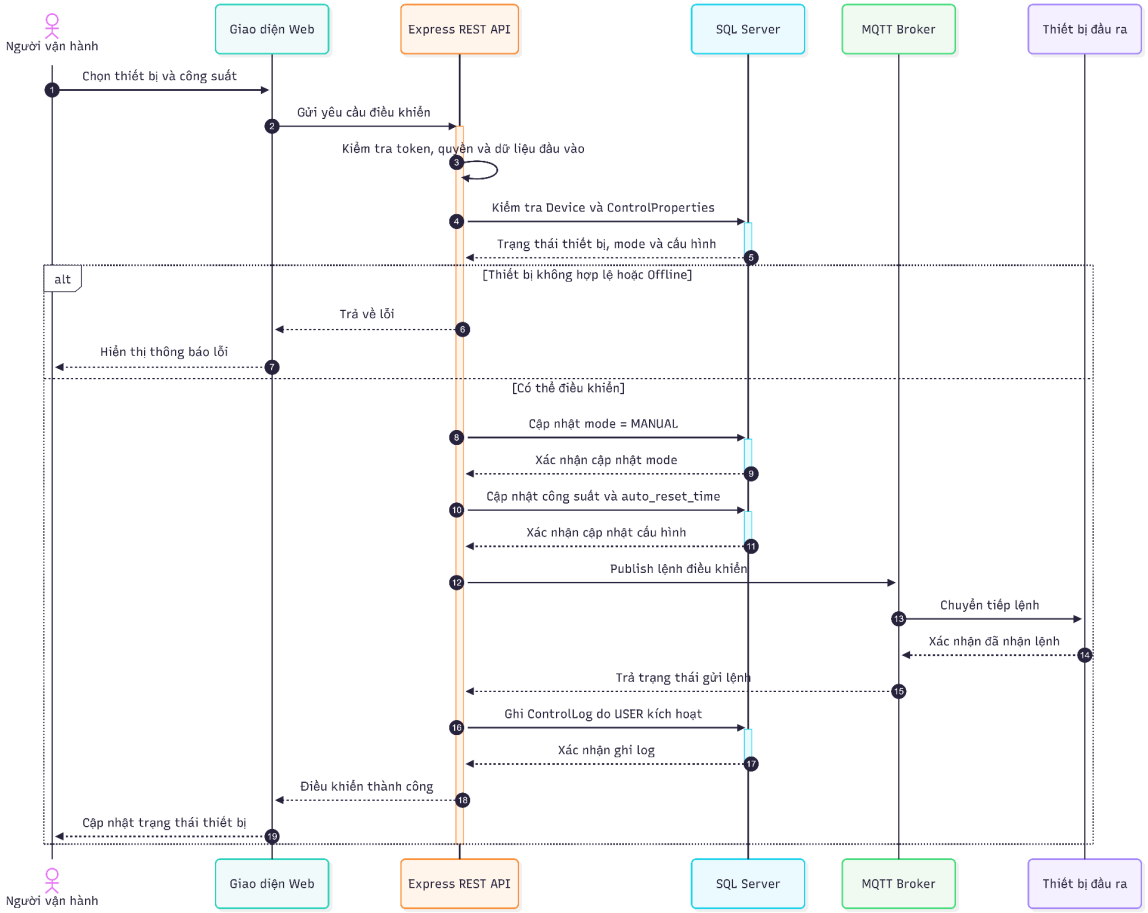
Hình 2.15. Sơ đồ tuần tự tiếp nhận và xử lý dữ liệu cảm biến



Hình 2.16. Sơ đồ tuần tự xử lý cảnh báo và điều khiển thiết bị

b) Điều khiển thiết bị tự động

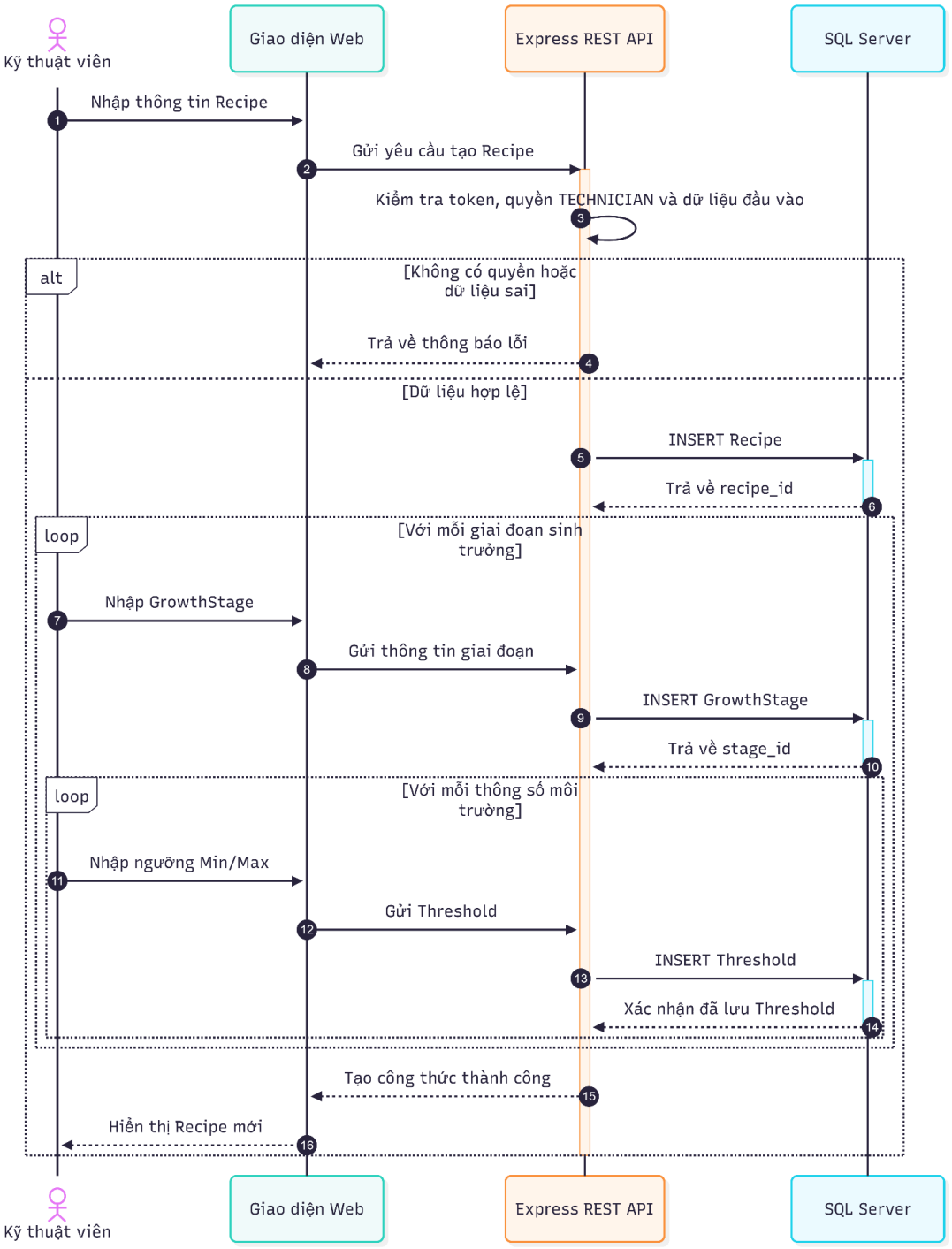
Khi dữ liệu cảm biến vượt ngưỡng, Rule Engine tìm thiết bị đầu ra phù hợp. Nếu thiết bị đang ở chế độ **AUTO**, hệ thống gửi lệnh điều khiển qua MQTT Broker và ghi lại hành động trong **Log.** Thiết bị ở chế độ **MANUAL** sẽ không bị hệ thống tự động ghi đè.



Hình 2.17. Sơ đồ tuần tự điều khiển thiết bị tự động

c) Quản lý công thức sinh trưởng

Kỹ thuật viên nhập thông tin công thức trên giao diện. Backend kiểm tra dữ liệu và lưu lần lượt vào các bảng **Recipe**, **GrowthStage** và **Threshold**. Sau khi hoàn thành, hệ thống trả kết quả và hiển thị công thức mới cho người dùng.



Hình 2.18. Sơ đồ tuần tự quản lý công thức sinh trưởng

2.1.4. Sơ đồ luồng dữ liệu (DFD)

Sơ đồ luồng dữ liệu (Data Flow Diagram – DFD) được sử dụng để mô tả quá trình luân chuyển dữ liệu giữa các tác nhân bên ngoài, các tiến trình xử lý và các kho dữ liệu trong hệ thống. DFD giúp thể hiện rõ cách thức hệ thống tiếp nhận, xử lý, lưu trữ và cung cấp dữ liệu cho người sử dụng.

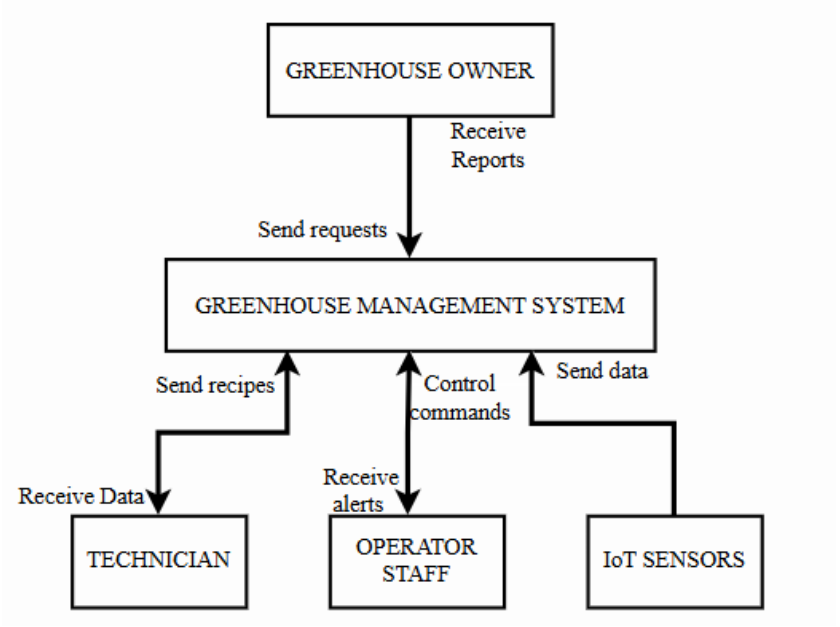
Dữ liệu được thu thập từ các cảm biến môi trường, xử lý tại máy chủ và được cung cấp cho người dùng thông qua giao diện quản lý. Đồng thời, hệ thống cũng hỗ trợ quản lý công thức sinh trưởng, giám sát điều kiện môi trường và phát hiện các sự cố bất thường.

a) DFD mức ngữ cảnh (Context Diagram)

Toàn bộ hệ thống được xem như một tiến trình tương tác với các tác nhân bên ngoài.

Bảng 2.1. Các tác nhân ngoài

Tác nhân	Vai trò
Chủ trang trại	Quản lý tài khoản, nhà kính, thiết bị và theo dõi toàn bộ hoạt động
Kỹ thuật viên	Thiết lập công thức sinh trưởng, ngưỡng môi trường, giám sát dữ liệu
NV vận hành	Theo dõi hệ thống, xử lý cảnh báo và thực hiện điều khiển thủ công
Thiết bị IoT	Gửi dữ liệu môi trường đến hệ thống



Hình 2.19. Sơ đồ luồng dữ liệu mức ngữ cảnh

Luồng dữ liệu:

- Chủ trang trại gửi yêu cầu quản lý hệ thống và nhận các báo cáo tổng hợp.
- Kỹ thuật viên gửi các công thức sinh trưởng và nhận dữ liệu phân tích.
- Nhân viên vận hành nhận cảnh báo và gửi lệnh điều khiển thiết bị.
- Thiết bị IoT gửi dữ liệu cảm biến môi trường về hệ thống.

b) DFD mức 1

Ở mức 1, hệ thống được phân rã thành các tiến trình chính tương ứng với các nhóm chức năng trong hệ thống.

Bảng 2.2. Các kho dữ liệu

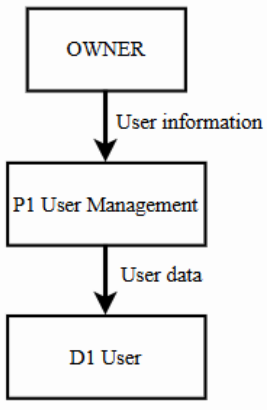
Ký hiệu	Kho dữ liệu	Ký hiệu	Kho dữ liệu
D1	User	D7	Recipe
D2	Farm	D8	GrowthStage
D3	Greenhouse	D9	Threshold
D4	Zone	D10	SensorData
D5	Gateway	D11	AlertLog
D6	Device	D12	Log

Bảng 2.3. Các tiến trình

Ký hiệu	Tiến trình	Ký hiệu	Tiến trình
P1	Quản lý người dùng	P5	Thu thập dữ liệu cảm biến
P2	Quản lý cơ sở hạ tầng	P6	Giám sát và cảnh báo
P3	Quản lý thiết bị IoT	P7	Tra cứu và thống kê
P4	Quản lý công thức sinh trưởng		

DFD mức 1 chia hệ thống quản lý nhà kính trồng hoa thành 7 tiến trình. Các tiến trình tương ứng với các nhóm chức năng và liên kết với các kho dữ liệu trong cơ sở dữ liệu.

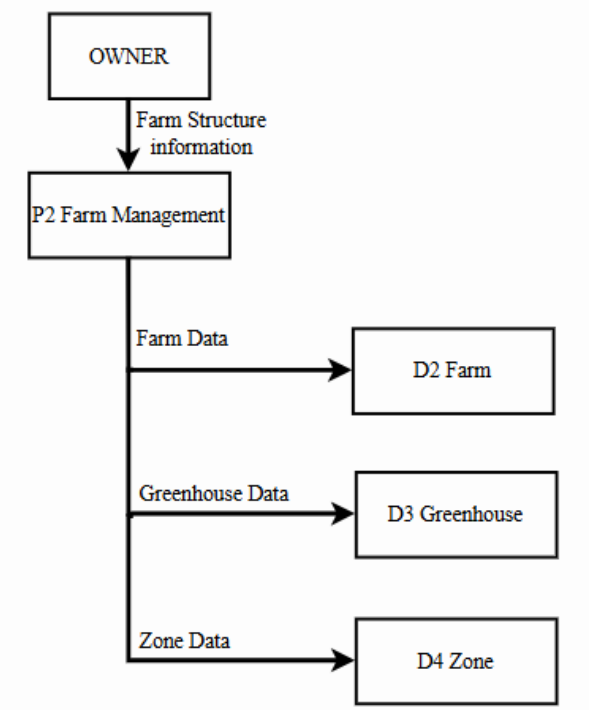
P1. User Management



Hình 2.20. Sơ đồ DFD User Management

Tiến trình User Management tiếp nhận thông tin tài khoản từ chủ trang trại, thực hiện thêm mới, cập nhật hoặc phân quyền người dùng. Dữ liệu sau khi xử lý được lưu vào kho dữ liệu User.

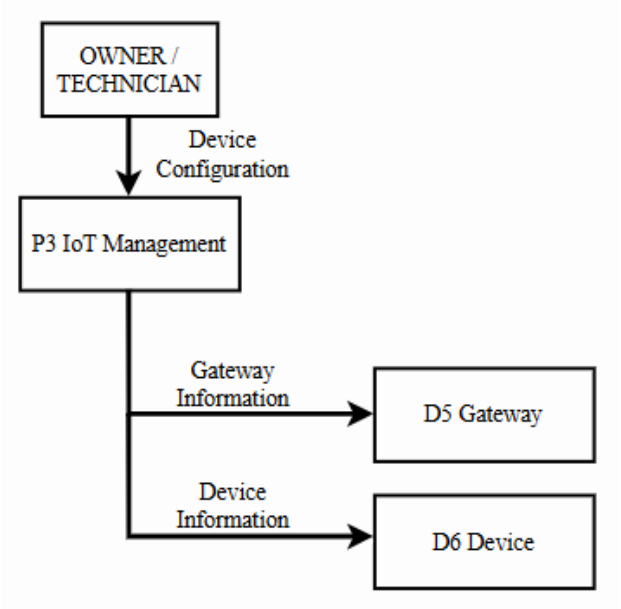
P2. Farm Management



Hình 2.21. Sơ đồ DFD Farm Management

Tiến trình Farm Management cho phép quản lý cấu trúc trang trại, nhà kính và khu vực trồng. Các thông tin được lưu trữ tại các kho dữ liệu Farm, Greenhouse và Zone.

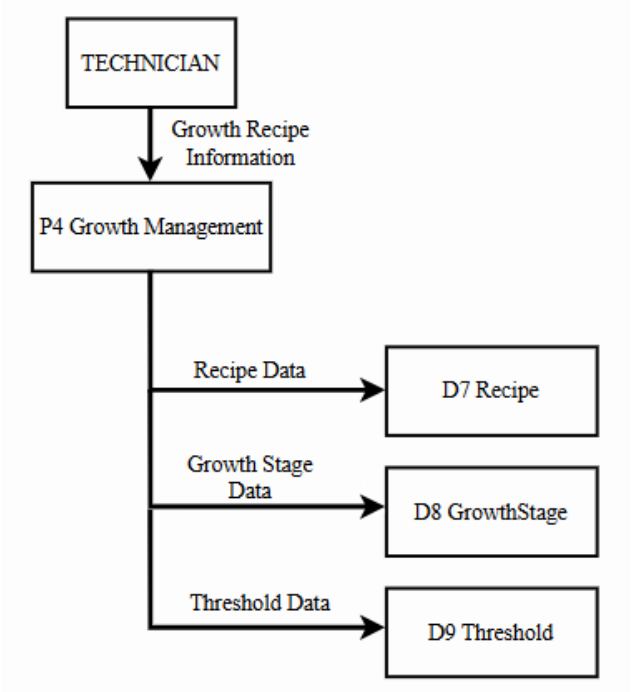
P3. IoT Management



Hình 2.22. Sơ đồ DFD IoT Management

Tiến trình IoT Management quản lý các Gateway và thiết bị IoT trong hệ thống, bao gồm cảm biến và các thiết bị giám sát môi trường.

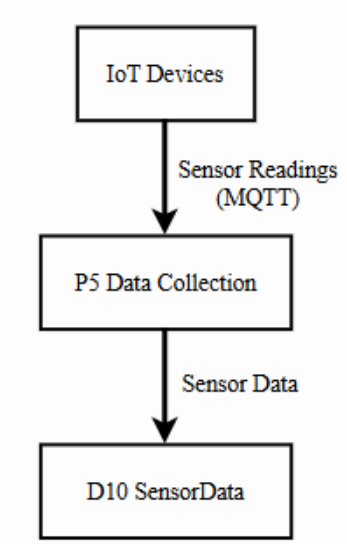
P4. Growth Management



Hình 2.23. Sơ đồ DFD Growth Management

Tiến trình Growth Management quản lý công thức sinh trưởng, các giai đoạn phát triển và ngưỡng môi trường phục vụ quá trình giám sát tự động.

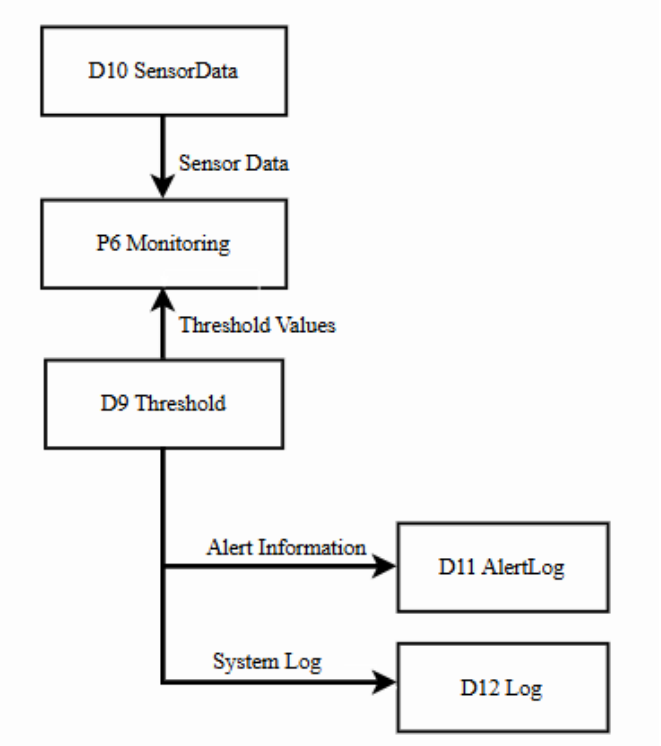
P5. Data Collection



Hình 2.24. Sơ đồ DFD Data Collection

Tiến trình Data Collection tiếp nhận dữ liệu cảm biến từ các thiết bị IoT thông qua MQTT và lưu trữ vào cơ sở dữ liệu.

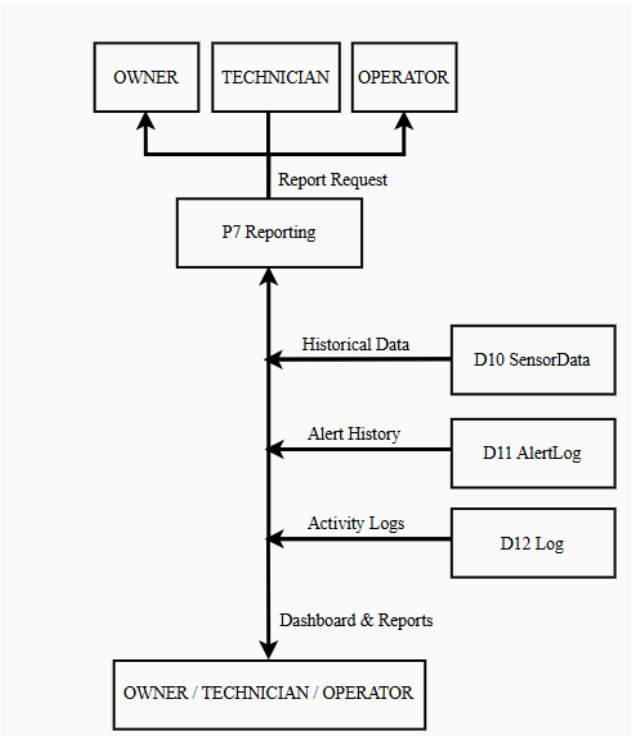
P6. Monitoring



Hình 2.25. Sơ đồ DFD Monitoring

Tiến trình Monitoring đọc dữ liệu cảm biến, so sánh với các ngưỡng sinh trưởng và tạo cảnh báo khi phát hiện bất thường.

P7. Reporting

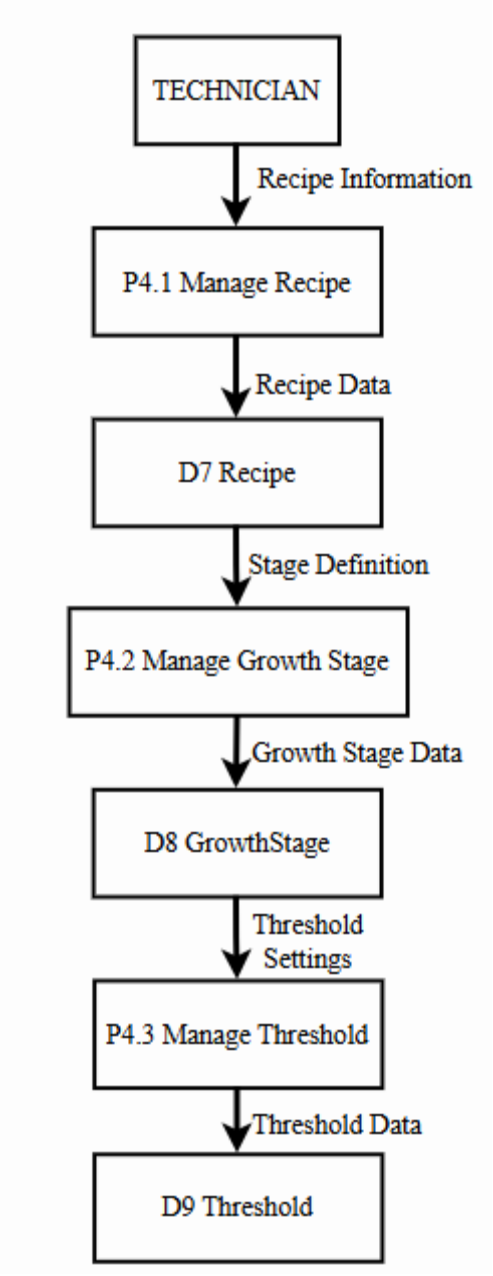


Hình 2.26. Sơ đồ DFD Reporting

Tiến trình Reporting tổng hợp dữ liệu cảm biến, lịch sử cảnh báo và nhật ký hệ thống để hiển thị dashboard, biểu đồ và báo cáo phục vụ người dùng.

c) DFD mức 2 - Quản lý sinh trưởng (P4)

Tiến trình quản lý sinh trưởng chịu trách nhiệm xây dựng công thức sinh trưởng cho từng loại hoa, định nghĩa các giai đoạn phát triển và các ngưỡng môi trường tương ứng.

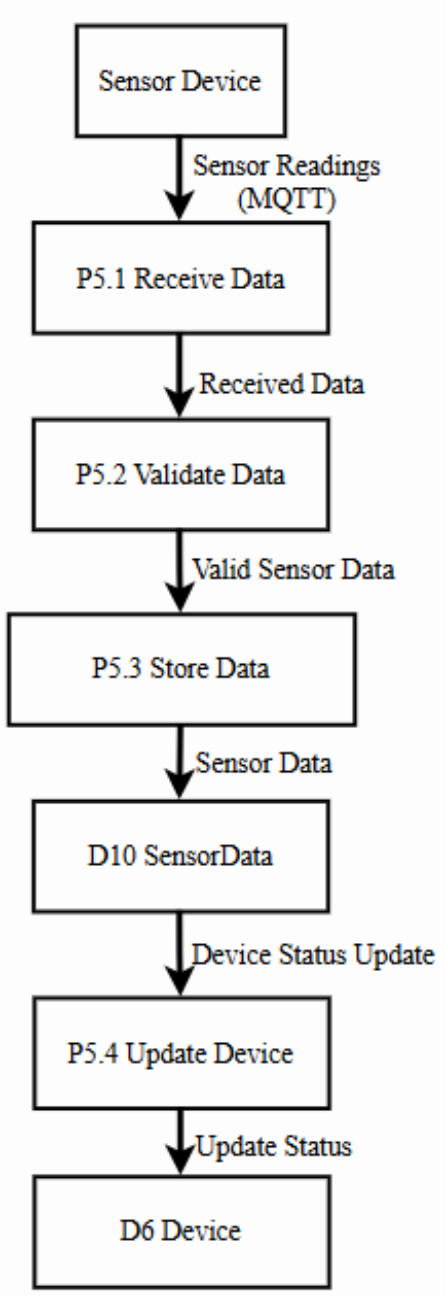


Hình 2.27. Sơ đồ DFD mức 2 - Quản lý sinh trưởng

Kỹ thuật viên tạo công thức sinh trưởng mới, trong đó, mỗi công thức gồm nhiều giai đoạn phát triển. Mỗi giai đoạn lại chứa các ngưỡng nhiệt độ, độ ẩm, ánh sáng, pH, CO_2 và độ ẩm đất. Các ngưỡng này được sử dụng trong quá trình giám sát môi trường.

d) DFD mức 2 – Thu thập dữ liệu cảm biến (P5)

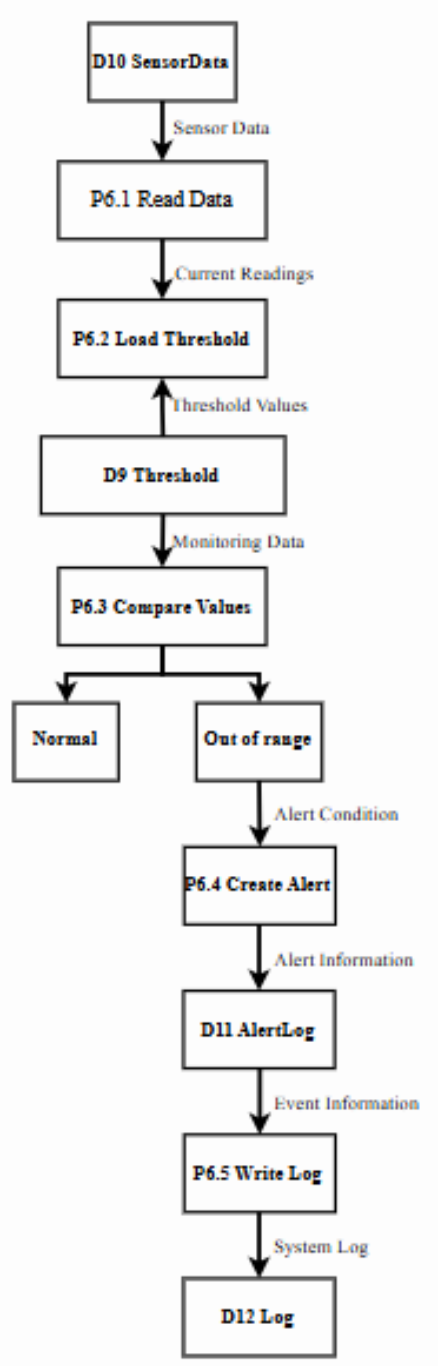
Tiến trình này tiếp nhận dữ liệu từ các cảm biến thông qua MQTT và lưu trữ vào cơ sở dữ liệu.



Hình 2.28. Sơ đồ DFD mức 2 - Thu thập dữ liệu cảm biến

Thiết bị cảm biến gửi dữ liệu định kỳ. Sau đó, hệ thống kiểm tra tính hợp lệ của dữ liệu. Tiếp theo, dữ liệu được lưu vào bảng SensorData. Cuối cùng, trạng thái hoạt động của cảm biến được cập nhật vào bảng Device.

e) DFD mức 2 – Giám sát và cảnh báo (P6)



Hình 2.29. Sơ đồ DFD mức 2 - Giám sát và cảnh báo

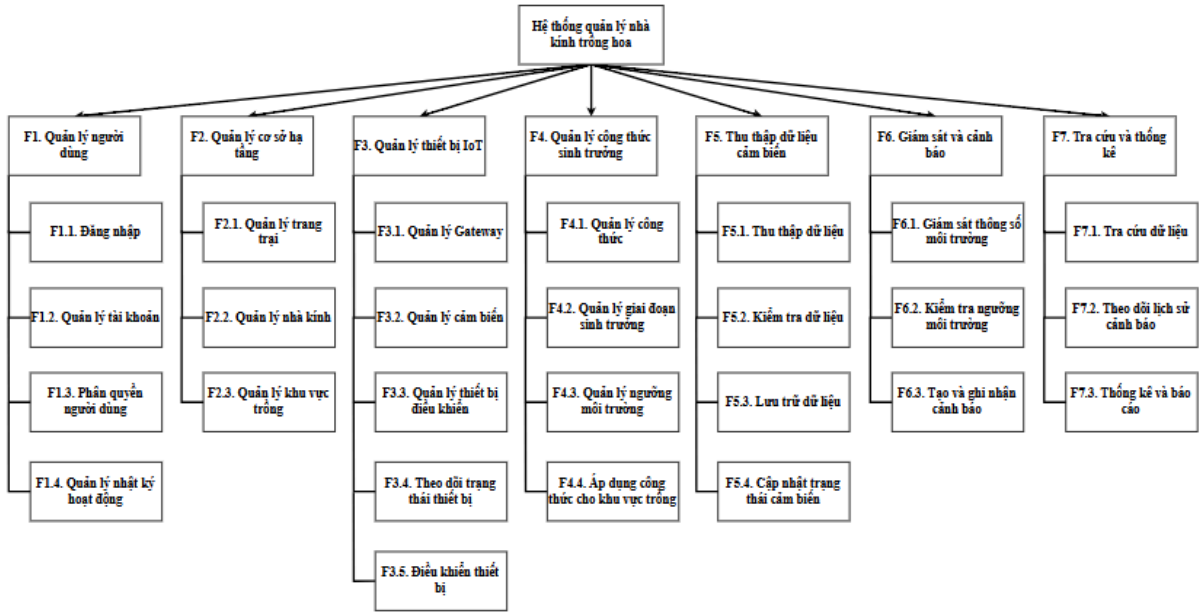
Tiến trình Monitoring là trung tâm xử lý dữ liệu của hệ thống:

- P6.1 Read Data đọc dữ liệu môi trường từ D10 SensorData.
- P6.2 Load Threshold tải các ngưỡng sinh trưởng tương ứng từ D9 Threshold.
- P6.3 Compare Values so sánh dữ liệu thực tế với các giá trị ngưỡng quy định.
- Nếu dữ liệu nằm trong phạm vi cho phép, hệ thống tiếp tục theo dõi.
- Nếu dữ liệu vượt ngưỡng, P6.4 Create Alert tạo cảnh báo và lưu vào D11 AlertLog.

- P6.5 Write Log ghi nhận toàn bộ sự kiện vào D12 Log để phục vụ kiểm tra và truy vết sau này.

2.1.5. Sơ đồ phân cấp chức năng (FHD)

Sơ đồ phân cấp chức năng được sử dụng để phân rã các chức năng của hệ thống từ mức tổng quát đến các chức năng chi tiết. Sơ đồ giúp xác định phạm vi chức năng và tổ chức các chức năng của hệ thống theo cấu trúc phân cấp, đồng thời làm cơ sở cho việc phân tích DFD và thiết kế các thành phần của hệ thống. Đối với hệ thống quản lý nhà kính trồng hoa, các chức năng được phân thành bảy nhóm chính gồm quản lý người dùng, quản lý cơ sở hạ tầng, quản lý thiết bị IoT, quản lý công thức sinh trưởng, thu thập dữ liệu cảm biến, giám sát và cảnh báo, tra cứu và thống kê.



Hình 2.30. Sơ đồ phân cấp chức năng

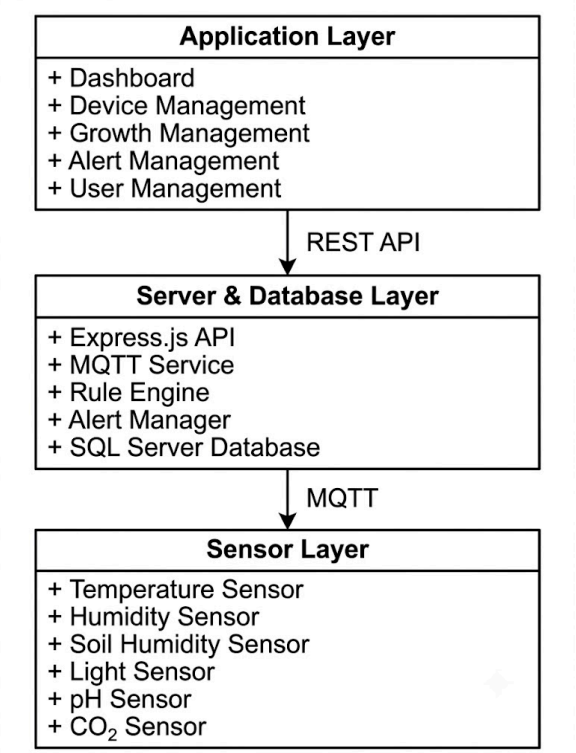
Sơ đồ FHD cho thấy hệ thống quản lý nhà kính trồng hoa được phân rã thành bảy nhóm chức năng chính. Các chức năng cấp cao tiếp tục được phân chia thành các chức năng con nhằm mô tả đầy đủ phạm vi nghiệp vụ của hệ thống. Cấu trúc phân cấp này là cơ sở để đối chiếu với các tiến trình trong DFD và hỗ trợ quá trình thiết kế hệ thống ở các bước tiếp theo.

2.2. Thiết kế hệ thống

2.2.1. Kiến trúc tổng thể hệ thống

Hệ thống quản lý nhà kính trồng hoa được xây dựng theo mô hình kiến trúc ba tầng kết hợp với giao thức truyền thông MQTT nhằm đáp ứng yêu cầu thu thập dữ liệu thời gian thực, quản lý thiết bị IoT và hỗ trợ điều khiển tự động trong môi trường nhà kính.

Kiến trúc này giúp tách biệt giữa tầng thu thập dữ liệu, tầng xử lý nghiệp vụ và tầng giao diện người dùng, từ đó nâng cao khả năng mở rộng, bảo trì và phát triển hệ thống.



2.31. Sơ đồ kiến trúc hệ thống

a) Sensor Layer

Tầng cảm biến chịu trách nhiệm thu thập các thông số môi trường trong nhà kính. Các cảm biến được kết nối với bộ điều khiển IoT và gửi dữ liệu định kỳ tới MQTT Broker.

Các thông số được theo dõi bao gồm nhiệt độ (Temperature), độ ẩm không khí (Humidity), cường độ ánh sáng (Light), độ ẩm đất (Soil Humidity), độ pH, nồng độ CO_2.

Dữ liệu cảm biến được gửi theo topic MQTT: greenhouse / {greenhouseId} / zone / {zoneId} / sensor / {deviceId} / data

b) Server & Database Layer

Đây là tầng xử lý trung tâm của hệ thống. Các thành phần chính bao gồm MQTT Service, Express REST API, Rule Engine, SQL Server Database.

Tầng này chịu trách nhiệm tiếp nhận dữ liệu từ các cảm biến thông qua giao thức MQTT, kiểm tra tính hợp lệ của dữ liệu trước khi lưu trữ vào bảng SensorData. Dữ liệu sau đó được so sánh với các ngưỡng được cấu hình trong Threshold để phát hiện các trường hợp bất thường và tạo hoặc cập nhật thông tin trong AlertLog. Đồng thời, tầng này thực hiện điều khiển thiết bị theo chế độ AUTO dựa trên dữ liệu cảm biến và cung cấp dữ liệu đã xử lý cho giao diện người dùng phục vụ việc giám sát và quản lý hệ thống

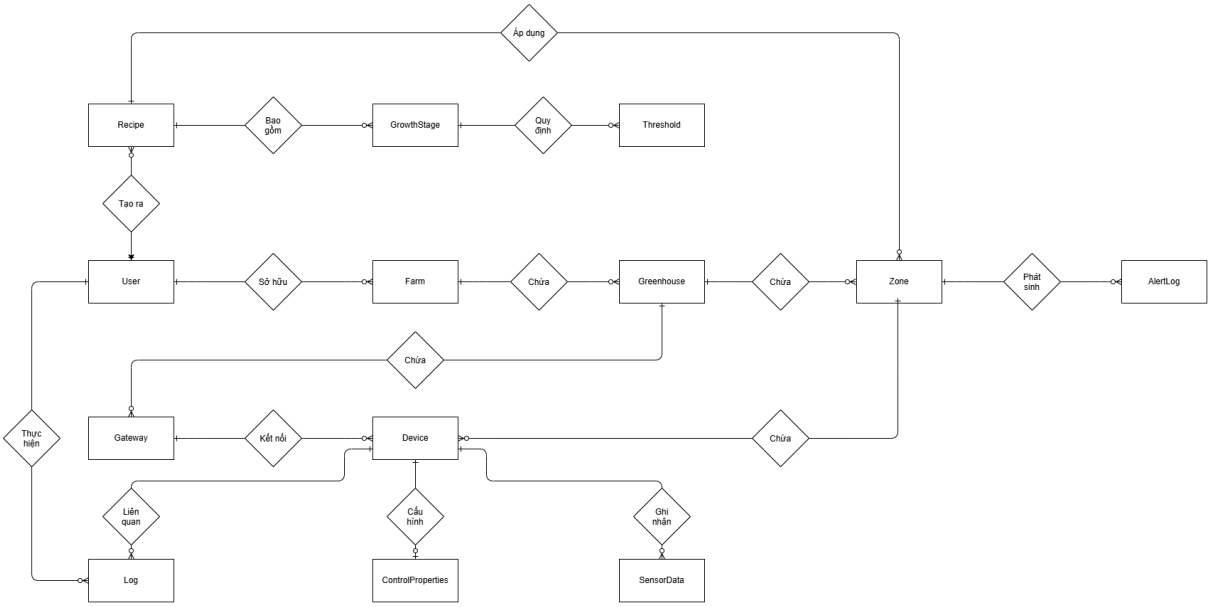
c) Application Layer

Application Layer là giao diện Web được xây dựng bằng HTML, CSS và JavaScript. Người dùng tương tác với hệ thống thông qua giao diện quản lý, bao gồm các chức năng giám sát thời gian thực trên Dashboard, quản lý Farm, Greenhouse và Zone, quản lý cảm biến và thiết bị, quản lý Recipe và GrowthStage, theo dõi cảnh báo, điều khiển thiết bị thủ công và quản lý người dùng, phân quyền. Dữ liệu và các yêu cầu thao tác được trao đổi giữa giao diện người dùng và Server thông qua các REST API, theo mô hình client–server, trong đó giao diện đảm nhiệm việc hiển thị và tiếp nhận thao tác của người dùng, còn Server xử lý nghiệp vụ và dữ liệu của hệ thống.

2.2.2. Sơ đồ quan hệ thực thể (ERD)

Sơ đồ này mô tả các thực thể chính trong hệ thống như User, Farm, Greenhouse, Zone, Device, SensorData và các mối quan hệ giữa chúng. Thông qua ERD, chúng ta có thể hình dung rõ hơn cách dữ liệu được tổ chức, liên kết và lưu trữ, từ đó làm cơ sở cho việc xây dựng cơ sở dữ liệu và triển khai các chức năng quản lý, giám sát nhà kính.

Đầu tiên là **ERD tổng quát**, giúp chúng ta có cái nhìn toàn cảnh về các thực thể chính của hệ thống và mối quan hệ giữa chúng.



Hình 2.32. Sơ đồ quan hệ thực thể tổng quát

Tiếp theo là **ERD chi tiết**, trong đó các thực thể được phân rã cụ thể hơn với các thuộc tính và mối quan hệ, làm cơ sở cho việc thiết kế cơ sở dữ liệu.



Hình 2.33. Sơ đồ quan hệ thực thể chi tiết

Các thực thể được chia thành 4 nhóm chính gồm nhóm Cơ sở hạ tầng, nhóm Thiết bị IoT, nhóm Công thức sinh trưởng và nhóm Dữ liệu, Lịch sử & Nhật ký.

a) Nhóm Cơ sở hạ tầng

User:

- user_id (PK): ID của người dùng.
- user_name: Tên tài khoản.
- password: Mật khẩu đăng nhập.
- email: Địa chỉ email của người dùng.
- phone_number: Số điện thoại của người dùng.
- role: Quyền của tài khoản. Có 3 role là OWNER, TECHNICIAN, OPERATOR.

- status: Trạng thái tài khoản. Có 3 trạng thái gồm ACTIVE, PENDING và REJECTED.
- is_primary_owner: Có 2 trạng thái 0 và 1. Xác định chủ sở hữu tối cao nhất, không thể bị thay thế. Chỉ có duy nhất một ID mang trạng thái 1.

User	
PK	**user_id**
	user_name
	password
	email
	phone_number
	role
	status
	is_primary_owner

Hình 2.34. Bảng thực thể User

Farm:

- farm_id (PK): ID của trang trại.
- owner_id (FK —> User): ID của chủ trang trại.
- address: Địa chỉ trang trại.
- farm_name: Tên trang trại.

Farm	
PK	**farm_id**
FK	owner_id
	address
	farm_name

Hình 2.35. Bảng thực thể Farm

Greenhouse:

- greenhouse_id (PK): ID của nhà kính.
- farm_id (FK —> Farm): ID của trang trại mà nhà kính thuộc về.
- greenhouse_name: Tên nhà kính.
- location_gps: Địa chỉ nhà kính qua gps.

Greenhouse	
PK	**greenhouse_id**
FK	farm_id
	greenhouse_name
	location_gps

Hình 2.36. Bảng thực thể Greenhouse

Zone:

- zone_id (PK): ID của khu vực trồng.
- greenhouse_id (FK —> Greenhouse): ID của nhà kính mà khu vực đó thuộc.
- recipe_id (FK —> Recipe): Công thức sinh trưởng đang áp dụng tại khu vực này, có thể để trống nếu chưa gieo trồng.
- zone_name: Tên khu vực trồng.
- start_date: Thời điểm bắt đầu trồng tại khu vực.
- temperature: Nhiệt độ hiện tại của khu vực.
- humidity: Độ ẩm hiện tại của khu vực.
- status: Trạng thái hoạt động của khu vực.
- cycle_adjustment_days: Số ngày nhân viên điều chỉnh chu kỳ sinh trưởng.
- adjustment_reason: Lý do điều chỉnh chu kỳ sinh trưởng.

Zone	
PK	**zone_id**
FK	greenhouse_id
FK	recipe_id
	zone_name
	start_date
	temperature
	humidity
	status
	cycle_adjustment_days
	adjustment_reason

Hình 2.37. Bảng thực thể Zone

b) Nhóm Thiết bị IoT:

Gateway:

- gateway_id (PK): ID của trung tâm điều khiển thiết bị.
- greenhouse_id (FK —> Greenhouse): ID của nhà kính mà trung tâm điều khiển thiết bị đó thuộc về.
- status: Trạng thái của trung tâm điều khiển thiết bị, gồm ONLINE, OFFLINE và MAINTENANCE.
- gateway_address: Địa chỉ cổng.

Gateway	
PK	**gateway_id**
FK	greenhouse_id
	status
	gateway_address

Hình 2.38. Bảng thực thể Gateway

Device:

- device_id (PK): ID của thiết bị IoT.
- gateway_id (FK —> Gateway): ID của trung tâm điều khiển thiết bị mà thiết bị kết nối đến.
- zone_id (FK —> Zone): ID của khu vực trồng mà thiết bị đó thuộc về.
- device_name: Tên thiết bị.
- device_type: Loại thiết bị. Có 2 loại gồm SENSOR (Cảm biến) và OUTPUT_DEVICE (Thiết bị đầu ra như bơm nước, mái che tự động,...).
- metric_type: Loại dữ liệu của thiết bị, dùng để xác định xem thiết bị đo dữ liệu gì từ môi trường hoặc điều khiển gì ở đầu ra (VD: Nhiệt độ, độ ẩm, bơm nước, quạt,...).
- status: Trạng thái của thiết bị, gồm ONLINE, OFFLINE, ERROR, PENDING và NEEDS-REPLACEMENT.
- mac_address: Địa chỉ MAC, các thiết bị có địa chỉ không trùng nhau.
- battery_level: Phần trăm pin, từ 0–100%.
- last_heartbeat: Thời điểm thiết bị liên lạc gần nhất.

Device	
PK	**device_id**
FK	gateway_id
FK	zone_id
	device_name
	device_type
	metric_type
	status
	mac_address
	battery_level
	last_heartbeat

Hình 2.39. Bảng thực thể Device

ControlProperties:

- device_id (FK → Device): ID của thiết bị cần điều khiển.
- mode: Chế độ của thiết bị, có 2 chế độ là AUTO và MANUAL.
- is_active: trạng thái xác định thiết bị đang bật hay là không.
- value_percent: Công suất hoạt động của thiết bị trong khoảng từ 0-100%.
- auto_reset_time: thời điểm chế độ của thiết bị tự động chỉnh về trạng thái ban đầu.

ControlProperties	
FK	device_id
	mode
	is_active
	value_percent
	auto_reset_time

Hình 2.40. Bảng thực thể ControlProperties

c) Nhóm Công thức sinh trưởng:

Recipe:

- recipe_id (PK): ID của công thức sinh trưởng.

- creator_id (FK —> User): ID của người tạo ra công thức. Chỉ người có role Technician mới được tạo công thức mới.
- recipe_name: Tên công thức sinh trưởng.
- flower_type: Tên loại hoa được trồng trong công thức.
- description: Mô tả về công thức.
- status: Trạng thái của công thức. Có 3 trạng thái gồm ACTIVE, DELAYED VÀ COMPLETED.
- created_date: Ngày tạo công thức.

Recipe	
PK	**recipe_id**
FK	creator_id
	recipe_name
	flower_type
	description
	status
	created_date

Hình 2.41. Bảng thực thể Recipe

GrowthStage:

- stage_id (PK): ID của giai đoạn sinh trưởng.
- recipe_id (FK —> Recipe): ID công thức sinh trưởng của giai đoạn đó.
- stage_name: Tên giai đoạn sinh trưởng.
- start_day: Ngày bắt đầu giai đoạn sinh trưởng. Luôn ≥0.
- end_day: Ngày kết thúc giai đoạn sinh trưởng. end_day ≥ start_day.
- completed: Trạng thái đã hoàn thành giai đoạn hay chưa.
- current_day: thời gian hiện tại của giai đoạn. Luôn ≥0 hoặc NULL.

GrowthStage	
PK	**stage_id**
FK	recipe_id
	stage_name
	start_day
	end_day
	completed
	current_day

Hình 2.42. Bảng thực thể GrowthStage

Threshold:

- threshold_id (PK): ID của ngưỡng giới hạn điều kiện sinh trưởng.
- stage_id (FK —> GrowthStage): ID của giai đoạn sinh trưởng mà ngưỡng thuộc về.
- metric_type: Loại dữ liệu của ngưỡng đó. Các loại dữ liệu phải khớp với loại thiết bị của Device.
- min_value: Giá trị ngưỡng tối thiểu.
- max_value: Giá trị ngưỡng tối đa. max_value ≥ min_value.

Threshold	
PK	**threshold_id**
FK	stage_id
	metric_type
	min_value
	max_value

Hình 2.43. Bảng thực thể Threshold

d) Nhóm Dữ liệu, Lịch sử & Nhật ký:

SensorData:

- timestamp (PK): Thời gian ghi nhận dữ liệu từ cảm biến.
- device_id (FK —> Device): ID của thiết bị gửi dữ liệu đến, thiết bị phải là loại SENSOR.
- raw_value: Giá trị ghi nhận được.

SensorData	
PK	**timestamp**
FK	device_id
	raw_value

Hình 2.44. Bảng thực thể SensorData

Log:

- log_id (PK): Mã định danh của bản ghi nhật ký.
- device_id (FK → Device, NULL): Thiết bị liên quan đến hành động. Có thể là cảm biến hoặc thiết bị đầu ra. Giá trị chuyển thành NULL nếu thiết bị đã bị xóa, nhằm giữ lại lịch sử.
- user_id (FK → User, NULL): Người thực hiện hành động. Bằng NULL nếu hành động do hệ thống tự động thực hiện.
- action: Nội dung mô tả hành động như thêm, cập nhật, xóa hoặc điều khiển thiết bị.
- triggered_by: Nguồn thực hiện hành động, gồm USER hoặc SYSTEM.
- log_time: Thời điểm hành động xảy ra.
- entity_type: Dùng để phân loại đối tượng được ghi nhận trong nhật ký hệ thống, bao gồm DEVICE, ZONE, GREENHOUSE, FARM, RECIPE, GROWTH_STAGE, ALERT, USER, SIMULATION.
- entity_id: Địa chỉ id của đối tượng được ghi nhận trong hệ thống (nếu có).

Log	
PK	**log_id**
FK	device_id
FK	user_id
	action
	triggered_by
	log_time
	entity_type
	entity_id

Hình 2.45. Bảng thực thể Log

AlertLog:

- alert_id (PK): ID của cảnh báo sự cố gửi về hệ thống.
- zone_id (FK —> Zone): ID của khu vực xảy ra sự cố.
- message: Nội dung cảnh báo được gửi đi.
- severity: Mức độ nghiêm trọng của sự cố, có 3 mức độ nghiêm trọng tăng dần gồm SIGNIFICANT, WARNING và CRITICAL.
- status: Tình trạng xử lý sự cố, gồm 3 trạng thái là UNSOLVED, ACKNOWLEDGED và RESOLVED.
- created_at: Thời điểm tạo cảnh báo.
- resolved_at: Thời điểm xử lý xong.
- acknowledged_by: Người xác nhận cảnh báo.
- escalation_level: Cấp độ cảnh báo thứ cấp.

AlertLog	
PK	**alert_id**
FK	zone_id
	message
	severity
	status
	timestamp
	created_at
	resolved_at
	acknowledged_by
	escalation_level

Hình 2.46. Bảng thực thể AlertLog

- Mối quan hệ giữa các thực thể:
  - Một User có thể sở hữu nhiều Farm, mỗi Farm thuộc một chủ sở hữu (1-0..N).
  - Một Farm chứa nhiều Greenhouse, mỗi Greenhouse thuộc một Farm (1-0..N).
  - Một Greenhouse chứa nhiều Zone, mỗi Zone thuộc một Greenhouse (1-0..N).
  - Một Greenhouse có nhiều Gateway, mỗi Gateway thuộc một Greenhouse (1-0..N).

- Một Gateway kết nối với nhiều Device, mỗi Device thuộc một Gateway (1-0..N).
- Một Zone có nhiều Device, mỗi Device có thể được gán vào một Zone (1-0..N).
- Một Zone có thể phát sinh nhiều AlertLog, mỗi cảnh báo thuộc một Zone (1-0..N).
- Một Device loại OUTPUT_DEVICE có tối đa một ControlProperties (1-0..1).
- Một Device loại SENSOR có thể tạo nhiều SensorData (1-0..N).
- Một Device có thể liên quan đến nhiều Log. Khi Device bị xóa, Log.device_id được đặt thành NULL để giữ lịch sử (0..1-0..N).
- Một User có thể thực hiện nhiều hành động và tạo nhiều Log. Với hành động do hệ thống thực hiện, user_id bằng NULL (0..1-0..N).
- Một User, thường có vai trò TECHNICIAN, có thể tạo nhiều Recipe; mỗi Recipe do một User tạo (1-0..N).
- Một Recipe có nhiều GrowthStage, mỗi GrowthStage thuộc một Recipe (1-0..N).
- Một GrowthStage có nhiều Threshold, mỗi Threshold thuộc một GrowthStage (1-0..N).
- Một Recipe có thể được áp dụng cho nhiều Zone; mỗi Zone áp dụng tối đa một Recipe tại một thời điểm (1-N).

2.3. Thiết kế lớp hệ thống

2.3.1. Cấu trúc lớp Sensor

a) Tổng quan

Lớp Sensor đại diện cho các thiết bị cảm biến được triển khai trong nhà kính nhằm thu thập dữ liệu môi trường phục vụ quá trình giám sát và điều khiển tự động. Các cảm biến được kết nối với bộ điều khiển IoT và định kỳ gửi dữ liệu lên máy chủ qua giao thức MQTT.

Nhiệm vụ chính của lớp Sensor là quản lý thông tin định danh, loại cảm biến, vị trí lắp đặt và trạng thái hoạt động của từng thiết bị. Dữ liệu đo đạc theo thời gian thực không được lưu trực tiếp trong lớp Sensor mà được lưu dưới dạng các bản ghi của lớp Data để tối ưu khả năng lưu trữ và truy xuất dữ liệu lịch sử.

Mỗi cảm biến được gán một mã định danh duy nhất (**sensorID**) để hệ thống có thể nhận diện chính xác nguồn dữ liệu. Khi một giá trị đo được gửi lên máy chủ, hệ thống sử dụng sensorID để liên kết dữ liệu với đúng cảm biến tương ứng. Để giảm dung lượng lưu trữ

và tăng tốc độ xử lý, các loại cảm biến và trạng thái hoạt động được mã hóa bằng các giá trị số nguyên thay vì lưu dưới dạng chuỗi ký tự.

b) Thuộc tính lớp

Bảng 2.4. Thuộc tính lớp Sensor

Attribute Code	Tên thuộc tính	Kiểu dữ liệu	Mô tả
100	sensorID	INT	Mã định danh duy nhất của cảm biến
200	sensorName	VARCHAR(100)	Tên cảm biến
300	sensorType	SMALLINT	Loại cảm biến
500	status	SMALLINT	Trạng thái hoạt động
700	lastUpdateTime	DATETIME	Thời điểm gửi dữ liệu gần nhất

c) Chức năng lớp

Bảng 2.5. Chức năng lớp Sensor

Function Code	Tên chức năng	Mô tả
1000	collectData()	Thu thập dữ liệu từ môi trường
1100	sendData()	Gửi dữ liệu đến máy chủ
1200	updateStatus()	Cập nhật trạng thái hoạt động
1300	checkConnection()	Kiểm tra kết nối với hệ thống

d) Loại cảm biến

Bảng 2.6. Mã hóa sensorType

Type Code	Loại cảm biến
310	Nhiệt độ
320	Độ ẩm không khí
330	Cường độ ánh sáng
350	Độ ẩm đất
360	Độ pH
380	Nồng độ CO_2

e) Trạng thái lớp

Bảng 2.7. Mã hóa Status lớp Sensor

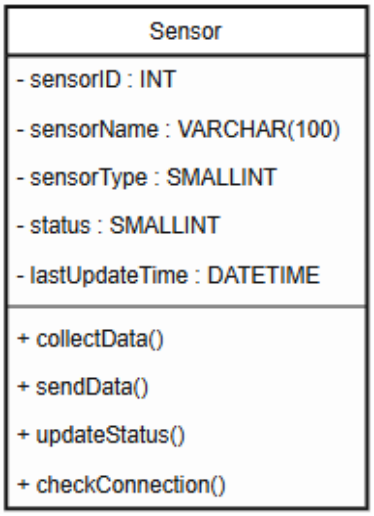
Status Code	Trạng thái
510	Offline
520	Online
530	Bảo trì
540	Lỗi thiết bị

Lớp Sensor không hoạt động độc lập mà tham gia vào chuỗi xử lý dữ liệu của toàn hệ thống. Sensor tạo ra nhiều bản ghi dữ liệu trong lớp Data.

f) Quan hệ với các lớp khác

Lớp Sensor có quan hệ 1–N với lớp Data, trong đó một cảm biến có thể tạo ra nhiều bản ghi dữ liệu theo thời gian, mỗi bản ghi dữ liệu chỉ thuộc về một cảm biến. Sensor gửi dữ liệu đến Server thông qua giao thức MQTT để hệ thống tiếp nhận và xử lý.

g) UML lớp Sensor



Hình 2.47. UML lớp Sensor

2.3.2. Cấu trúc lớp Data

a) Tổng quan

Lớp Data đại diện cho các bản ghi dữ liệu môi trường được tạo ra từ các cảm biến trong nhà kính. Mỗi khi một cảm biến thực hiện phép đo, hệ thống sẽ tạo một đối tượng Data mới để lưu trữ giá trị đo được cùng thời điểm ghi nhận.

Việc tách dữ liệu đo đạc thành một lớp riêng giúp hệ thống dễ dàng lưu trữ lịch sử, truy xuất dữ liệu theo thời gian, xây dựng biểu đồ thống kê và phục vụ các thuật toán điều khiển tự động. Một cảm biến có thể sinh ra nhiều đối tượng Data trong quá trình hoạt động, tạo thành mối quan hệ một-nhiều giữa lớp Sensor và lớp Data.

b) Thuộc tính lớp

Bảng 2.8. Thuộc tính lớp Data

Attribute Code	Tên thuộc tính	Kiểu dữ liệu	Mô tả
100	dataID	INT	Mã định danh bản ghi dữ liệu
200	sensorID	INT	Mã cảm biến tạo ra dữ liệu
300	value	FLOAT	Giá trị đo được
400	timestamp	DATETIME	Thời điểm ghi nhận dữ liệu

c) Chức năng lớp

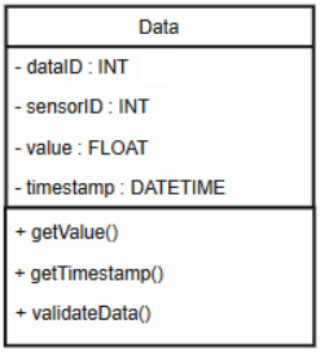
Bảng 2.9. Chức năng lớp Data

Function Code	Tên chức năng	Mô tả
1000	getValue()	Trả về giá trị đo được
1100	getTimestamp()	Trả về thời gian ghi nhận
1200	validateData()	Kiểm tra tính hợp lệ của dữ liệu

d) Quan hệ với các lớp khác

Lớp Data có quan hệ N–1 với lớp Sensor, trong đó mỗi bản ghi dữ liệu được tạo ra bởi một cảm biến cụ thể. Dữ liệu sau khi được Server tiếp nhận sẽ được lưu trữ trong cơ sở dữ liệu và được cung cấp cho Application để phục vụ giám sát, thống kê và tra cứu.

e) UML lớp Data



Hình 2.48. UML lớp Data

2.3.3. Cấu trúc lớp Server & Database

a) Tổng quan

Lớp Server & Database đại diện cho tầng xử lý trung tâm của hệ thống. Lớp này chịu trách nhiệm tiếp nhận dữ liệu từ các cảm biến thông qua giao thức MQTT, xử lý dữ liệu, lưu trữ vào cơ sở dữ liệu, kiểm tra các ngưỡng môi trường và cung cấp dữ liệu cho giao diện người dùng. Ngoài chức năng lưu trữ, Server còn đóng vai trò trung gian giữa các cảm biến, thiết bị đầu ra và hệ thống điều khiển tự động.

b) Thuộc tính lớp

Bảng 2.10. Thuộc tính lớp Server & Database

Attribute Code	Tên thuộc tính	Kiểu dữ liệu	Mô tả
100	serverID	INT	Mã định danh máy chủ
200	serverName	VARCHAR(100)	Tên máy chủ
300	status	SMALLINT	Trạng thái hoạt động
400	databaseName	VARCHAR(100)	Tên cơ sở dữ liệu
500	lastBackupTime	DATETIME	Thời gian sao lưu gần nhất

c) Chức năng lớp

Bảng 2.11. Chức năng lớp Server & Database

Function Code	Tên chức năng	Mô tả
1000	receiveData()	Nhận dữ liệu từ Sensor
1100	storeData()	Lưu dữ liệu vào CSDL
1200	analyzeData()	Phân tích dữ liệu môi trường
1300	generateAlert()	Sinh cảnh báo sự cố
1400	sendCommand()	Gửi lệnh điều khiển thiết bị
1500	provideData()	Cung cấp dữ liệu cho Application

d) Trạng thái lớp

Bảng 2.12. Mã hóa Status lớp Sensor & Database

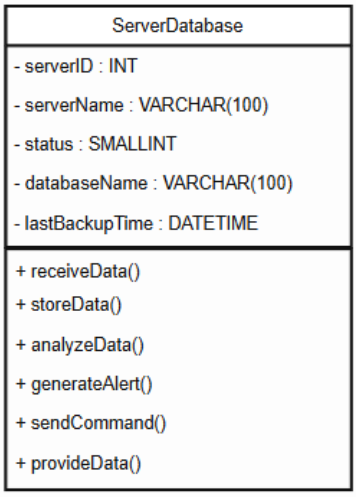
Status Code	Trạng thái
610	Running

620	Stopped
630	Maintenance
640	Error

e) Quan hệ với các lớp khác

Lớp Server & Database đóng vai trò trung gian kết nối giữa Sensor, Data và Application. Server tiếp nhận dữ liệu từ Sensor thông qua MQTT, xử lý và lưu trữ dữ liệu vào Database, đồng thời cung cấp dữ liệu và các chức năng quản lý cho Application thông qua REST API.

f) UML lớp Server & Database



Hình 2.49. UML lớp Server & Database

2.3.4. Cấu trúc lớp Application (Interface)

a) Tổng quan

Lớp Application đại diện cho giao diện tương tác giữa người dùng và hệ thống. Thông qua Application, người dùng có thể giám sát dữ liệu môi trường, quản lý nhà kính, cấu hình công thức sinh trưởng, theo dõi cảnh báo và điều khiển thiết bị khi cần thiết. Application không giao tiếp trực tiếp với cảm biến mà chỉ trao đổi dữ liệu thông qua Server.

b) Thuộc tính lớp

Bảng 2.13. Thuộc tính lớp Application

Attribute Code	Tên thuộc tính	Kiểu dữ liệu	Mô tả
100	applicationID	INT	Mã định danh ứng dụng

200	applicationName	VARCHAR(100)	Tên ứng dụng
300	version	VARCHAR(20)	Phiên bản hệ thống
400	currentUserID	INT	Người dùng hiện tại
500	status	SMALLINT	Trạng thái ứng dụng

c) Chức năng lớp

Bảng 2.14. Chức năng lớp Application

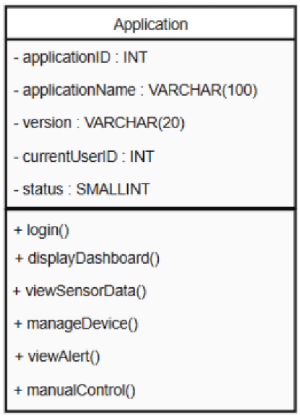
Function Code	Tên chức năng	Mô tả
1000	login()	Đăng nhập hệ thống
1100	displayDashboard()	Hiển thị dữ liệu tổng quan
1200	viewSensorData()	Xem dữ liệu cảm biến
1300	manageDevice()	Quản lý thiết bị
1400	viewAlert()	Xem cảnh báo
1500	manualControl()	Điều khiển thủ công

d) Trạng thái

Bảng 2.15. Mã hóa Status lớp Application

Status Code	Trạng thái
710	Active
720	Inactive
730	Maintenance

e) UML lớp Application



Hình 2.50. UML lớp Application

2.4. Thiết kế giao tiếp hệ thống

Hệ thống sử dụng REST API làm giao tiếp giữa ứng dụng giao diện và máy chủ Backend. Frontend được xây dựng bằng HTML, CSS và JavaScript, gửi các yêu cầu HTTP đến Backend Node.js/Express thông qua các REST API để thực hiện các chức năng quản lý, giám sát và điều khiển. Đối với giao tiếp với các thiết bị IoT, hệ thống định hướng sử dụng giao thức MQTT. Hệ thống đã tích hợp hoàn chỉnh giao thức và Broker MQTT; hạn chế hiện tại chỉ là các node cảm biến đầu cuối đang chạy trên tiến trình giả lập (Simulator) qua MQTT thay vì cắm vi điều khiển phần cứng thật.

2.4.1. Thiết kế REST API

REST API là giao tiếp chính giữa Frontend và Backend của hệ thống. Frontend gửi yêu cầu HTTP đến các endpoint có tiền tố /api, Backend tiếp nhận yêu cầu, xử lý nghiệp vụ và truy xuất hoặc cập nhật dữ liệu trong SQL Server, sau đó trả kết quả dưới dạng JSON.

Các yêu cầu có dữ liệu gửi lên sử dụng Content-Type: application/json. Khi người dùng đăng nhập thành công, Frontend lưu token và gửi token trong HTTP header Authorization cho các yêu cầu tiếp theo.

Bảng 2.16. Xác thực và tài khoản

STT	Chức năng	Method	URL	Request	Response
1	Đăng nhập	POST	/api/auth/login	username, password	Thông tin người dùng và token
2	Đăng xuất	POST	/api/auth/logout	–	{message}
3	Đổi mật khẩu	PUT	/api/auth/change-password	oldPassword, newPassword, confirmPassword	{message}
4	Quên mật khẩu	PUT	/api/auth/forgot-password	identifier, phone, newPassword, confirmPassword	{message}
5	Đăng ký tài khoản	POST	/api/auth/register	username, password, email, phone, role	Thông tin tài khoản mới
6	Lấy thông tin người dùng hiện tại	GET	/api/auth/me	–	Thông tin người dùng

Trong đó, tài khoản đăng ký với vai trò TECHNICIAN hoặc OPERATOR sẽ được tạo ở trạng thái chờ phê duyệt.

Bảng 2.17. Quản lý người dùng

STT	Chức năng	Method	URL	Request	Response
7	Lấy danh sách người dùng	GET	/api/users	–	Danh sách người dùng
8	Cập nhật vai trò	PUT	/api/users/:id/role	{role}	{message}
9	Cập nhật trạng thái	PUT	/api/users/:id/status	{status}	{message}
10	Xóa người dùng	DELETE	/api/users/:id	–	{message}

Các API quản lý người dùng yêu cầu quyền OWNER.

Bảng 2.18. Quản lý cấu trúc nhà kính

STT	Chức năng	Method	URL	Request	Response
11	Lấy danh sách Farm	GET	/api/farms	–	Danh sách Farm
12	Lấy danh sách Greenhouse	GET	/api/greenhouses	–	Danh sách Greenhouse
13	Lấy Greenhouse theo Farm	GET	/api/greenhouses/:farmId	–	Danh sách Greenhouse
14	Lấy cây cấu trúc Farm–Greenhouse–Zone	GET	/api/zones	–	Danh sách cấu trúc
15	Thêm Farm/Greenhouse/Zone	POST	/api/zones	{name, type, parent_id, greenhouse_id, recipe_id, start_date, status}	{id, message}
16	Cập nhật Farm/Greenhouse/Zone	PUT	/api/zones/:id	Thông tin tương ứng với type	{message}
17	Điều chỉnh chu kỳ Zone	PUT	/api/zones/:id/cycle-adjustment	{adjustmentDays, reason}	{message}

18	Xóa Farm/Greenhouse /Zone	DELETE	/api/zones/:id?type=...	–	{message}

API /api/zones sử dụng trường type để xác định đối tượng cần thao tác là farm, greenhouse hay zone.

Bảng 2.19. Quản lý thiết bị và Gateway

STT	Chức năng	Method	URL	Request	Response
19	Lấy danh sách thiết bị	GET	/api/devices	–	Danh sách thiết bị
20	Lấy thông tin thiết bị	GET	/api/devices/:id	–	Thông tin thiết bị
21	Thêm thiết bị	POST	/api/devices	{name, device_type, metric_type, macAddress, zone_id, gateway_id, batteryLevel}	{id, message}
22	Cập nhật thiết bị	PUT	/api/devices/:id	Thông tin thiết bị	{message}
23	Xóa thiết bị	DELETE	/api/devices/:id	–	{message}
24	Lấy thông tin điều khiển	GET	/api/control-properties	–	Danh sách thuộc tính điều khiển
25	Cập nhật chế độ điều khiển	PUT	/api/control-properties/:deviceId	{mode, isActive, valuePercent, autoResetTime}	{message, control}
26	Lấy danh sách Gateway	GET	/api/gateways	–	Danh sách Gateway

Thiết bị được phân thành SENSOR và OUTPUT_DEVICE. Thiết bị đầu ra có các thuộc tính điều khiển gồm AUTO hoặc MANUAL, trạng thái hoạt động và mức công suất từ 0 đến 100%.

Bảng 2.20. Quản lý Recipe, GrowthStage và Threshold

STT	Chức năng	Method	URL	Request	Response
27	Lấy danh sách Recipe	GET	/api/recipes	–	Danh sách Recipe
28	Lấy Recipe	GET	/api/recipes/:id	–	Thông tin Recipe
29	Thêm Recipe	POST	/api/recipes	{name, flower_type, description}	{id, message}
30	Cập nhật Recipe	PUT	/api/recipes/:id	{name, flower_type, description}	{message}
31	Xóa Recipe	DELETE	/api/recipes/:id	–	{message}
32	Lấy GrowthStage	GET	/api/recipes/:recipeId/stages	–	Danh sách GrowthStage
33	Lấy Threshold	GET	/api/stages/:stageId/thresholds	–	Danh sách Threshold
34	Thêm GrowthStage	POST	/api/recipes/:recipeId/stages	{name, start_day, end_day, thresholds}	{id, message}
35	Cập nhật GrowthStage	PUT	/api/stages/:stageId	{name, start_day, end_day, thresholds}	{message}
36	Xóa GrowthStage	DELETE	/api/stages/:stageId	–	{message}

Trong request của GrowthStage, có thresholds là danh sách các ngưỡng môi trường, mỗi phần tử gồm metric_type, min_value và max_value.

Bảng 2.21. Giám sát dữ liệu và cảnh báo

STT	Chức năng	Method	URL	Request	Response
37	Lấy danh sách cảnh báo	GET	/api/alerts	–	Danh sách cảnh báo

38	Cập nhật trạng thái cảnh báo	PUT	/api/alerts/:id/status	{status, acknowledgedBy}	{message}
39	Xóa cảnh báo	DELETE	/api/alerts/:id	–	{message}
40	Lấy Audit Log	GET	/api/logs	–	Danh sách nhật ký
41	Lấy dữ liệu cảm biến	GET	/api/sensor-data	greenhouseId, limit	Danh sách dữ liệu
42	Thống kê theo Greenhouse	GET	/api/stats/greenhouse/:greenhouseId	–	Đối tượng thống kê
43	Thống kê toàn hệ thống	GET	/api/stats/global	–	Đối tượng thống kê

Đối với /api/sensor-data, tham số limit được giới hạn từ 1 đến 1000 và mặc định là 200. Có thể truyền greenhouseId để giới hạn dữ liệu theo nhà kính.

Bảng 2.22. API phục vụ mô phỏng dữ liệu

STT	***Chức năng***	***Method***	***URL***	***Request***	***Response***
44	Xem trạng thái Simulator	GET	/api/simulation/status	–	running, intervalMs, tickNumber, lastTickAt, lastError
45	Khởi động Simulator	POST	/api/simulation/start	{intervalMs} (tùy chọn)	Trạng thái Simulator
46	Dừng Simulator	POST	/api/simulation/stop	–	Trạng thái Simulator
47	Chạy một lượt mô phỏng	POST	/api/simulation/tick	–	{skipped, readings, results}
48	Gửi giá trị cảm biến thử nghiệm	POST	/api/simulation/readings	{deviceId, value, timestamp}	Kết quả xử lý dữ liệu

2.4.2. Thiết kế giao tiếp MQTT

MQTT được lựa chọn làm giao thức giao tiếp giữa thiết bị IoT và Backend trong kiến trúc triển khai thực tế do phù hợp với việc truyền dữ liệu cảm biến và gửi lệnh điều khiển. Tuy nhiên, phiên bản hiện tại của hệ thống chưa tích hợp MQTT. Dữ liệu cảm biến hiện được tạo bởi Backend Simulator và đưa trực tiếp vào cùng luồng xử lý dữ liệu cảm biến. Do đó, các topic và payload dưới đây là thiết kế giao tiếp dự kiến khi tích hợp thiết bị IoT thực tế, không phải các topic đang được sử dụng trong source code hiện tại.

a) MQTT Publish

Thiết bị cảm biến gửi dữ liệu đo được lên MQTT Broker. Backend sẽ đăng ký topic tương ứng để tiếp nhận dữ liệu và chuyển vào luồng xử lý SensorData → Threshold → AlertLog → điều khiển AUTO.

Bảng 2.23. Thiết kế MQTT Publish

STT	Thiết bị gửi	Topic	Payload	Ý nghĩa
1	Sensor	greenhouse/{greenhouseId}/zone/{zoneId}/sensor/{deviceId}/data	JSON	Gửi dữ liệu cảm biến
2	Sensor	greenhouse/{greenhouseId}/zone/{zoneId}/sensor/{deviceId}/status	JSON	Gửi trạng thái thiết bị

Cấu trúc trên tương thích với mô hình dữ liệu của hệ thống, trong đó Device xác định thiết bị và loại thông số đo, còn SensorData lưu device_id, thời điểm đo và giá trị đo.

b) MQTT Subscribe

Backend hoặc Gateway đăng ký các topic điều khiển để nhận lệnh gửi đến thiết bị đầu ra, trong đó thiết bị đầu ra có chế độ AUTO hoặc MANUAL, trạng thái hoạt động và giá trị điều khiển theo phần trăm.

Bảng 2.24. Thiết kế MQTT Subscribe

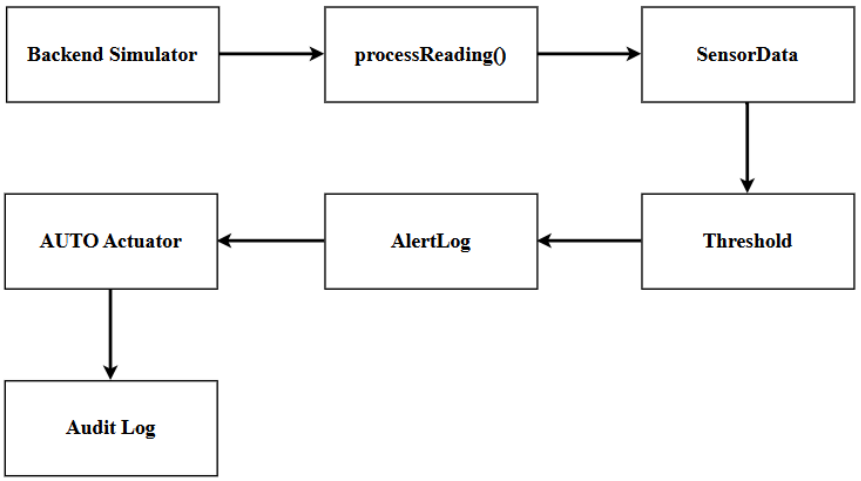
STT	Đối tượng nhận	Topic	Payload	Ý nghĩa
1	Output Device	greenhouse/{greenhouseId}/zone/{zoneId}/device/{deviceId}/command	JSON	Nhận lệnh điều khiển thiết bị
2	Output Device	greenhouse/{greenhouseId}/zone/{zoneId}/device/{deviceId}/config	JSON	Nhận cấu hình điều khiển

Payload phản ánh các thuộc tính điều khiển đang tồn tại trong ControlProperties: mode, isActive, valuePercent và thời gian điều khiển.

2.4.3. Luồng giao tiếp dữ liệu

Để làm rõ mối quan hệ giữa các giao thức giao tiếp và các thành phần của hệ thống, luồng xử lý được xác định như sau:

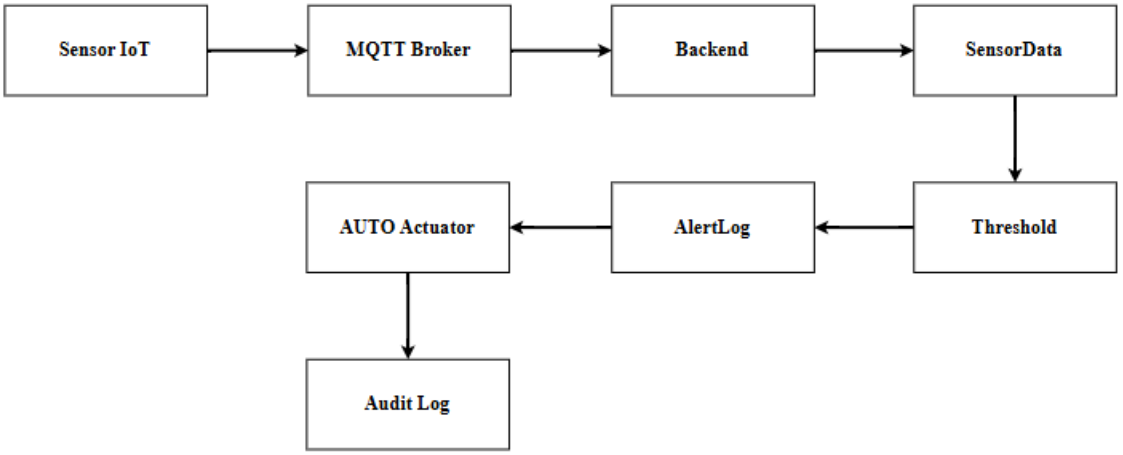
a) Dữ liệu cảm biến hiện tại



Hình 2.51. Luồng dữ liệu cảm biến hiện tại

Simulator định kỳ tạo dữ liệu cho các cảm biến đang hoạt động. Mỗi giá trị được lưu vào SensorData, sau đó được so sánh với Threshold của GrowthStage hiện tại. Nếu dữ liệu vượt ngưỡng, hệ thống tạo hoặc cập nhật AlertLog; đồng thời cơ chế điều khiển tự động có thể thay đổi trạng thái thiết bị đầu ra đang ở chế độ AUTO.

b) Dữ liệu cảm biến sau khi tích hợp MQTT



Hình 2.52. Luồng dữ liệu cảm biến sau khi tích hợp MQTT

Như vậy, MQTT chỉ thay thế nguồn cung cấp dữ liệu cảm biến hiện do Simulator đảm nhiệm; phần xử lý dữ liệu phía Backend có thể tiếp tục sử dụng luồng xử lý hiện có.

2.5. Kết luận chương 2

Chương 2 đã trình bày quá trình phân tích và thiết kế hệ thống quản lý nhà kính trồng hoa, từ việc xác định các tác nhân, chức năng và luồng xử lý đến thiết kế kiến trúc, cơ sở dữ liệu, các lớp hệ thống và giao tiếp giữa các thành phần. Các sơ đồ Use Case, Activity, Sequence, DFD, FHD, ERD và Class Diagram đã mô tả tương đối đầy đủ cấu trúc, chức năng và hoạt động của hệ thống. Bên cạnh đó, thiết kế REST API và giao tiếp MQTT được xác định nhằm phục vụ trao đổi dữ liệu giữa giao diện, máy chủ và các thiết bị IoT. Những kết quả này là cơ sở để tiến hành xây dựng và triển khai hệ thống trong Chương 3.

CHƯƠNG 3. TRIỂN KHAI HỆ THỐNG

Chương này trình bày chi tiết quá trình hiện thực hóa và triển khai hệ thống quản lý nhà kính trồng hoa, bao gồm kiến trúc triển khai, môi trường công nghệ, hiện thực hóa cơ sở dữ liệu trên Microsoft SQL Server (DDL tạo bảng và View), các đoạn mã nguồn xử lý cốt lõi, danh mục REST API, giao thức giao tiếp MQTT, kết quả giao diện chạy thử và đánh giá kiểm thử hệ thống.

3.1. Công nghệ và môi trường triển khai

Bảng 3.1. Tổng hợp các công nghệ, phiên bản và vai trò cụ thể của từng thành phần trong môi trường triển khai

Thành phần	Công nghệ / Nền tảng	Phiên bản	Cổng (Port)	Mục đích sử dụng
Frontend	HTML5, CSS3, Vanilla JS	ES6+	8080 / 5000	Giao diện điều khiển, giám sát Dashboard thời gian thực
Backend	Node.js, Express.js	v18+	5000	Xử lý logic nghiệp vụ, Rule Engine, API Server
MQTT Client	MQTT.js	v5.x	—	Thư viện kết nối, xử lý sự kiện Publish/Subscribe MQTT
MQTT Broker	Eclipse Mosquitto	v2.x	1883	Điều phối tin nhắn thời gian thực giữa thiết bị và server
Cơ sở dữ liệu	Microsoft SQL Server	2019+	1433	Quản trị và lưu trữ cơ sở dữ liệu hệ thống (SmartFarmDB)
DB Driver	mssql	v9.x+	—	Thư viện kết nối và thực thi truy vấn SQL từ Node.js

3.1.1. Hiện thực hóa cấu trúc bảng SQL (DDL Tables)

Dưới đây là mã lệnh T-SQL khởi tạo toàn bộ 13 bảng thực thể trong hệ thống:

Bảng 3.2. User (Tài khoản và phân quyền)

```
CREATE TABLE [User] (
        user_id INT IDENTITY(1,1) PRIMARY KEY,
        user_name NVARCHAR(100) NOT NULL,
        password NVARCHAR(255) NOT NULL,
        email NVARCHAR(100) NOT NULL,
        phone_number NVARCHAR(20),
```

```
        role NVARCHAR(30) NOT NULL CHECK (role IN ('OWNER', 'TECHNICIAN',
'OPERATOR')),
        is_primary_owner BIT NOT NULL DEFAULT 0,
        status NVARCHAR(20) NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE',
'PENDING', 'REJECTED')),
        CONSTRAINT UQ_User_UserName UNIQUE (user_name),
        CONSTRAINT UQ_User_Email UNIQUE (email)
);
```

Bảng 3.3. Farm (Trang trại)

```
CREATE TABLE Farm (
        farm_id INT IDENTITY(1,1) PRIMARY KEY,
        owner_id INT NOT NULL,
        address NVARCHAR(MAX),
        farm_name NVARCHAR(100) NOT NULL,
        FOREIGN KEY (owner_id) REFERENCES [User](user_id)
);
```

Bảng 3.4. Bảng Greenhouse (Nhà kính)

```
CREATE TABLE Greenhouse (
        greenhouse_id INT IDENTITY(1,1) PRIMARY KEY,
        farm_id INT NOT NULL,
        greenhouse_name NVARCHAR(100) NOT NULL,
        location_gps NVARCHAR(100),
        FOREIGN KEY (farm_id) REFERENCES Farm(farm_id)
);
```

Bảng 3.5. Recipe (Công thức sinh trưởng)

```
CREATE TABLE Recipe (
	recipe_id INT IDENTITY(1,1) PRIMARY KEY,
	creator INT NOT NULL,
	recipe_name NVARCHAR(100) NOT NULL,
	flower_type NVARCHAR(100),
	description NVARCHAR(MAX),
```

```
	status NVARCHAR(20) NOT NULL DEFAULT N'active' CHECK (status IN (N'active', N'delayed', N'completed')),
	created_date DATE NOT NULL DEFAULT GETDATE(),
	FOREIGN KEY (creator) REFERENCES [User](user_id)
);
```

Bảng 3.6. Zone (Khu vực trồng)

```
CREATE TABLE Zone (
	zone_id INT IDENTITY(1,1) PRIMARY KEY,
	greenhouse_id INT NOT NULL,
	recipe_id INT NULL,
	zone_name NVARCHAR(100) NOT NULL,
	start_date DATE,
	temperature DECIMAL(5,2) NULL,
	humidity INT NULL,
	status NVARCHAR(20) NULL,
	cycle_adjustment_days INT NOT NULL DEFAULT 0 CHECK (cycle_adjustment_days >= 0),
	adjustment_reason NVARCHAR(255) NULL,
	FOREIGN KEY (greenhouse_id) REFERENCES Greenhouse(greenhouse_id),
	FOREIGN KEY (recipe_id) REFERENCES Recipe(recipe_id)
);
```

Bảng 3.7. Gateway (Trung tâm kết nối IoT)

```
CREATE TABLE Gateway (
	gateway_id INT IDENTITY(1,1) PRIMARY KEY,
	greenhouse_id INT NOT NULL,
```

```
        status NVARCHAR(20) NOT NULL CHECK (status IN ('ONLINE', 'OFFLINE',
'MAINTENANCE')),
        gateway_address NVARCHAR(100),
        FOREIGN KEY (greenhouse_id) REFERENCES Greenhouse(greenhouse_id)
);
```

Bảng 3.8. Device (Cảm biến & Thiết bị đầu ra)

```
CREATE TABLE Device (
        device_id INT IDENTITY(1,1) PRIMARY KEY,
        gateway_id INT NOT NULL,
        zone_id INT NOT NULL,
        device_name NVARCHAR(100) NOT NULL,
        device_type NVARCHAR(20) NOT NULL CHECK (device_type IN ('SENSOR',
'OUTPUT_DEVICE')),
        metric_type NVARCHAR(50) NOT NULL,
        mac_address NVARCHAR(17) NULL,
        battery_level INT NULL,
        status NVARCHAR(20) NOT NULL DEFAULT 'ONLINE' CHECK (status IN ('ONLINE',
'OFFLINE', 'ERROR', 'NEEDS_REPLACEMENT')),
        last_heartbeat DATETIME2 NOT NULL DEFAULT GETDATE(),
        CHECK (battery_level IS NULL OR battery_level BETWEEN 0 AND 100),
        FOREIGN KEY (gateway_id) REFERENCES Gateway(gateway_id),
        FOREIGN KEY (zone_id) REFERENCES Zone(zone_id)
);
```

Bảng 3.9. Bảng ControlProperties (Thuộc tính điều khiển chấp hành)

```
CREATE TABLE ControlProperties (
        device_id INT PRIMARY KEY,
        mode NVARCHAR(10) NOT NULL CHECK (mode IN ('AUTO', 'MANUAL')),
        is_active BIT NOT NULL DEFAULT 0,
        value_percent INT NOT NULL DEFAULT 0 CHECK (value_percent BETWEEN 0 AND
100),
        auto_reset_time DATETIME2 NULL,
        FOREIGN KEY (device_id) REFERENCES Device(device_id) ON DELETE CASCADE
);
```

Bảng 3.10. GrowthStage (Giai đoạn sinh trưởng)

```
CREATE TABLE GrowthStage (
        stage_id INT IDENTITY(1,1) PRIMARY KEY,
        recipe_id INT NOT NULL,
        stage_name NVARCHAR(100) NOT NULL,
        start_day INT NOT NULL,
        end_day INT NOT NULL,
        completed BIT NOT NULL DEFAULT 0,
        current_day INT NULL,
        CHECK (start_day >= 0 AND end_day >= start_day),
        CHECK (current_day IS NULL OR current_day >= 0),
        FOREIGN KEY (recipe_id) REFERENCES Recipe(recipe_id)
);
```

Bảng 3.11. Bảng Threshold (Ngưỡng môi trường theo giai đoạn)

```
CREATE TABLE Threshold (
        threshold_id INT IDENTITY(1,1) PRIMARY KEY,
        stage_id INT NOT NULL,
        metric_type NVARCHAR(50) NOT NULL,
        min_value DECIMAL(10,2),
        max_value DECIMAL(10,2),
        CHECK (min_value IS NULL OR max_value IS NULL OR min_value <= max_value),
        FOREIGN KEY (stage_id) REFERENCES GrowthStage(stage_id)
);
```

3.12. Bảng SensorData (Dữ liệu chuỗi thời gian)

```
CREATE TABLE SensorData (
        device_id INT NOT NULL,
        [timestamp] DATETIME2 NOT NULL,
        raw_value DECIMAL(10,2) NOT NULL,
        PRIMARY KEY (device_id, [timestamp]),
        FOREIGN KEY (device_id) REFERENCES Device(device_id)
);
```

Bảng 3.13. Bảng Log (Nhật ký kiểm toán hệ thống)

```
CREATE TABLE Log (
	log_id INT IDENTITY(1,1) PRIMARY KEY,
	device_id INT NULL,
	user_id INT NULL,
	entity_type NVARCHAR(20) NOT NULL DEFAULT 'DEVICE' CHECK (entity_type IN
('DEVICE', 'ZONE', 'GREENHOUSE', 'FARM', 'RECIPE', 'GROWTH_STAGE', 'ALERT', 'USER',
'SIMULATION')),
	entity_id INT NULL,
	action NVARCHAR(255) NOT NULL,
	triggered_by NVARCHAR(10) NOT NULL CHECK (triggered_by IN ('USER',
'SYSTEM')),
	log_time DATETIME2 NOT NULL DEFAULT GETDATE(),
	FOREIGN KEY (device_id) REFERENCES Device(device_id) ON DELETE SET NULL,
	FOREIGN KEY (user_id) REFERENCES [User](user_id),
	CHECK ((triggered_by = 'SYSTEM' AND user_id IS NULL) OR (triggered_by = 'USER'
AND user_id IS NOT NULL))
);
```

Bảng 3.14. Bảng AlertLog (Quản lý cảnh báo sự cố)

```
CREATE TABLE AlertLog (
	alert_id INT IDENTITY(1,1) PRIMARY KEY,
	zone_id INT NOT NULL,
	message NVARCHAR(MAX) NOT NULL,
	severity NVARCHAR(20) NOT NULL CHECK (severity IN ('SIGNIFICANT',
'WARNING', 'CRITICAL')),
	status NVARCHAR(20) NOT NULL DEFAULT 'UNSOLVED' CHECK (status IN
('UNSOLVED', 'ACKNOWLEDGED', 'RESOLVED')),
	created_at DATETIME2 DEFAULT GETDATE(),
	resolved_at DATETIME2 NULL,
	acknowledged_by NVARCHAR(100) NULL,
	escalation_level INT NOT NULL DEFAULT 0,
	FOREIGN KEY (zone_id) REFERENCES Zone(zone_id)
);
```

3.1.2. Hiện thực hóa các khung nhìn (Database Views)

Dưới đây là 9 View phục vụ tối ưu hóa truy vấn dữ liệu:

Bảng 3.15. ViewUserRole (Thông tin và vai trò người dùng)

```
CREATE VIEW [dbo].[ViewUserRole] AS
SELECT u.user_id, u.user_name, u.email, u.phone_number, u.role, u.is_primary_owner, u.status
FROM [User] u;
```

Bảng 3.16. ViewFarmGreenhouse (Cấu trúc Trang trại & Nhà kính)

```
CREATE VIEW [dbo].[ViewFarmGreenhouse] AS
SELECT f.farm_id, f.farm_name, f.address, u.user_name AS owner_name, g.greenhouse_id,
g.greenhouse_name, g.location_gps
FROM Farm f
JOIN [User] u ON f.owner_id = u.user_id
JOIN Greenhouse g ON f.farm_id = g.farm_id;
```

Bảng 3.17. ViewZoneOverview (Tổng quan khu vực trồng)

```
CREATE VIEW [dbo].[ViewZoneOverview] AS
SELECT z.zone_id, z.zone_name, g.greenhouse_name, f.farm_name, r.recipe_name, r.flower_type,
z.start_date, z.temperature, z.humidity, z.status, z.cycle_adjustment_days, z.adjustment_reason
FROM Zone z
JOIN Greenhouse g ON z.greenhouse_id = g.greenhouse_id
JOIN Farm f ON g.farm_id = f.farm_id
LEFT JOIN Recipe r ON z.recipe_id = r.recipe_id;
```

Bảng 3.18. ViewDeviceStatus (Tình trạng thiết bị & Gateway)

```
CREATE VIEW [dbo].[ViewDeviceStatus] AS
SELECT d.device_id, d.device_name, d.device_type, d.metric_type, d.mac_address, d.battery_level,
d.status AS device_status, d.last_heartbeat, g.gateway_id, g.status AS gateway_status,
gh.greenhouse_name, z.zone_name
FROM Device d
JOIN Gateway g ON d.gateway_id = g.gateway_id
JOIN Greenhouse gh ON g.greenhouse_id = gh.greenhouse_id
JOIN Zone z ON d.zone_id = z.zone_id;
```

Bảng 3.19. ViewControlDevice (Điều khiển thiết bị chấp hành)

```
CREATE VIEW [dbo].[ViewControlDevice] AS
SELECT d.device_id, d.device_name, d.metric_type, d.status AS device_status, cp.mode,
cp.is_active, cp.value_percent, cp.auto_reset_time, z.zone_name, gh.greenhouse_name
FROM Device d
JOIN ControlProperties cp ON d.device_id = cp.device_id
JOIN Zone z ON d.zone_id = z.zone_id
JOIN Greenhouse gh ON z.greenhouse_id = gh.greenhouse_id;
```

Bảng 3.20. ViewLatestSensorData (Dữ liệu cảm biến mới nhất)

```
CREATE VIEW [dbo].[ViewLatestSensorData] AS
WITH LatestData AS (
        SELECT sd.device_id, sd.[timestamp], sd.raw_value,
        ROW_NUMBER() OVER (PARTITION BY sd.device_id ORDER BY sd.[timestamp]
DESC) AS rn
        FROM SensorData sd
)
SELECT d.device_id, d.device_name, d.metric_type, ld.[timestamp], ld.raw_value, z.zone_id,
z.zone_name, gh.greenhouse_id, gh.greenhouse_name
FROM LatestData ld
JOIN Device d ON ld.device_id = d.device_id
JOIN Zone z ON d.zone_id = z.zone_id
JOIN Greenhouse gh ON z.greenhouse_id = gh.greenhouse_id
WHERE ld.rn = 1;
```

Bảng 3.21. ViewThreshold (Tổng hợp ngưỡng sinh trưởng)

```
CREATE VIEW [dbo].[ViewThreshold] AS
SELECT t.threshold_id, r.recipe_id, r.recipe_name, gs.stage_id, gs.stage_name, gs.start_day,
gs.end_day, gs.current_day, t.metric_type, t.min_value, t.max_value
FROM Threshold t
JOIN GrowthStage gs ON t.stage_id = gs.stage_id
JOIN Recipe r ON gs.recipe_id = r.recipe_id;
```

Bảng 3.22. ViewAlertHistory (Lịch sử cảnh báo sự cố)

```
CREATE VIEW [dbo].[ViewAlertHistory] AS
SELECT a.alert_id, a.zone_id, z.zone_name, gh.greenhouse_name, a.message, a.severity, a.status,
```

```
a.created_at, a.resolved_at, a.acknowledged_by, a.escalation_level
FROM AlertLog a
JOIN Zone z ON a.zone_id = z.zone_id
JOIN Greenhouse gh ON z.greenhouse_id = gh.greenhouse_id;
```

Bảng 3.23. ViewSystemLog (Truy vết Audit Log)

```
CREATE VIEW [dbo].[ViewSystemLog] AS
SELECT l.log_id, l.log_time, l.entity_type, l.entity_id, l.action, l.triggered_by, u.user_name,
d.device_name, z.zone_name
FROM [Log] l
LEFT JOIN [User] u ON l.user_id = u.user_id
LEFT JOIN Device d ON l.device_id = d.device_id
LEFT JOIN Zone z ON d.zone_id = z.zone_id;
```

3.1.3. Cài đặt các thủ tục lưu trữ (Stored Procedures Implementation

Các Stored Procedure được thiết kế để đóng gói logic xử lý dữ liệu phức tạp tại máy chủ CSDL, đảm bảo toàn vẹn giao dịch (Transaction) và nâng cao hiệu năng hệ thống.

Bảng 3.24. Thủ tục ghi nhận dữ liệu cảm biến (sp_RecordSensorReading)

```
CREATE PROCEDURE [dbo].[sp_RecordSensorReading]
        @DeviceId INT,
        @RawValue DECIMAL(10,2),
        @ReadingTime DATETIME2
AS
BEGIN
        SET NOCOUNT ON;
        BEGIN TRANSACTION;
        BEGIN TRY
        -- 1. Thêm bản ghi dữ liệu chuỗi thời gian
        INSERT INTO SensorData (device_id, [timestamp], raw_value)
        VALUES (@DeviceId, @ReadingTime, @RawValue);

        -- 2. Cập nhật nhịp tim và trạng thái thiết bị
        UPDATE Device
        SET last_heartbeat = @ReadingTime, status = 'ONLINE'
```

```
        WHERE device_id = @DeviceId;

        IF @MetricType = 'Temperature'
        UPDATE Zone SET temperature = @RawValue WHERE zone_id = @ZoneId;
        ELSE IF @MetricType = 'Humidity'
        UPDATE Zone SET humidity = CAST(@RawValue AS INT) WHERE zone_id =
@ZoneId;

        COMMIT TRANSACTION;
        END TRY
        BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
        END CATCH
END;
```

Bảng 3.25. Thủ tục kiểm tra ngưỡng và kích hoạt cảnh báo (sp_EvaluateThresholdAndAlert)

```
CREATE PROCEDURE [dbo].[sp_EvaluateThresholdAndAlert]
        @ZoneId INT,
        @MetricType NVARCHAR(50),
        @RawValue DECIMAL(10,2)
AS
BEGIN
        SET NOCOUNT ON;
        DECLARE @MinValue DECIMAL(10,2), @MaxValue DECIMAL(10,2);

        -- Lấy dải ngưỡng của giai đoạn đang kích hoạt
        SELECT TOP 1 @MinValue = t.min_value, @MaxValue = t.max_value
        FROM Threshold t
        JOIN GrowthStage gs ON t.stage_id = gs.stage_id
        JOIN Zone z ON z.recipe_id = gs.recipe_id
        WHERE z.zone_id = @ZoneId AND t.metric_type = @MetricType AND gs.completed = 0;

        IF @MinValue IS NOT NULL AND @MaxValue IS NOT NULL
        BEGIN
        IF @RawValue < @MinValue OR @RawValue > @MaxValue
```

```
        BEGIN
        DECLARE @Severity NVARCHAR(20) = 'WARNING';
        IF (@RawValue < @MinValue * 0.75 OR @RawValue > @MaxValue * 1.25)
        SET @Severity = 'CRITICAL';

        DECLARE @Msg NVARCHAR(MAX) = N'[AUTO:' + @MetricType + N'] Chỉ số '
        + CAST(@RawValue AS NVARCHAR(20)) + N' vượt ngưỡng ['
        + CAST(@MinValue AS NVARCHAR(20)) + N' - ' + CAST(@MaxValue AS
NVARCHAR(20)) + N']';

        INSERT INTO AlertLog (zone_id, message, severity, status, created_at)
        VALUES (@ZoneId, @Msg, @Severity, 'UNSOLVED', GETDATE());
        END
        END
END;
```

Bảng 3.26. Thủ tục điều khiển thiết bị chấp hành và ghi log (sp_UpdateDeviceControl)

```
CREATE PROCEDURE [dbo].[sp_UpdateDeviceControl]
        @DeviceId INT,
        @Mode NVARCHAR(10),
        @IsActive BIT,
        @ValuePercent INT,
        @DurationMinutes INT,
        @UserId INT
AS
BEGIN
        SET NOCOUNT ON;
        BEGIN TRANSACTION;
        BEGIN TRY
        DECLARE @AutoReset DATETIME2 = NULL;
        IF @Mode = 'MANUAL' AND @DurationMinutes > 0
        SET @AutoReset = DATEADD(MINUTE, @DurationMinutes, GETDATE());

        -- Cập nhật cấu hình điều khiển
        UPDATE ControlProperties
```

```
        SET mode = @Mode,
        is_active = @IsActive,
        value_percent = @ValuePercent,
        auto_reset_time = @AutoReset
        WHERE device_id = @DeviceId;

        -- Ghi nhận lịch sử vào Audit Log
        DECLARE @ActionMsg NVARCHAR(255) = N'Cập nhật điều khiển mode ' + @Mode
        + N', Công suất: ' + CAST(@ValuePercent AS NVARCHAR(10)) + N'%';

        INSERT INTO [Log] (device_id, user_id, entity_type, entity_id, action, triggered_by,
log_time)
        VALUES (@DeviceId, @UserId, 'DEVICE', @DeviceId, @ActionMsg, 'USER',
GETDATE());

        COMMIT TRANSACTION;
        END TRY
        BEGIN CATCH
        ROLLBACK TRANSACTION;
        THROW;
        END CATCH
END;
```

Bảng 3.27. Thủ tục xử lý xác nhận cảnh báo sự cố (sp_ResolveAlert)

```
CREATE PROCEDURE [dbo].[sp_ResolveAlert]
        @AlertId INT,
        @Status NVARCHAR(20),
        @AcknowledgedBy NVARCHAR(100)
AS
BEGIN
        SET NOCOUNT ON;
```

```
        UPDATE AlertLog
        SET status = @Status,
        acknowledged_by = @AcknowledgedBy,
        resolved_at = CASE WHEN @Status = 'RESOLVED' THEN GETDATE() ELSE
resolved_at END
        WHERE alert_id = @AlertId;
END;
```

3.2. Một số mã nguồn triển khai

Dưới đây là 3 đoạn mã nguồn thể hiện các tiến trình nghiệp vụ cốt lõi:

Bảng 3.28. Mã nguồn xử lý nhận gói tin cảm biến và phát lệnh điều khiển qua MQTT (mqtt-service.js)

```
// smartfarm-api/mqtt-service.js
async handleMessage(topic, payload) {
 try {
  const match = topic.match(SENSOR_TOPIC_PATTERN);
  if (!match) throw new Error("Topic cảm biến không hợp lệ");
  const [, greenhouseIdText, zoneIdText, deviceIdText] = match;
  const greenhouseId = Number(greenhouseIdText);
  const zoneId = Number(zoneIdText);
  const deviceId = Number(deviceIdText);
  const data = JSON.parse(payload.toString("utf8"));
  const value = Number(data.value);
  if (!Number.isFinite(value)) throw new Error("Payload phải có value dạng số");

  const result = await this.simulator.processReading(
   deviceId,
   value,
   data.timestamp || null,
   "MQTT",
   { greenhouseId, zoneId }
  );
  for (const control of result.controls || []) {
   this.publishCommand({ greenhouseId, zoneId, ...control });
  }
  console.log(`📥 MQTT result.metricType ={result.value} tại Zone #${zoneId}`);
```

```
  } catch (error) {
    console.error(`❌ Bỏ qua MQTT message '${topic}':`, error.message);
  }
}
```

Bảng 3.29. Mã nguồn đồng bộ AlertLog và đánh giá mức độ nghiêm trọng (simulation.js)

```
// smartfarm-api/simulation.js
async syncAlert(pool, context, value, direction, source) {
  if (context.minValue === null || context.minValue === undefined) return null;
  const marker = `[AUTO:${context.metricType}]`;
  const activeResult = await pool
    .request()
    .input("zoneId", this.sql.Int, context.zoneId)
    .input("marker", this.sql.NVarChar, marker).query(`
      SELECT TOP 1 alert_id FROM AlertLog
      WHERE zone_id = @zoneId AND LEFT(message, LEN(@marker)) = @marker AND status <>
'RESOLVED'
      ORDER BY created_at DESC
    `);

  if (direction === "normal") {
    if (activeResult.recordset.length > 0) {
      await pool.request()
        .input("id", this.sql.Int, activeResult.recordset[0].alert_id)
        .query(`UPDATE AlertLog SET status = 'RESOLVED', resolved_at = GETDATE() WHERE
alert_id = @id`);
      return { action: "resolved", id: activeResult.recordset[0].alert_id };
    }
    return null;
  }

  const severity = severityFor(value, context.minValue, context.maxValue, direction);
  const limit = direction === "low" ? context.minValue : context.maxValue;
  const message = `${marker} ${context.deviceName} tại ${context.zoneName}: ${value}
${direction === "low" ? "thấp hơn" : "cao hơn"} ngưỡng ${limit}. Nguồn: ${source}.`;

  if (activeResult.recordset.length > 0) {
```

```
    await pool.request()
      .input("id", this.sql.Int, activeResult.recordset[0].alert_id)
      .input("message", this.sql.NVarChar, message)
      .input("severity", this.sql.NVarChar, severity)
      .query(`UPDATE AlertLog SET message = @message, severity = @severity,
            escalation_level = CASE WHEN DATEDIFF(MINUTE, created_at, GETDATE()) >= 10
THEN 2 ELSE 1 END
            WHERE alert_id = @id`);
    return { action: "updated", id: activeResult.recordset[0].alert_id, severity };
  }

  const inserted = await pool.request()
    .input("zoneId", this.sql.Int, context.zoneId)
    .input("message", this.sql.NVarChar, message)
    .input("severity", this.sql.NVarChar, severity)
    .query(`INSERT INTO AlertLog (zone_id, message, severity, status, escalation_level)
          OUTPUT INSERTED.alert_id AS id
          VALUES (@zoneId, @message, @severity, 'UNSOLVED', 1)`);
  return { action: "created", id: inserted.recordset[0].id, severity };
}
```

Bảng 3.30. Mã nguồn Endpoint cập nhật thuộc tính điều khiển và xác thực vai trò (server.js)

```
// smartfarm-api/server.js
app.put("/api/control-properties/:deviceId", async (req, res) => {
  try {
    const { deviceId } = req.params;
    const { mode, isActive, valuePercent, autoResetTime } = req.body;
    if (!["AUTO", "MANUAL"].includes(mode)) {
      return res.status(400).json({ error: "Chế độ chỉ có thể là AUTO hoặc MANUAL" });
    }
    const parsedValue = Number(valuePercent);
    const pool = await getConnection();
    const actorUserId = getTokenUserId(req);
    if (!actorUserId) return res.status(401).json({ error: "Phiên đăng nhập không hợp lệ" });

    const result = await pool.request()
      .input("deviceId", sql.Int, deviceId)
```

```
      .input("mode", sql.NVarChar, mode)
      .input("isActive", sql.Bit, isActive)
      .input("valuePercent", sql.Int, parsedValue)
      .input("autoResetTime", sql.DateTime2, mode === "MANUAL" ? autoResetTime : null).query(`
        UPDATE ControlProperties SET
            mode = @mode,
            is_active = @isActive,
            value_percent = @valuePercent,
            auto_reset_time = @autoResetTime
        WHERE device_id = @deviceId
      `);

    await writeAuditLog(pool, {
      userId: actorUserId,
      action: `Điều khiển thiết bị #
deviceId: mode ={mode}, trạng thái=isActive? "bật": "tắt", mức ={parsedValue}%.`,
      entityType: "DEVICE",
      entityId: Number(deviceId),
      deviceId: Number(deviceId)
    });
    res.json({ message: "Cập nhật control property thành công" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

3.3. API và giao tiếp MQTT

Bảng 3.31. liệt kê 10 REST API tiêu biểu và Bảng 3.32. định nghĩa cấu trúc Topic MQTT:

Bảng 3.31. Danh mục 10 REST API tiêu biểu trong hệ thống

STT	Method	Endpoint URL	Phân quyền	Chức năng chính
1	POST	/api/auth/login	Public	Xác thực đăng nhập, kiểm tra trạng thái tài khoản
2	GET	/api/zones	All Roles	Lấy danh sách cây cấu trúc Farm → Greenhouse → Zone

3	POST	/api/zones	OWNER	Thêm mới trang trại, nhà kính hoặc khu vực trồng
4	GET	/api/devices	All Roles	Truy vấn toàn bộ danh sách thiết bị cảm biến và thiết bị đầu ra
5	PUT	/api/control-properties/:id	OPERATOR, OWNER	Thiết lập trạng thái bật/tắt, công suất (%) và thời gian chạy
6	POST	/api/recipes	TECHNICIAN	Thiết lập công thức sinh trưởng cho loại hoa cụ thể
7	GET	/api/sensor-data	All Roles	Truy xuất dữ liệu chuỗi thời gian để hiển thị đồ thị Dashboard
8	GET	/api/alerts	All Roles	Lấy danh sách cảnh báo sự cố môi trường và thiết bị
9	PUT	/api/alerts/:id/status	OPERATOR, OWNER	Cập nhật tiến độ xử lý cảnh báo (ACKNOWLEDGED / RESOLVED)
10	GET	/api/logs	OWNER	Truy vấn nhật ký kiểm toán hệ thống (Audit Log)

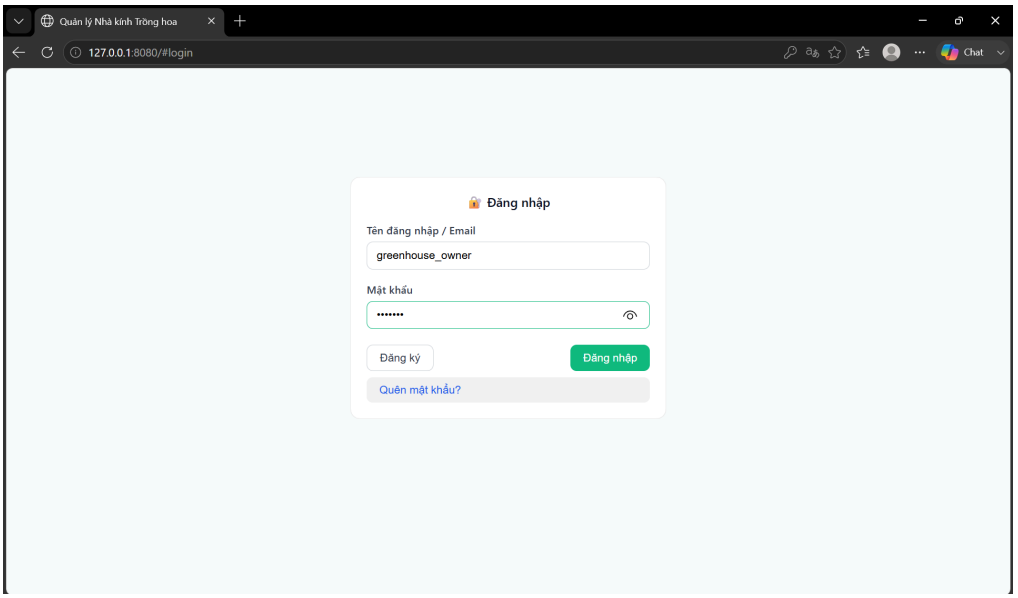
Bảng 3.32. Cấu trúc Topic và định dạng Payload MQTT

Giao thức	Topic Pattern	Payload JSON mẫu	Mô tả chức năng
Publish (Sensor → Broker)	greenhouse/{gId}/zone/{zId}/sensor/{dId}/data	{"deviceId": 1, "metric": "Temperature", "value": 26.5, "timestamp": "2026-08-22T08:00:00Z" }	Gửi định kỳ dữ liệu đo từ các đầu đo cảm biến
Publish (Sensor → Broker)	greenhouse/{gId}/zone/{zId}/sensor/{dId}/status	{"deviceId": 1, "status": "ONLINE", "batteryLevel": 85}	Cập nhật nhịp tim và tình trạng pin của thiết bị
Subscribe (Backend → Actuator)	greenhouse/{gId}/zone/{zId}/actuator/{dId}/command	{"deviceId": 5, "action": "SET_POWER", "valuePercent": 100, "mode": "AUTO"}	Phát lệnh tự động hoặc thủ công tới thiết bị chấp hành

3.4. Kết quả triển khai và chạy thử

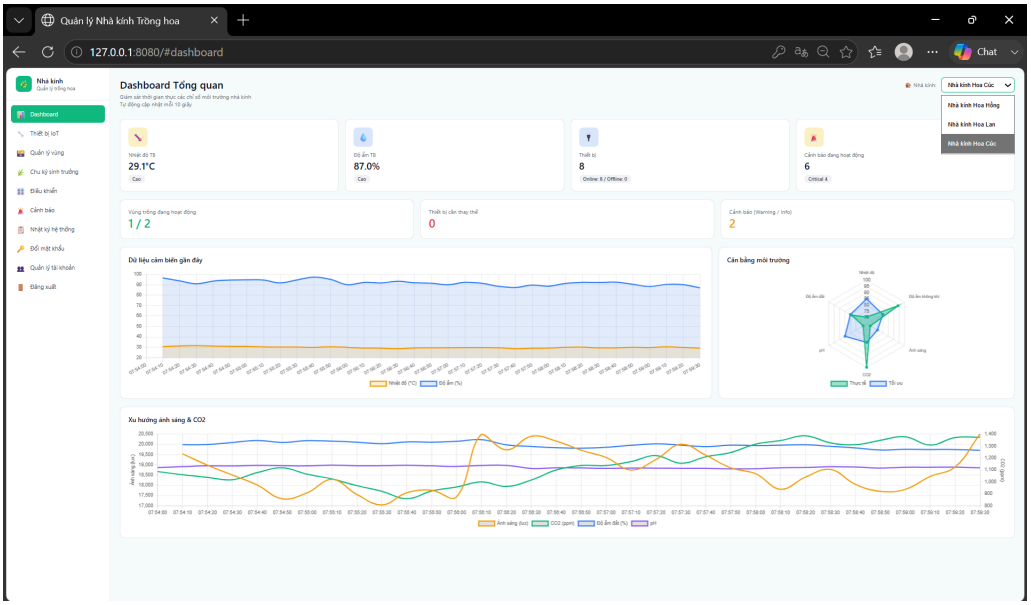
Hệ thống đã được lập trình hoàn chỉnh và thử nghiệm thực tế với các màn hình chức năng hoạt động đồng bộ:

Màn hình đăng nhập & Xác thực: Hỗ trợ đăng nhập, đăng ký và tự động phân luồng chức năng theo vai trò người dùng (OWNER, TECHNICIAN, OPERATOR).



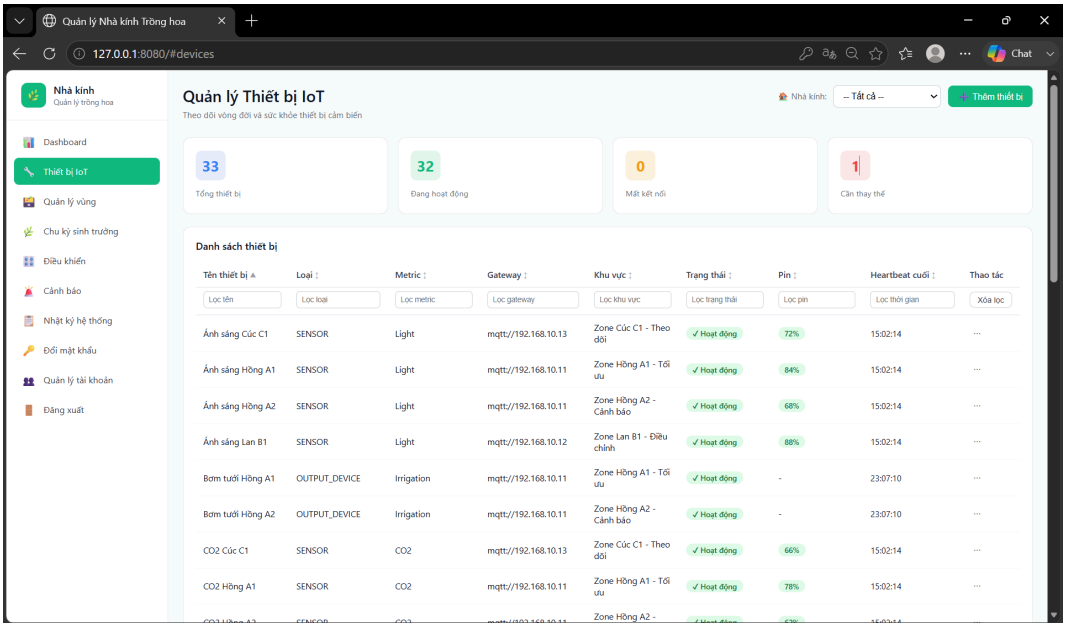
Hình 3.1. Giao diện đăng nhập hệ thống

Màn hình Dashboard tổng quan: Trực quan hóa dữ liệu đo đạc theo thời gian thực với biểu đồ đa trục (Nhiệt độ/Độ ẩm, Ánh sáng/CO_2) và đồ thị Radar cân bằng môi trường.

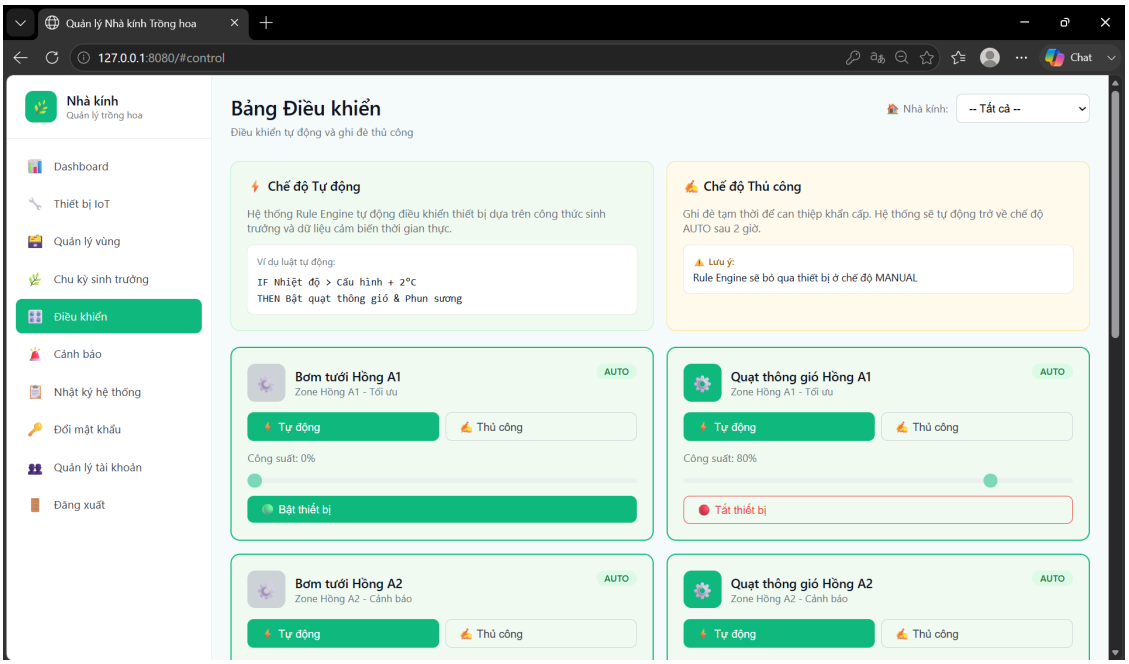


Hình 3.2. Màn hình Dashboard theo dõi các chỉ số môi trường

Quản lý thiết bị & Bảng điều khiển: Danh sách thiết bị phân loại rõ ràng (Sensor/Output Device), hiển thị chi tiết phần trăm pin, trạng thái trực tuyến và bảng điều khiển AUTO/MANUAL.

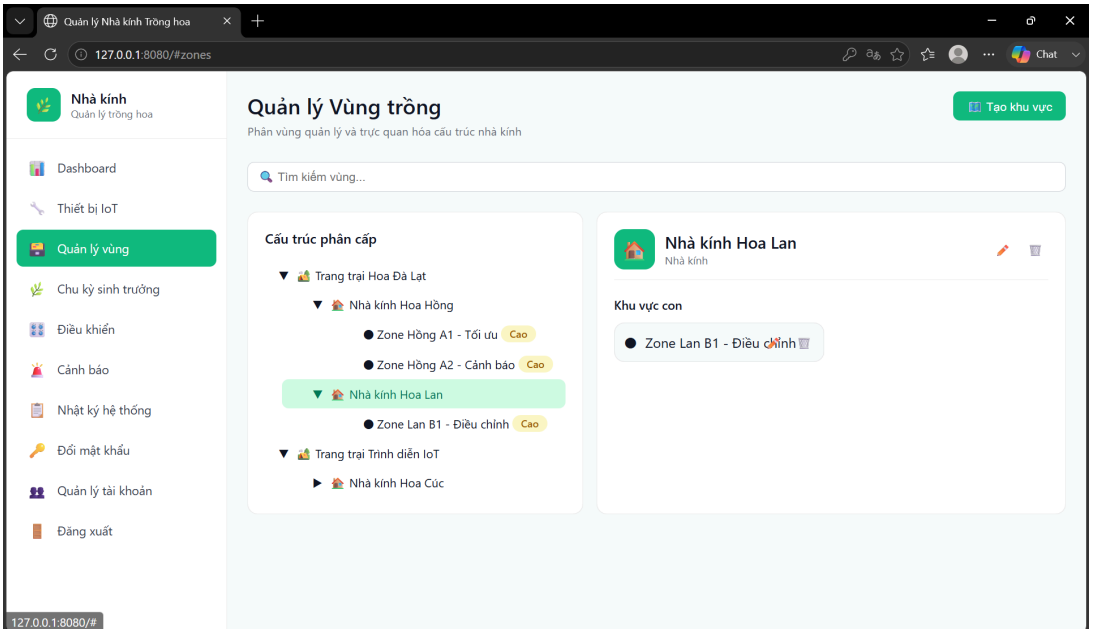


Hình 3.3. Màn hình Quản lý danh sách thiết bị IoT

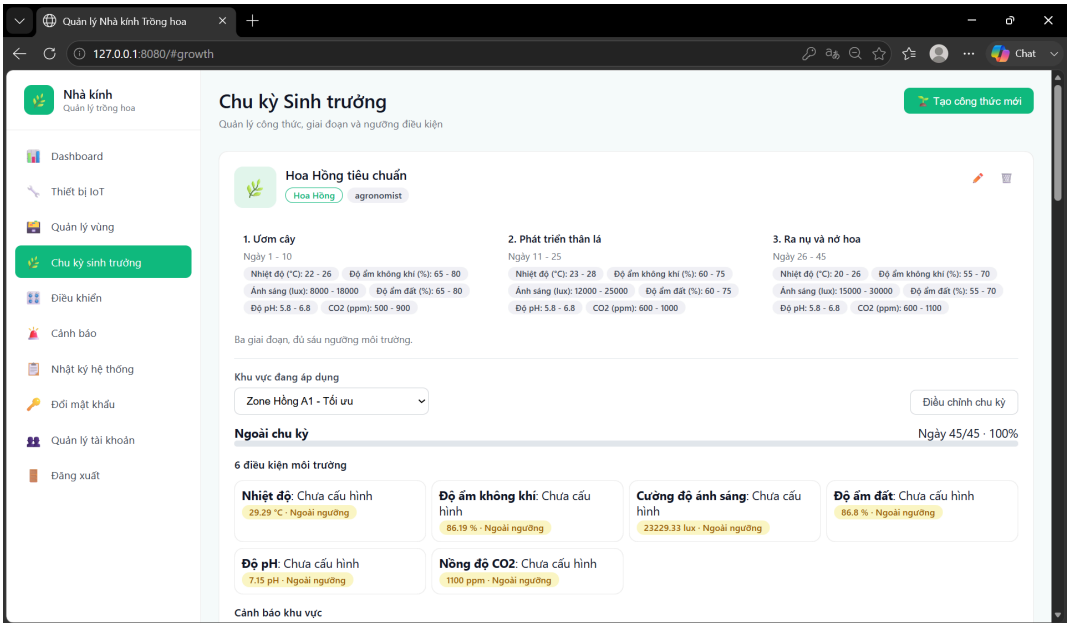


Hình 3.4. Giao diện Bảng điều khiển thiết bị chấp hành

Quản lý phân vùng & Chu kỳ sinh trưởng: Cho phép tùy biến công thức sinh trưởng cho từng giống hoa và áp dụng linh hoạt cho từng khu vực nhà kính.

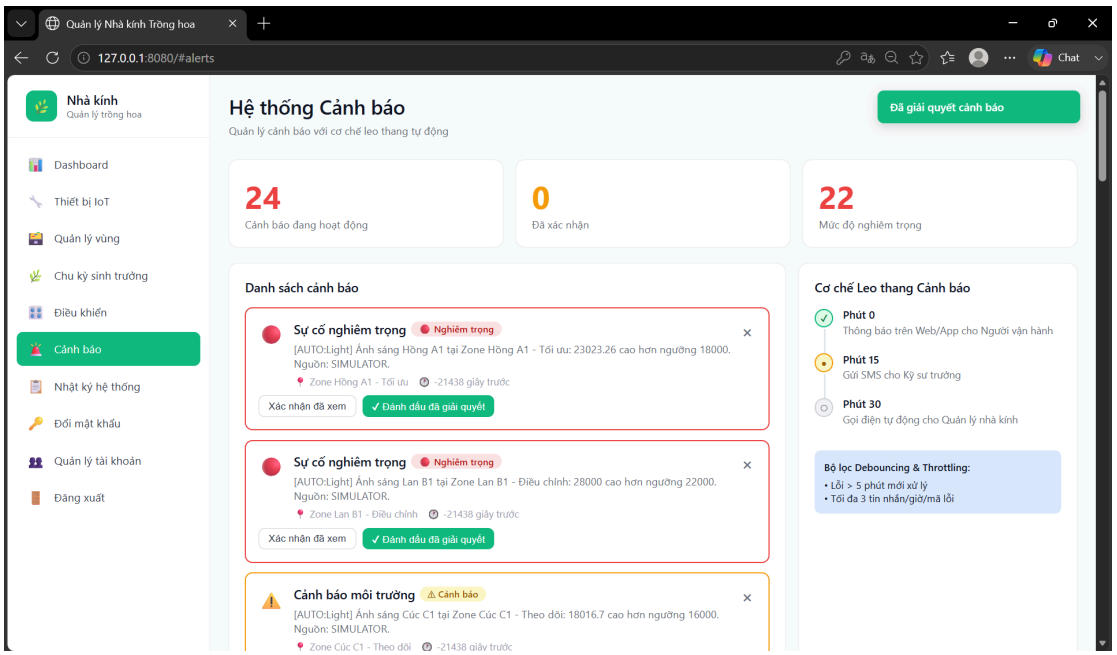


Hình 3.5. Giao diện Quản lý phân vùng trồng

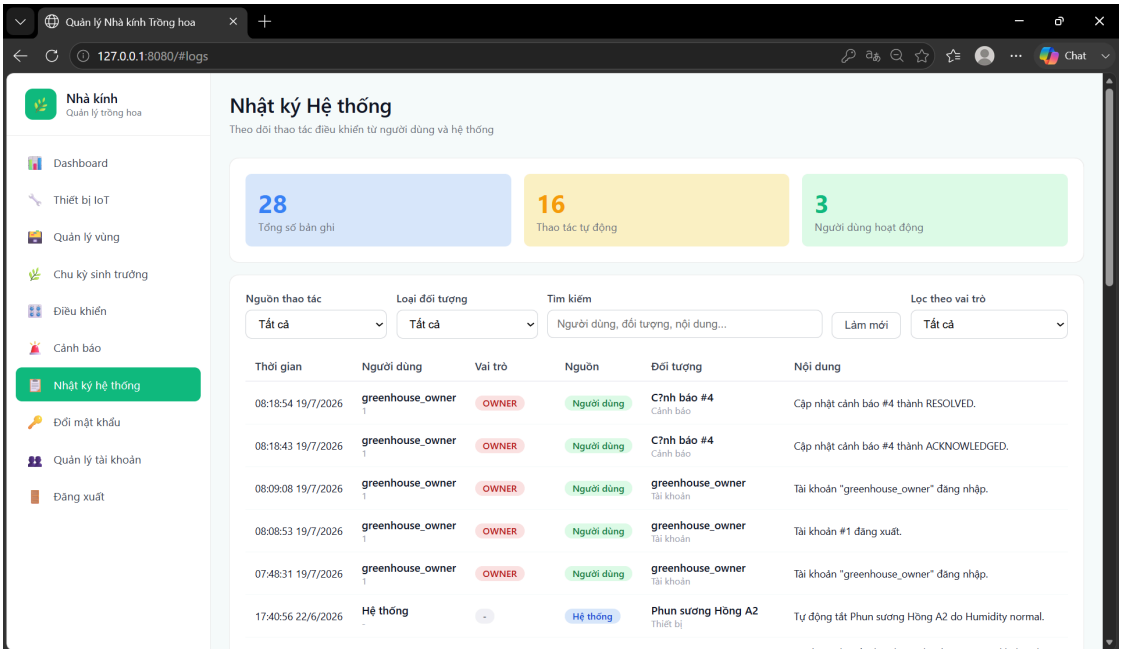


Hình 3.6. Màn hình Cấu hình chu kỳ sinh trưởng & ngưỡng giới hạn

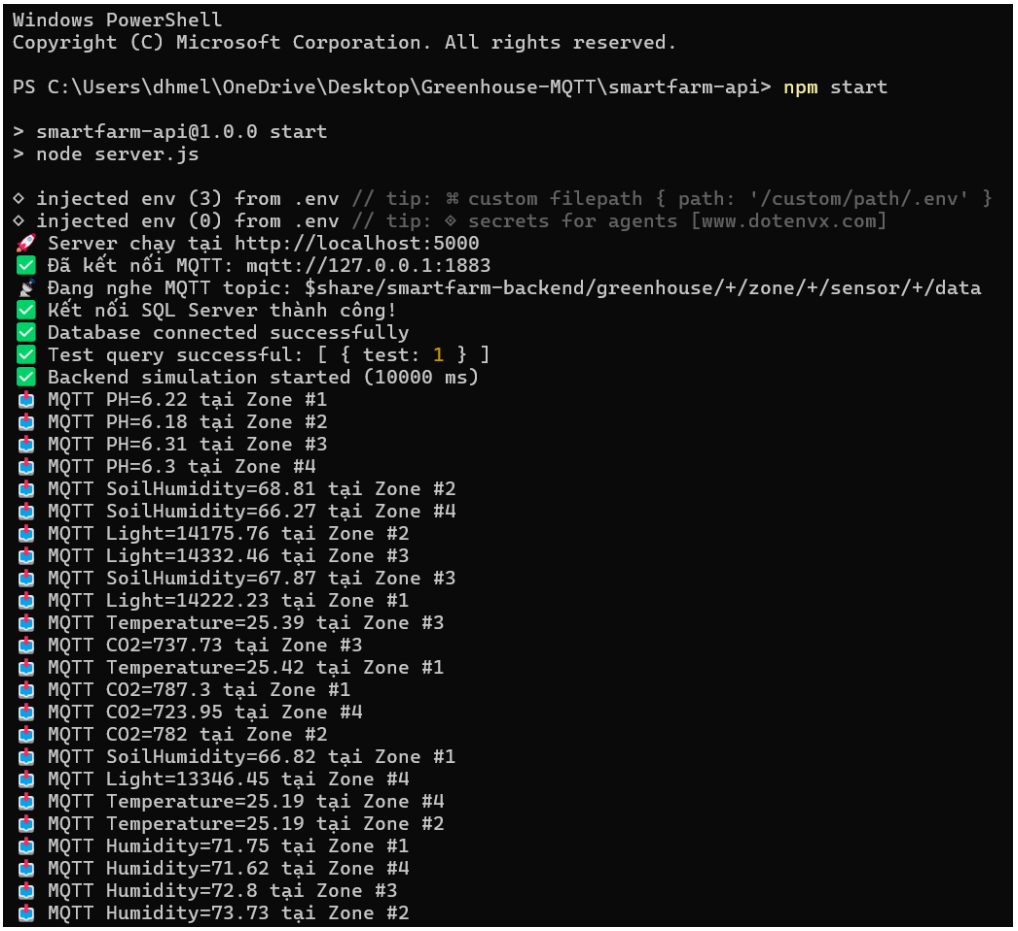
Hệ thống Cảnh báo & Nhật ký kiểm toán: Quản lý quy trình xử lý lỗi theo cơ chế leo thang (Escalation) và lưu vết toàn bộ hành vi hệ thống trong Audit Log.



Hình 3.7. Trung tâm quản lý và leo thang cảnh báo



Hình 3.8. Màn hình Nhật ký hoạt động Audit Log



Hình 3.9. Màn hình Terminal kiểm tra kết nối MQTT Broker & Simulator Backend

3.5. Kiểm thử hệ thống

3.5.1. Kiểm thử xác thực và phân quyền

Mục tiêu của nhóm kiểm thử này là xác minh chức năng đăng nhập và phân quyền người dùng hoạt động đúng theo yêu cầu thiết kế.

Bảng 3.33. Kiểm thử xác thực và phân quyền

Mã TC	Nội dung kiểm thử	Kết quả mong đợi	Kết quả
TC01	Đăng nhập với tài khoản hợp lệ	Đăng nhập thành công và chuyển đến Dashboard	Đạt
TC02	Đăng nhập sai mật khẩu	Hiển thị thông báo lỗi	Đạt
TC03	Kiểm tra phân quyền người dùng	Chỉ hiển thị các chức năng phù hợp với vai trò	Đạt

Nhận xét: Chức năng xác thực và phân quyền hoạt động ổn định, chỉ cho phép người dùng truy cập các chức năng tương ứng với quyền được cấp.

3.5.2. Kiểm thử quản lý dữ liệu

Nhóm kiểm thử này đánh giá các chức năng quản lý dữ liệu chính của hệ thống như quản lý trang trại, nhà kính, vùng trồng và công thức sinh trưởng.

Bảng 3.34. Kiểm thử quản lý dữ liệu

Mã TC	**Nội dung kiểm thử**	**Kết quả mong đợi**	**Kết quả**
TC04	Thêm, sửa, xóa Farm/Greenhouse/Zone	Dữ liệu cập nhật chính xác	Đạt
TC05	Tạo và áp dụng Recipe cho Zone	Recipe được lưu và áp dụng	Đạt

Nhận xét: Các thao tác được thực hiện thành công, dữ liệu được đồng bộ chính xác với cơ sở dữ liệu SQL Server.

3.5.3. Kiểm thử thu thập và xử lý dữ liệu cảm biến

Phiên bản hiện tại của hệ thống sử dụng **Sensor Simulator** tích hợp trong backend để mô phỏng dữ liệu cảm biến theo chu kỳ. Dữ liệu được xử lý theo cùng luồng nghiệp vụ như khi kết nối với thiết bị IoT thực tế.

Bảng 3.35. Kiểm thử thu thập và xử lý dữ liệu cảm biến

Mã TC	**Nội dung kiểm thử**	**Kết quả mong đợi**	**Kết quả**
TC06	Simulator sinh dữ liệu cảm biến	Dữ liệu được tạo theo chu kỳ	Đạt
TC07	Lưu dữ liệu vào SensorData	Bản ghi được lưu thành công	Đạt
TC08	Dashboard cập nhật dữ liệu mới	Giá trị cảm biến hiển thị chính xác	Đạt

Nhận xét: Luồng xử lý dữ liệu từ Simulator đến Dashboard hoạt động ổn định, đáp ứng yêu cầu giám sát gần thời gian thực.

3.5.4. Kiểm thử điều khiển tự động và cảnh báo

Nhóm kiểm thử này nhằm đánh giá khả năng so sánh dữ liệu cảm biến với ngưỡng (Threshold), tự động điều khiển thiết bị và sinh cảnh báo.

Bảng 3.36. Kiểm thử điều khiển tự động và cảnh báo

Mã TC	**Nội dung kiểm thử**	**Kết quả mong đợi**	**Kết quả**
TC09	Giá trị cảm biến vượt ngưỡng	Thiết bị AUTO được kích hoạt	Đạt
TC10	Giá trị trở về bình thường	Thiết bị tự động dừng	Đạt

TC11	Điều kiện vượt ngưỡng	Hệ thống tạo AlertLog	Đạt

Nhận xét: Cơ chế điều khiển tự động và cảnh báo hoạt động đúng theo thiết lập Recipe.

3.5.5. Kiểm thử ghi nhật ký hệ thống

Hệ thống sử dụng **Audit Log** để ghi nhận các thao tác của người dùng và các hành động tự động của hệ thống nhằm phục vụ việc theo dõi và truy vết.

Bảng 3.37. Kiểm thử ghi nhật ký hệ thống

Mã TC	**Nội dung kiểm thử**	**Kết quả mong đợi**	**Kết quả**
TC12	Thực hiện thao tác quản lý dữ liệu	Audit Log được tạo	Đạt
TC13	Điều khiển thiết bị tự động	Audit Log được ghi nhận	Đạt

Nhận xét: Các thao tác quan trọng đều được lưu vào Audit Log, giúp hỗ trợ kiểm tra lịch sử hoạt động và nâng cao khả năng quản lý hệ thống.

3.5.6. Đánh giá chung kết quả kiểm thử

Quá trình kiểm thử được thực hiện trên các chức năng cốt lõi của hệ thống, bao gồm xác thực người dùng, quản lý dữ liệu, mô phỏng dữ liệu cảm biến, điều khiển tự động, cảnh báo và ghi nhật ký hoạt động. Kết quả cho thấy tất cả các trường hợp kiểm thử đều đạt yêu cầu, hệ thống hoạt động ổn định và đáp ứng các chức năng đã đề ra trong quá trình phân tích và thiết kế.

3.6. Kết luận chương 3

Chương 3 đã hoàn thành việc trình bày toàn diện quá trình hiện thực hóa và triển khai phần mềm quản lý nhà kính trồng hoa. Các module từ kiến trúc phân lớp, cấu hình cơ sở dữ liệu SQL Server (đầy đủ 13 bảng DDL và 9 Views), bộ dịch vụ xử lý Backend (Express.js, MQTT.js) đến giao diện Web tương tác đều được xây dựng hoàn chỉnh và liên kết chặt chẽ. Bộ 10 kịch bản kiểm thử thực tế đã chứng minh hệ thống hoạt động ổn định, độ trễ thấp và hoàn toàn đáp ứng các mục tiêu thiết kế ban đầu. Không chỉ vậy, quá trình kiểm thử và đánh giá hệ thống cũng đã được thực hiện. Kết quả cho thấy các chức năng được kiểm thử đều hoạt động đúng theo yêu cầu thiết kế, dữ liệu được xử lý và lưu trữ chính xác, đồng thời cơ chế cảnh báo và điều khiển thiết bị tự động hoạt động theo các ngưỡng đã cấu hình. Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế như chưa tích hợp trực tiếp với thiết bị IoT thực tế và chưa có ứng dụng dành cho thiết bị di động. Đây là cơ sở để đề xuất các hướng phát triển tiếp theo nhằm mở rộng khả năng ứng dụng của hệ thống.

KẾT LUẬN

Kết quả đạt được

Sau quá trình phân tích, thiết kế, xây dựng và kiểm thử, nhóm đã hoàn thành hệ thống quản lý nhà kính trồng hoa theo mô hình ứng dụng Web ba lớp (Presentation – Business Logic – Database). Hệ thống đáp ứng các yêu cầu chức năng đã đề ra, bao gồm quản lý trang trại, nhà kính, vùng trồng, thiết bị, công thức sinh trưởng và dữ liệu môi trường.

Hệ thống hỗ trợ thu thập và xử lý dữ liệu cảm biến thông qua **Sensor Simulator**, cho phép mô phỏng hoạt động của các cảm biến trong môi trường nhà kính. Dữ liệu sau khi được xử lý sẽ được lưu trữ trong cơ sở dữ liệu SQL Server, đồng thời phục vụ việc giám sát, cảnh báo và điều khiển thiết bị ở chế độ tự động.

Kiến trúc phân tầng giúp tách biệt giao diện, xử lý nghiệp vụ và cơ sở dữ liệu, góp phần nâng cao khả năng bảo trì, mở rộng và phát triển hệ thống trong các giai đoạn tiếp theo.

Ưu điểm của hệ thống

Qua quá trình xây dựng và kiểm thử, nhận thấy hệ thống có những ưu điểm:

- Kiến trúc phân tầng giúp mã nguồn có tính mô-đun cao, rõ ràng, dễ bảo trì và mở rộng.
- Giao diện Web trực quan, kết hợp cơ chế tự động hóa điều khiển thiết bị theo ngưỡng cấu hình và công thức sinh trưởng.
- Thiết kế hướng mở, thuận lợi cho việc tích hợp thiết bị IoT thực tế trong tương lai.

Hạn chế của hệ thống

Hệ thống còn những điểm hạn chế:

- Chưa triển khai kết nối phần cứng với thiết bị IoT và cảm biến thực tế (dữ liệu hiện tại được mô phỏng qua *Sensor Simulator*).
- Chưa tích hợp các cơ chế bảo mật nâng cao như HTTPS, JWT, mã hóa mật khẩu (bcrypt/Argon2) hay bảo mật MQTT bằng TLS/SSL.
- Chưa hỗ trợ triển khai trên môi trường điện toán đám mây hoặc hệ thống phân tán và chưa có ứng dụng di động.

Hướng phát triển trong tương lai

Dựa vào tệp khách hàng mà hệ thống nhắm tới, cũng như xu hướng công nghệ, đề xuất hướng phát triển trong tương lai:

- Tích hợp AI để dự báo sinh trưởng và năng suất cây trồng.

- Phát triển ứng dụng trên nền tảng Android và iOS, đồng thời bổ sung dữ liệu dự báo thời tiết.
- Hỗ trợ đa dạng hóa loại cây trồng, tăng cường bảo mật hệ thống và triển khai trên nền tảng điện toán đám mây.

Tổng kết chung

Quá trình thực hiện đề tài "Xây dựng hệ thống quản lý nhà kính trồng hoa ứng dụng công nghệ IoT" không chỉ giúp nhóm giải quyết các bài toán thực tế trong nông nghiệp thông minh mà còn củng cố vững chắc kiến thức về kỹ thuật phần mềm, cơ sở dữ liệu, lập trình Web và hệ thống IoT. Mặc dù vẫn còn một số hạn chế cần khắc phục, đề tài đã hoàn thành các mục tiêu đề ra, qua đó tích lũy cho nhóm kinh nghiệm quý báu trong phân tích yêu cầu, thiết kế hệ thống và quản lý dự án phần mềm cho chặng đường học tập và nghiên cứu tiếp theo.

TÀI LIỆU THAM KHẢO

[1] Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2015.

[2] Roger S. Pressman, Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 9th Edition, McGraw-Hill Education, 2019.

[3] MQTT Version 5.0 Specification, OASIS Standard, 2019.

[4] Node.js Documentation: https://nodejs.org

[5] Express.js Documentation: https://expressjs.com

[6] MQTT.js Documentation: https://mqtt.js.org

[7] Microsoft SQL Server Documentation: https://learn.microsoft.com/sql

[8] GitHub Documentation: https://docs.github.com

[9] HiveMQ MQTT Essentials: https://www.hivemq.com/mqtt

[10] Các tài liệu báo cáo môn Kỹ thuật phần mềm, Trường Đại học Bách khoa Hà Nội.

[11] Tài liệu tham khảo về Internet of Things (IoT) và nông nghiệp thông minh từ các nguồn học thuật và tài liệu kỹ thuật công khai.